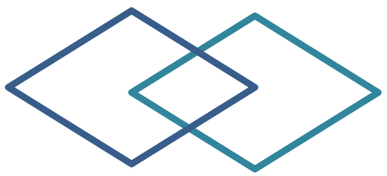


第3章

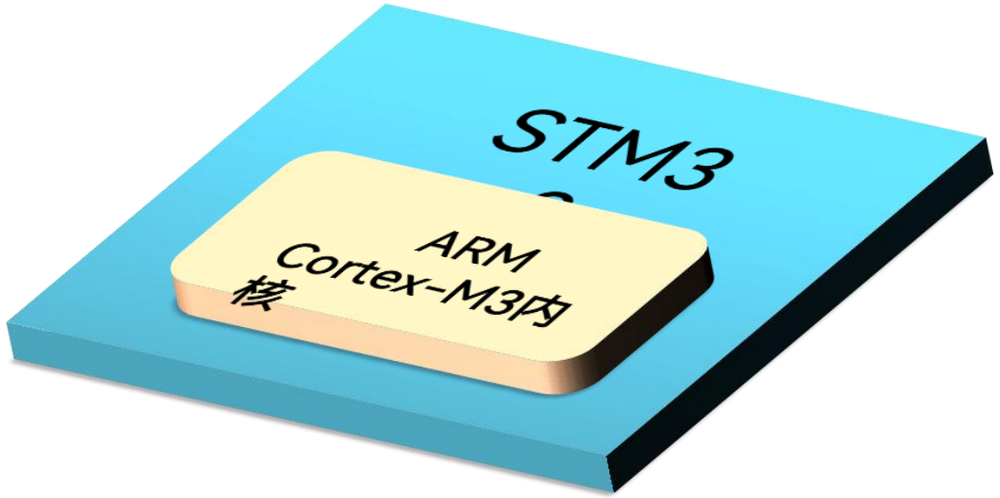
STM32微控制器及开发环境搭建



本章知识与能力要求

- 了解STM32的性能参数，能够进行基本的器件选型；
- 能够从ST官网上下载相应芯片的数据手册、用户手册等官方文档；
- 掌握和理解STM32的三种开发方式，即寄存器方式、标准外设库方式和HAL库方式；
- 能够独立搭建STM32的开发环境，掌握keil 的调试方法。

3.1 STM32介绍



S T — 意法半导体公司

M — Microelectronics的缩写，表示微控制器

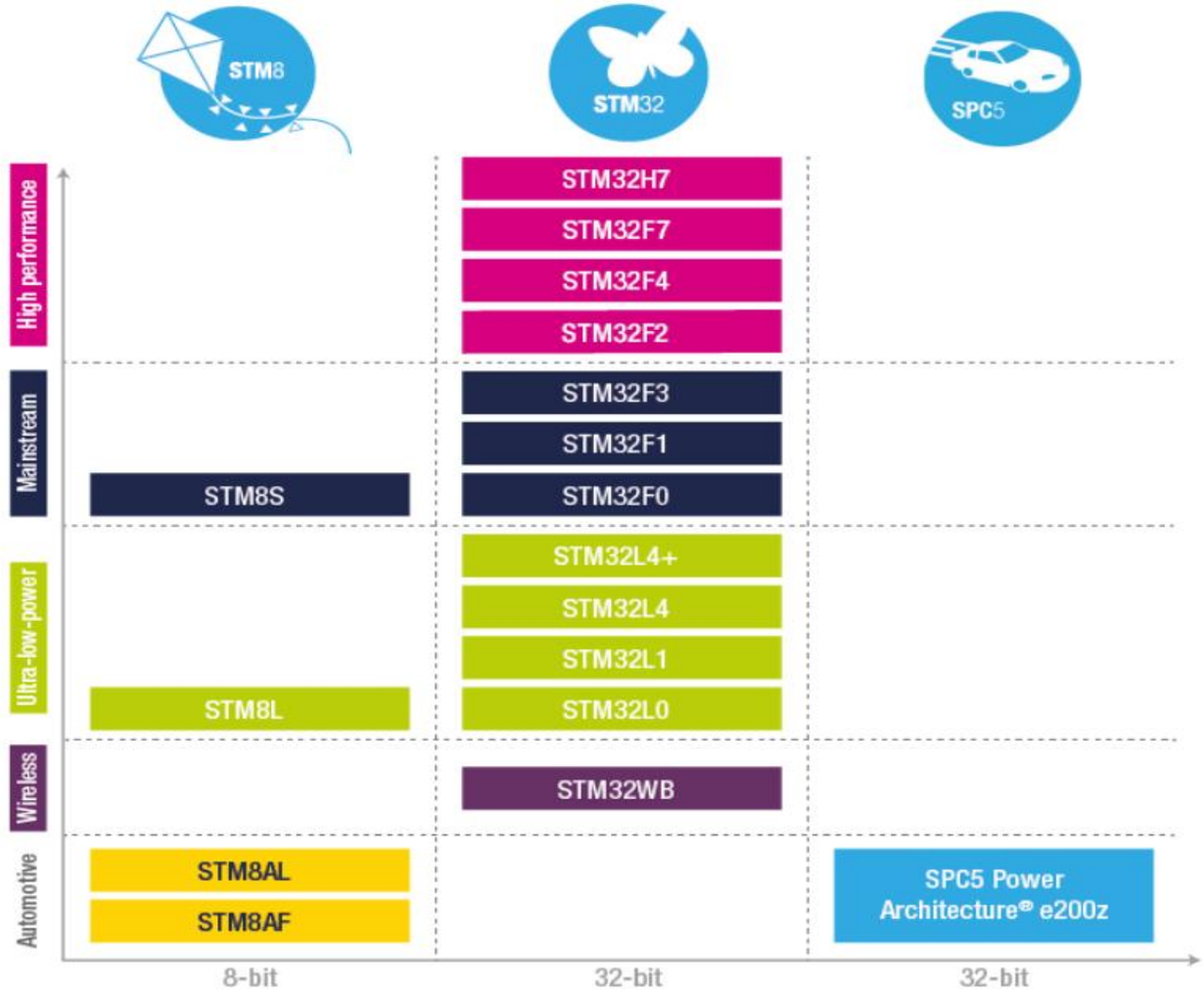
3 2 — 32位的意思，表示这是一个32位的微处理器芯片

3.1 STM32介绍

STM8主打低功耗，主要针对8位微控制器市场：

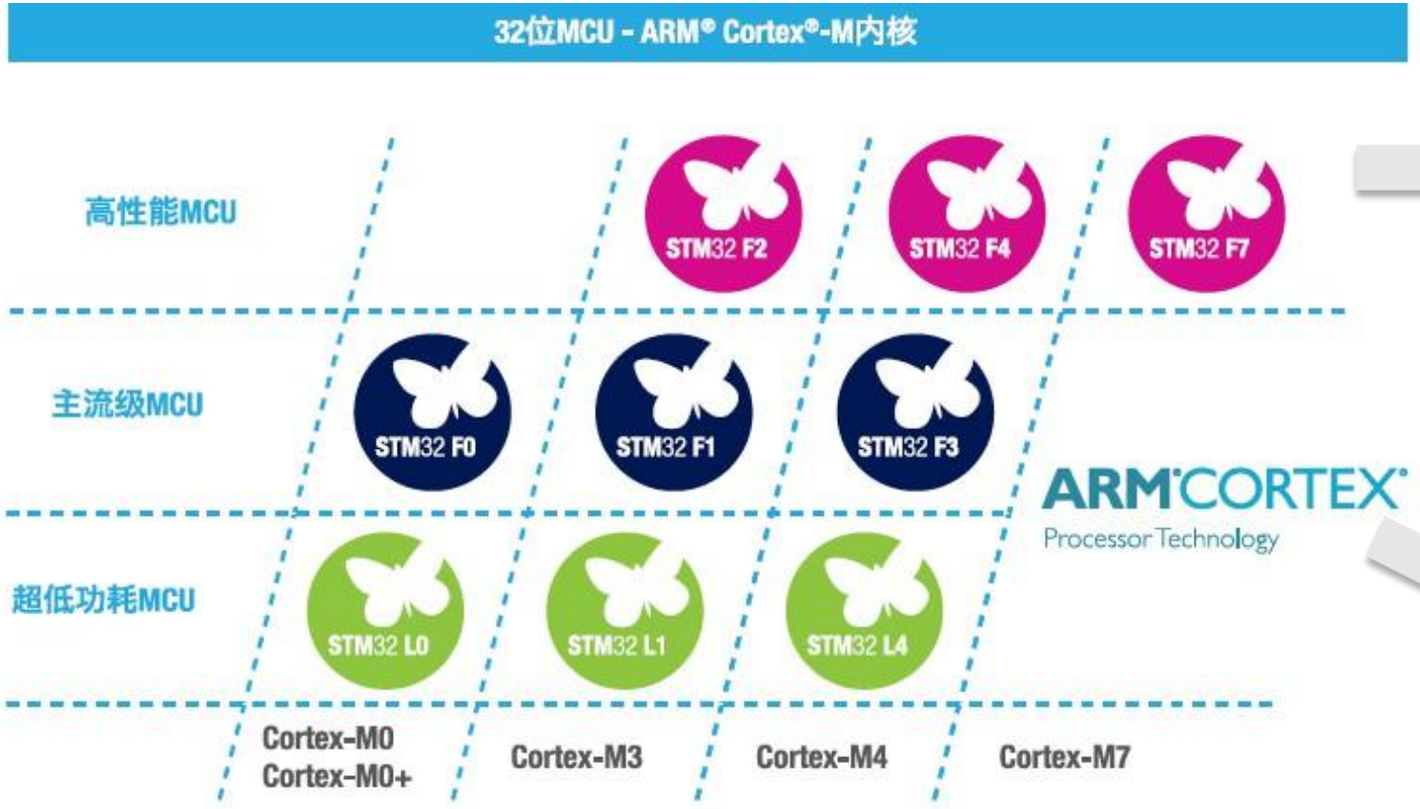
- STM8A(汽车用MCU)
- STM8S(主流MCU标准系列)
- STM8L(超低功耗MCU) 三个系列

基于STM8内核，采用3级流水线的哈佛结构。



ST公司微处理器系列产品

3.1 STM32介绍



STM32系列MCU

STM32F2、STM32F4、STM32F7为高性能MCU

STM32F4系列基于ARM Cortex-M4内核，主频高达180MHz，具有DSP和FPU等功能。

STM32F1系列是ST公司主推的STM32系列MCU，基于ARM Cortex-M3内核，主频

3.1 STM32介绍

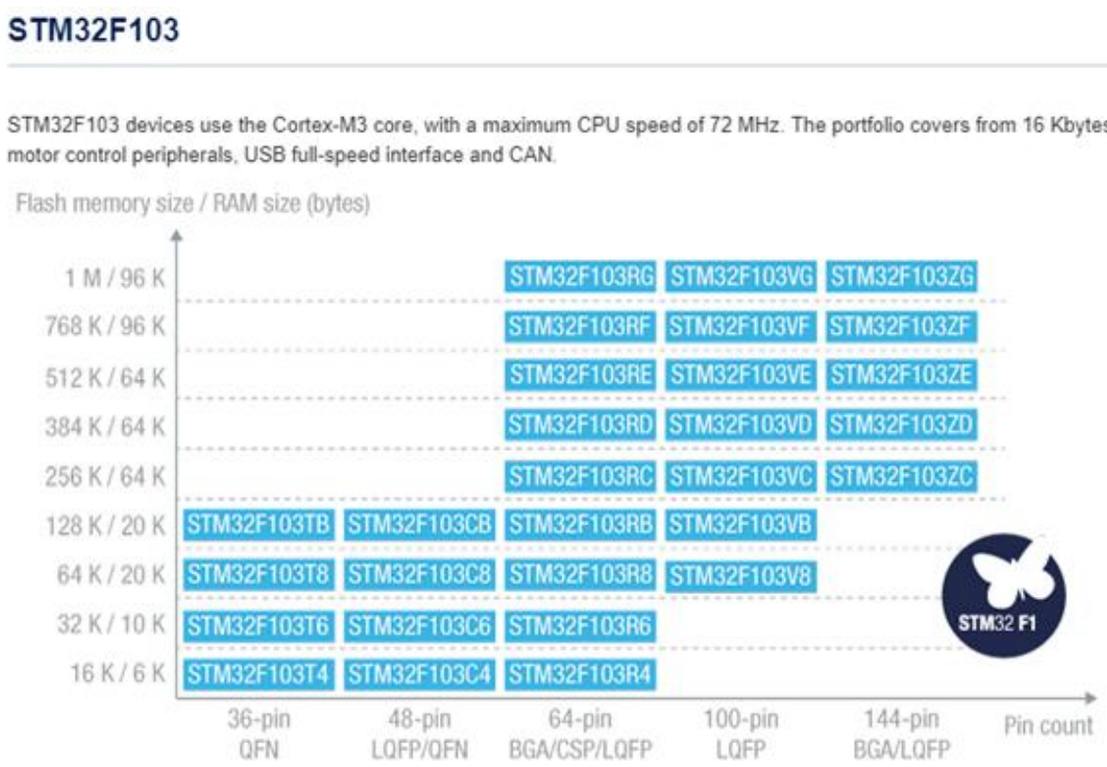
STM32F1产品系列

STM32F100	主频为24MHz，具有电机控制和消费电子控制功能；
STM32F101	基本型，主频为36MHz，具有高达1MB的Flash；
STM32F102	USB基本型，主频为48MHz，具有USB FS device接口；
STM32F103	增强型，主频为72MHz，具有高达1MB的Flash、电机控制、USB和CAN；
STM32F105/107	互联型，主频为72MHz，具有以太网MAC、CAN和USB 2.0 OTG。

STM32F1系列MCU是ST针对工业、医疗和消费类市场的应用需求而设计的低功耗、高性价比的产品，该系列从功能上又可分为5个子系列，其引脚、外设和软件均相互兼容

3.1 STM32介绍

STM32F1系列产品中，各个子系列按闪存容量的大小又分为小容量、中容量、大容量产品。



STM32F103系列产品

低密度（LD）：闪存容量为16~32KB

中密度（MD）：闪存容量为64~128KB

高密度（HD）：闪存容量为256~512KB

超密度（XL）：闪存容量为768KB~1MB

3.1 STM32介绍

以STM32F103ZET6为例说明芯片命名规范，该芯片型号由七部分组成



STM32芯片命名规范

序号	符号	说明
1	STM32	ARM Cortex-M3内核的32位微控制器
2	F	芯片的系列
3	103	增强型系列
4	Z	芯片有144管脚
5	E	512K字节的Flash
6	T	采用LQFP封装
7	6	工作温度范围为-40--+85℃

3.1 STM32介绍

半导体芯片常见的封装方式

(Plastic) Dual In-line Package



DIP——(塑料)双列直插式封装，适用于中小规模集成电路，引脚数一般不超过100个

Small Outline Package



SOP——小外型表面贴片封装，广泛应用于10-40个引脚的芯片

Plastic Quad Flat Package



PQFP——塑料方形扁平封装，管脚很细，一般应用于大规模或超大型集成电路，引脚数一般在100个以上

Low-profile Quad Flat Package



LQFP——薄型QFP指封装，本体厚度为1.4mm的QFP

3.1 STM32介绍

半导体芯片常见的封装方式

（Plastic）Ball Grid Array



BGA——(塑料)球栅阵列封装，Intel系列的80486和Pentium等CPU多采用这种封装方式

Pin Grid Array Package



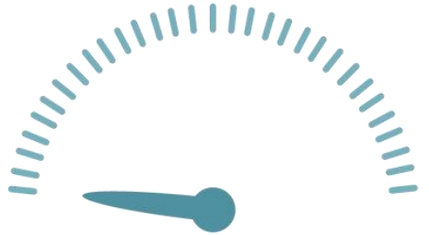
PGA——插针网格阵列封装，有专门的PGA插座，插拔方便，常用于CPU的封装

Plastic J-Leaded Chip Carrie



PLCC——塑封方形引脚插入式封装，外形尺寸比DIP封装小得多，必须采用SMT(表面贴装技术)进行焊接

3.1 STM32介绍



- 用户选型参考资料
  - 选型手册
  - 数据手册（Datasheet）
    - 内容包括：产品的基本配置(如内置Flash和RAM的容量、外设模块的种类和数量等）、引脚数量和分配、电气特性、封装信息和定购代码等
  - 参考手册（Reference manual）
    - 对芯片每一个外设的功能进行了详细说明，并对寄存器进行了详细描述

3.1 STM32介绍

STM32芯片数据手册中对具体的引脚名称、引脚类型、引脚复用功能及I/O结构等进行了详细说明

STM32数据手册中有关引脚的定义在Pinouts and pin description部分

Table 5. High-density STM32F103xC/D/E pin definitions

1 Pins						2	3	4	5	Alternate functions(4)	
LFBGA144	LFBGA100	WLCSP64	LQFP64	LQFP100	LQFP144	Pin name	Type(1)	I / O Level(2)	Main function(3) (after reset)	6 Default	7 Remap
A3	A3	-	-	1	1	PE2	I/O	FT	PE2	TRACECK/ FSMC_A23	-
A2	B3	-	-	2	2	PE3	I/O	FT	PE3	TRACED0/FSMC_A19	-
B2	C3	-	-	3	3	PE4	I/O	FT	PE4	TRACED1/FSMC_A20	-
B3	D3	-	-	4	4	PE5	I/O	FT	PE5	TRACED2/FSMC_A21	-
B4	E3	-	-	5	5	PE6	I/O	FT	PE6	TRACED3/FSMC_A22	-
C2	B2	C6	1	6	6	V_{BAT}	S	-	V_{BAT}	-	-
A1	A2	C8	2	7	7	PC13-TAMPER-RTC(5)	I/O	-	PC13(6)	TAMPER-RTC	-
B1	A1	B8	3	8	8	PC14-OSC32_IN(5)	I/O	-	PC14(6)	OSC32_IN	-
C1	B1	B7	4	9	9	PC15-OSC32_OUT(5)	I/O	-	PC15(6)	OSC32_OUT	-
C3	-	-	-	-	10	PF0	I/O	FT	PF0	FSMC_A0	-
C4	-	-	-	-	11	PF1	I/O	FT	PF1	FSMC_A1	-
D4	-	-	-	-	12	PF2	I/O	FT	PF2	FSMC_A2	-
E2	-	-	-	-	13	PF3	I/O	FT	PF3	FSMC_A3	-
E3	-	-	-	-	14	PF4	I/O	FT	PF4	FSMC_A4	-
E4	-	-	-	-	15	PF5	I/O	FT	PF5	FSMC_A5	-

STM32F103xx引脚定义

3.1 STM32介绍

STM32F103xx引脚功能说明

编号	功能	说明
1	引脚序号	有 6 种封装型号：BGA144、BGA100、WLCSP64、LQFP64、LQFP100、LQFP144。其中用阿拉伯数字的表示是 LQFP 封装，英文字母开头的表示 BGA 封装
2	引脚名称	指复位状态下的引脚名称
3	引脚类型	S 表示电源引脚
		I 表示输入引脚
		I/O 为输入/输出引脚
4	I/O 结构	FT：表示兼容 5V
		TTa：只支持 3.3V
		B：表示 BOOT 引脚
		RST：表示复位引脚，内部带有弱上拉
5	主功能	每个引脚复位后的功能
6	默认复用功能	指 I/O 的默认复用功能
7	重映射功能	I/O 引脚除了具有默认的复用功能之外，还可以通过重映射的方式将 I/O 引脚映射到其他的 I/O，增加 I/O 口功能的多样性和灵活性

3.2 STM32微控制器开发模式



寄存器开发方式



标准外设库开发方式

将底层寄存器的操作进行了统一封装，包括所有标准器件外设的驱动器，采用C语言实现，开发人员只需要熟悉并调用相应的应用程序编程接口函数，即可实现对相关外设的驱动操作



HAL库开发方式

与STM32CubeMX软件(配置STM32代码的工具）配套使用的，它把底层硬件相关的内容封装起来并及进行抽象，通过图形化的操作方式自动生成相关外设的驱动代码，简单易用

3.2.1 寄存器开发模式

8.2.5 端口位设置/清除寄存器(GPIOx_BSRR) (x=A..E)

地址偏移：0x10

复位值：0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

位31:16	**BRy**: 清除端口x的位y (y = 0…15) (Port x Reset bit y) 这些位只能写入并只能以字(16位)的形式操作。 0：对对应的ODRy位不产生影响 1：清除对应的ODRy位为0 注：如果同时设置了BSy和BRy的对应位，BSy位起作用。
位15:0	**BSy**: 设置端口x的位y (y = 0…15) (Port x Set bit y) 这些位只能写入并只能以字(16位)的形式操作。 0：对对应的ODRy位不产生影响 1：设置对应的ODRy位为1

根据STM32定义好的寄存器，查阅官方用户手册对组成寄存器各bit位的操作说明，配置相应寄存器的各bit位，从而直接进行寄存器编程，完成相应功能。

如：

```
GPIOB->BSRR=0x0001;
```

BSRR为GPIOB(端口B)的位设置/清除寄存器

3.2.1 寄存器开发模式

将GPIOB端口的BS0设置为“1”，设置BS0为“1”，即设置对应的ODR0为“1”，即0x0001对应PB0引脚。

8.2.4 端口输出数据寄存器(GPIOx_ODR) (x=A..E)

地址偏移：0Ch

复位值：0x0000 0000

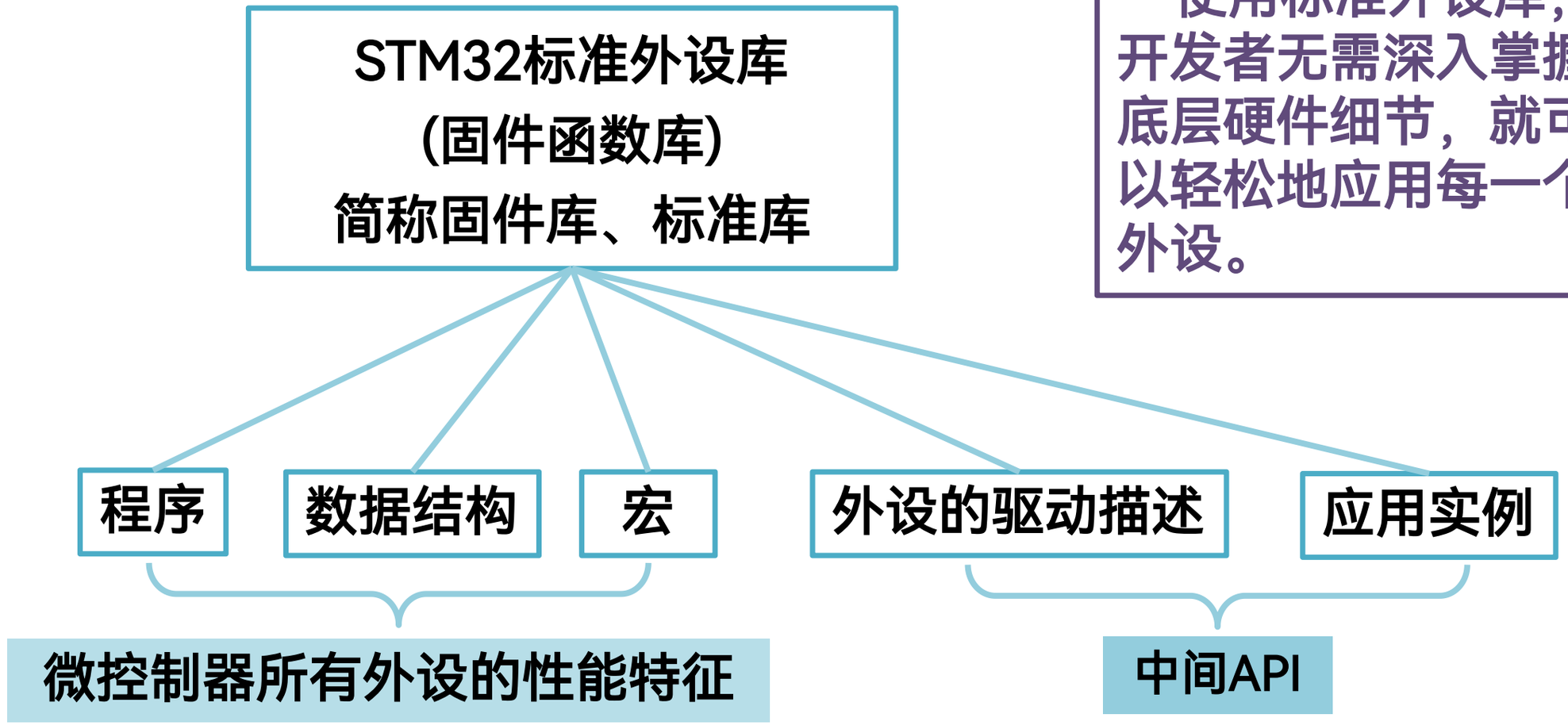
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位31:16	保留，始终读为0。
位15:0	**ODRy[15:0]**：端口输出数据(y = 0…15) (Port output data) 这些位可读可写并只能以字(16位)的形式操作。 注：对GPIOx_BSRR(x = A…E)，可以分别地对各个ODR位进行独立的设置/清除。

需要深入学习和掌握某个具体芯片，对开发人员的要求较高，学习过程较长

3.2.2 标准外设库开发模式

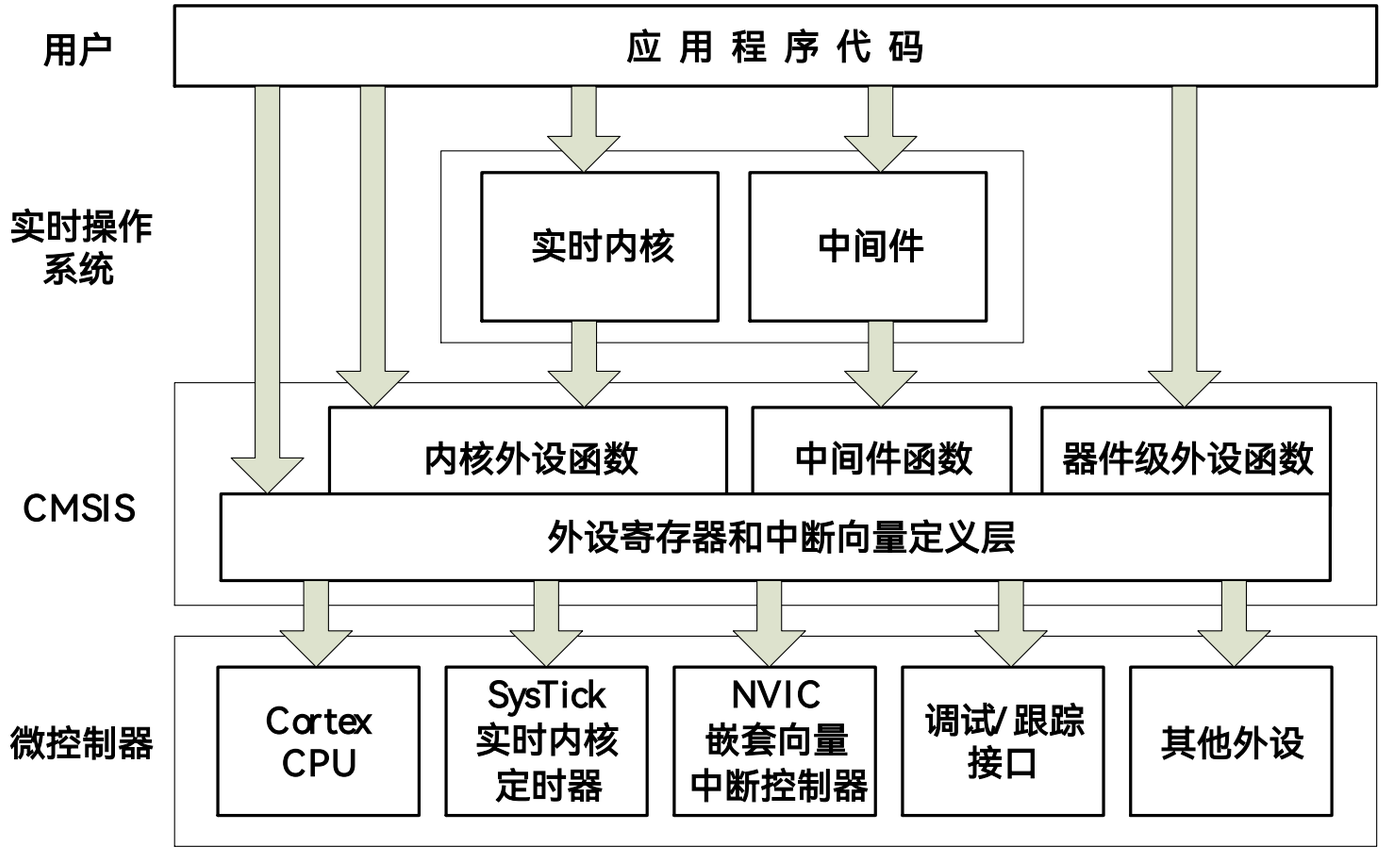


使用标准外设库，开发者无需深入掌握底层硬件细节，就可以轻松地应用每一个外设。

3.2.2 标准外设库开发模式

为了解决不同芯片厂商生产的基于Cortex内核的微处理器在软件上的兼容问题，ARM公司与众多芯片和软件厂商共同制定了CMSIS标准（Cortex Microcontroller Software Interface Standard，Cortex微控制器软件接口标准），意在将所有Cortex芯片厂商产品的软件接口标准化。

CMSIS标准的软件架构图



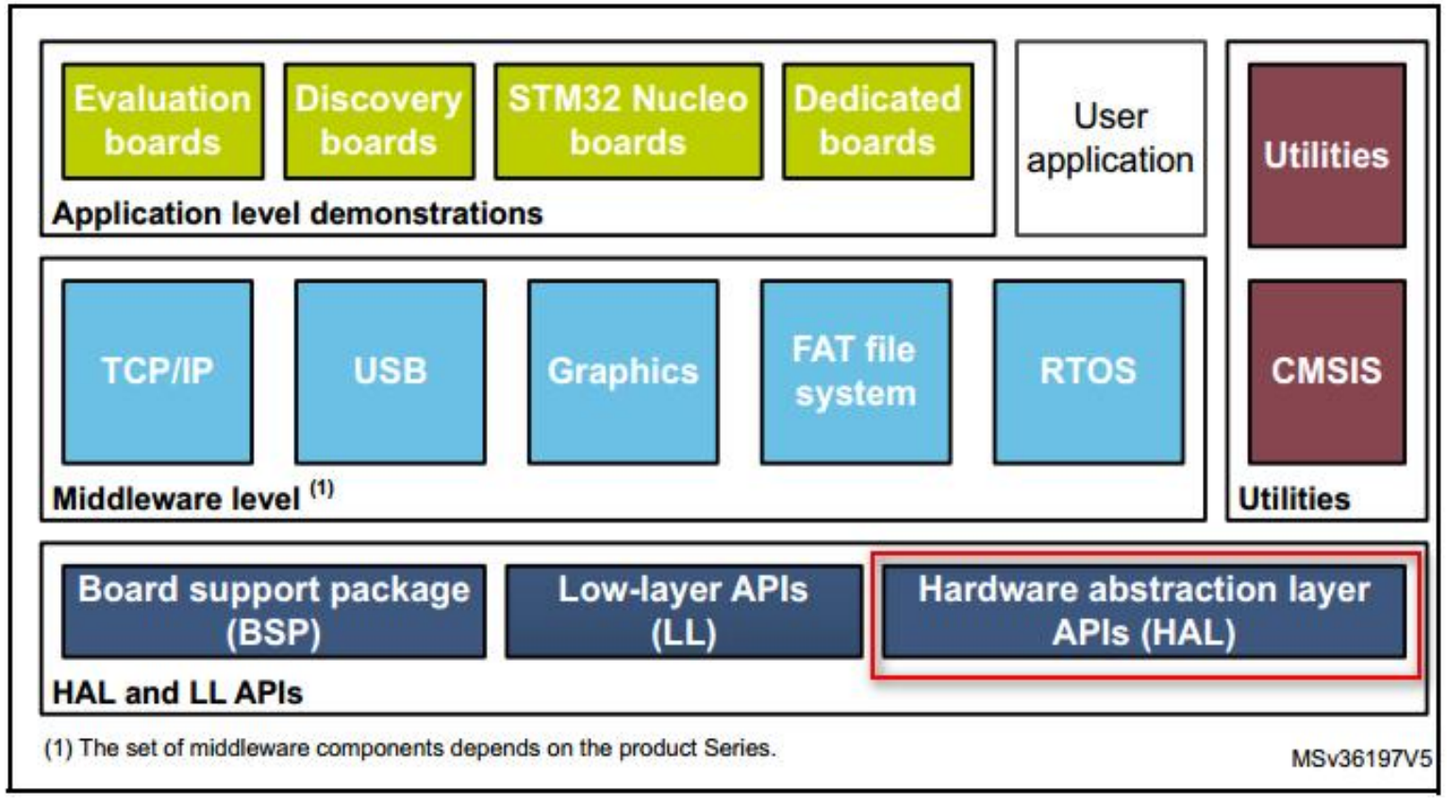
标准外设库的架构遵循CMSIS标准

3.2.3 HAL库开发模式

HAL（硬件抽象层）是位于内核与硬件电路之间的接口层，其目的是将硬件抽象化。

文件构架

- 用户编写程序
- CMSIS驱动
- HAL驱动



HAL库框架图

3.2.3 HAL库开发模式

基于HAL库的开发方式，其实质是仍然是通过操作底层寄存器来实现相应功能，只是将寄存器操作进行了封装，以函数形式提供给开发人员使用。

如HAL库的GPIO写操作函数HAL_GPIO_WritePin()

```
void HAL_GPIO_WritePin(GPIO_TypeDef* GPIOx,uint16_t GPIO_Pin, GPIO_PinState PinState)
{
   if(PinState != GPIO_PIN_RESET)
   {
       GPIOx->BSRR = GPIO_Pin;
    }
  else
  {
    GPIOx->BSRR = (uint32_t)GPIO_Pin << 16;
  }
}
```

3.2.3 HAL库开发模式

STM32CubeMX为图形化初始化代码配置工具/软件，基于HAL库进行配置。

在图形化界面下对芯片引脚、时钟及相应外设做初始化配置并最终生成C语言代码。

HAL库开发模式需要用到两个软件：STM32CubeMX用于外设功能模块的参数配置、生成相应编译器的工程代码；编译器用于应用程序的开发。



MCU选型



外设模块功能配置

生成工程代码



Keil编写应用程序

3.3 开发平台的搭建



3.3.1 嵌入式开发环境



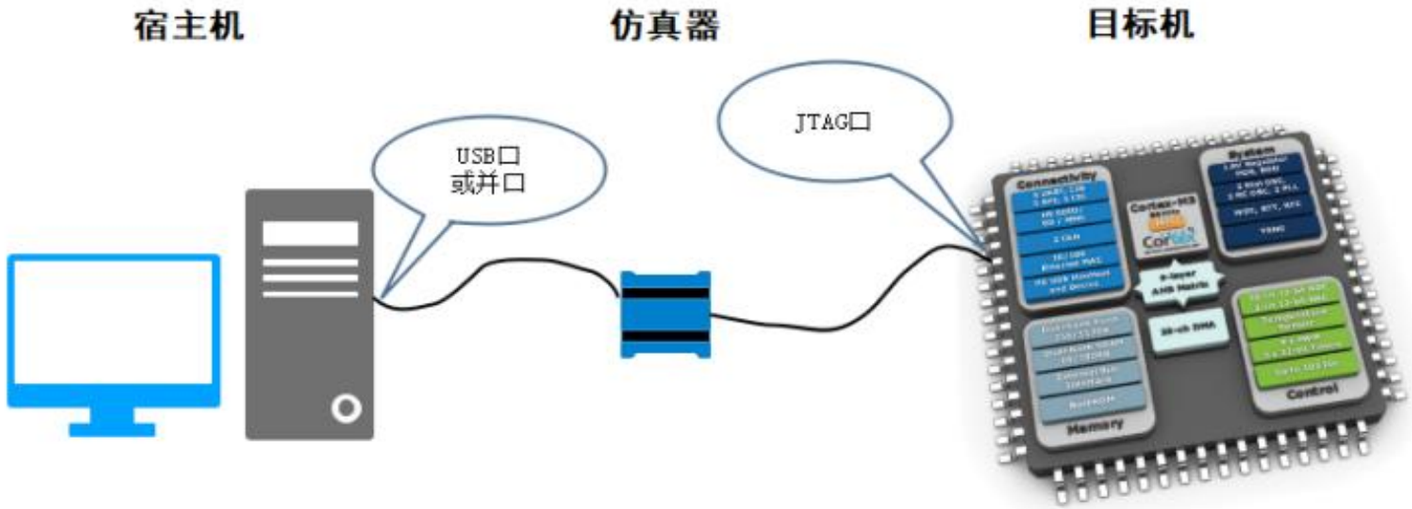
3.3.2 集成开发环境Keil



3.3.3 下载和安装packs包

3.3.1 嵌入式开发环境

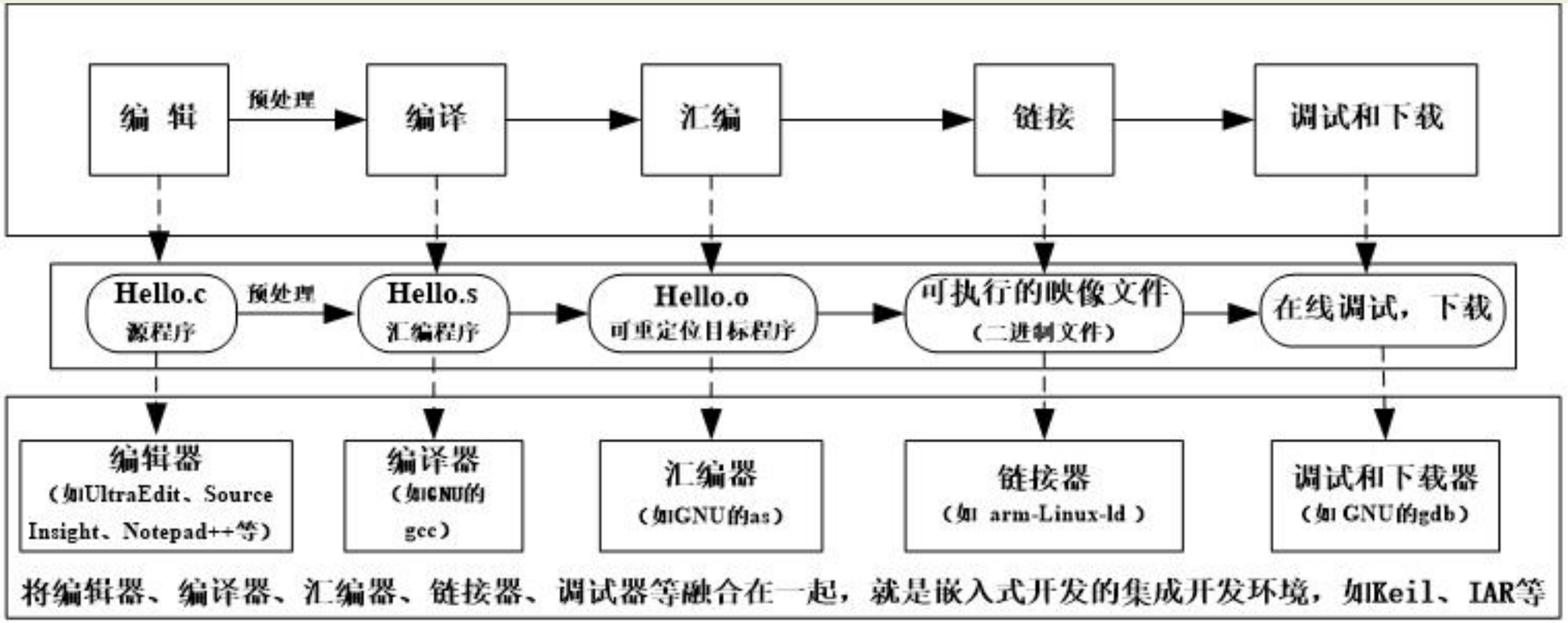
- 下载工具将宿主机和目标机连接起来。



- 宿主机：常指通用PC机，在通用PC机上安装软件开发工具（如keil ），进行代码编辑、程序编译和链接，最后生成目标机运行所需的可执行文件。
- 目标机：嵌入式系统运行的载体，如开发板，不能直接进行嵌入式系统的开发，由宿主机将可执行二进制代码文件下载或烧写进目标机中，再由目标机执行的方式。

3.3.1 嵌入式开发环境

嵌入式系统软件开发一般分为四个阶段：代码编辑、程序编译、链接、调试和下载，每个阶段都需要不同的开发工具来完成，这些开发工具都需要运行在宿主机上。



3.3.1 嵌入式开发环境

STM32常用的调试和下载工具——J-Link



- J-Link是德国SEGGER公司推出的基于JTAG的仿真器；
- J-Link是一个通用的开发工具，配合IAR EWAR，ADS，Keil，RealView等集成开发环境；
- 支持所有ARM7/ARM9/ARM11、Cortex M0/M1/M3/M4、Cortex A5/A8/A9等内核芯片的仿真；
- 与IAR、Keil等编译环境无缝连接。

在下载速度、效率、功能等方面具有良好的特性，是目前ARM开发中最好最实用的JTAG仿真工具。

3.3.1 嵌入式开发环境

STM32常用的调试和下载工具
——ST-Link/V2和ISP下载器



- ST-Link/V2是专门针对意法半导体STM8和STM32系列芯片的仿真器；
- 采用USB2.0接口进行SWIM/JTAG/SWD下载；
- 下载速度快；
- 支持全速运行、单步调试、断点调试等各种调试方法，可查看IO状态，变量数据等。

ISP下载器是串口下载电路，通过上位机仅需一根USB线即可实现一键ISP下载，缺点是只能烧写下载程序，不能进行在线仿真调试。

3.3.1 嵌入式开发环境

嵌入式集成开发环境（Integrated Development Environment, IDE）

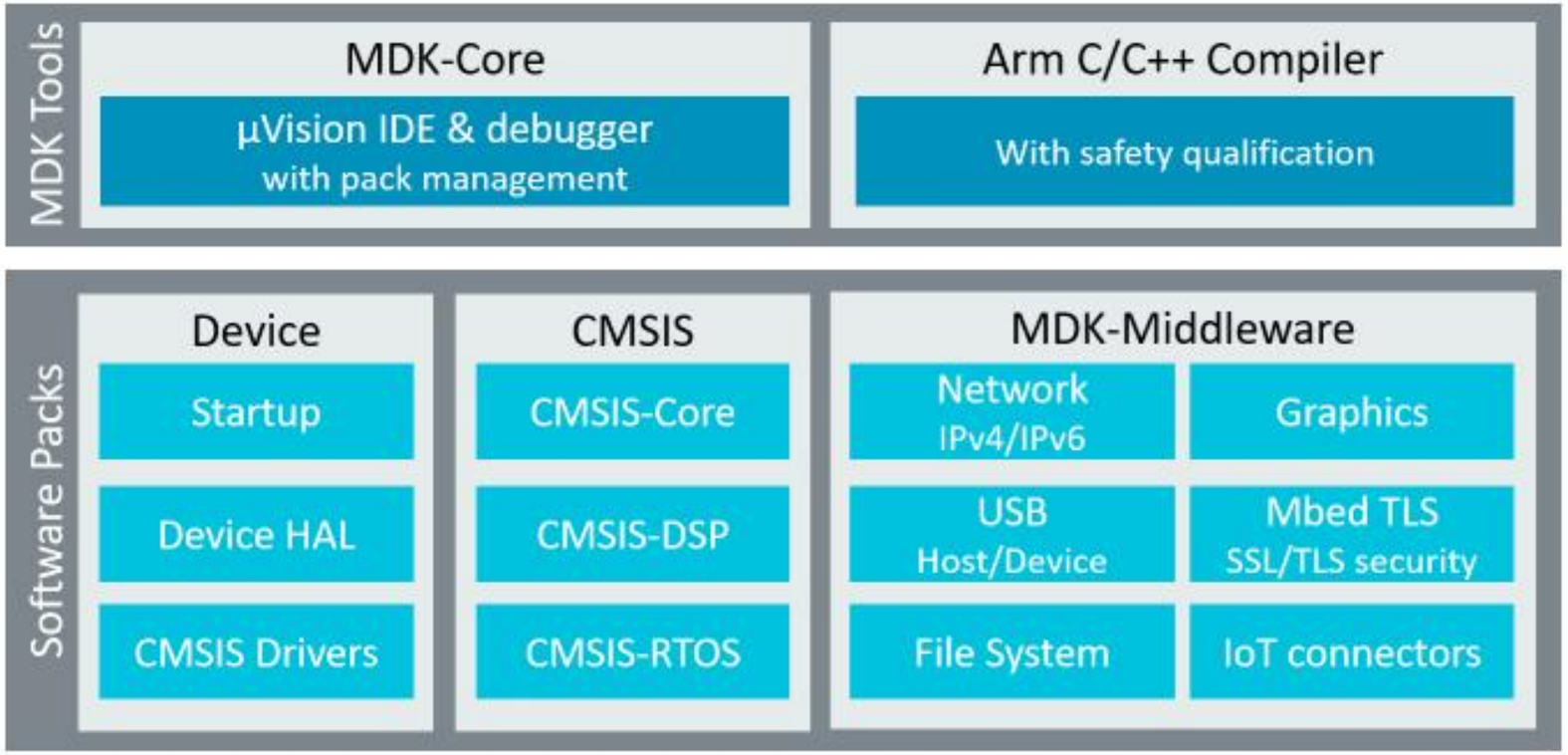


将嵌入式系统开发的各个阶段所用到的软件开发工具如编辑器、编译器、链接器、调试和下载器等集成在一个软件中，形成一个集成化、可视化的综合开发工具供开发者使用。

针对ARM内核的主流集成开发环境主要有Keil MDK和IAR EWARM。

3.3.2 集成开发环境Keil MDK

MDK——Keil ARM Microcontroller Development Kit，简称MDK，也称作Keil MDK



Keil MDK组成框图

3.3.2 集成开发环境Keil MDK

MDK当前最新的版本是MDK 5.27，使用keil uVision5 IDE集成开发环境

- 包含所有基于ARM的微控制器的嵌入式应用组件
- 由MDK-Core和Arm编译器两大部分组成
- MDK-Core主要指keil 5的软件安装exe文件

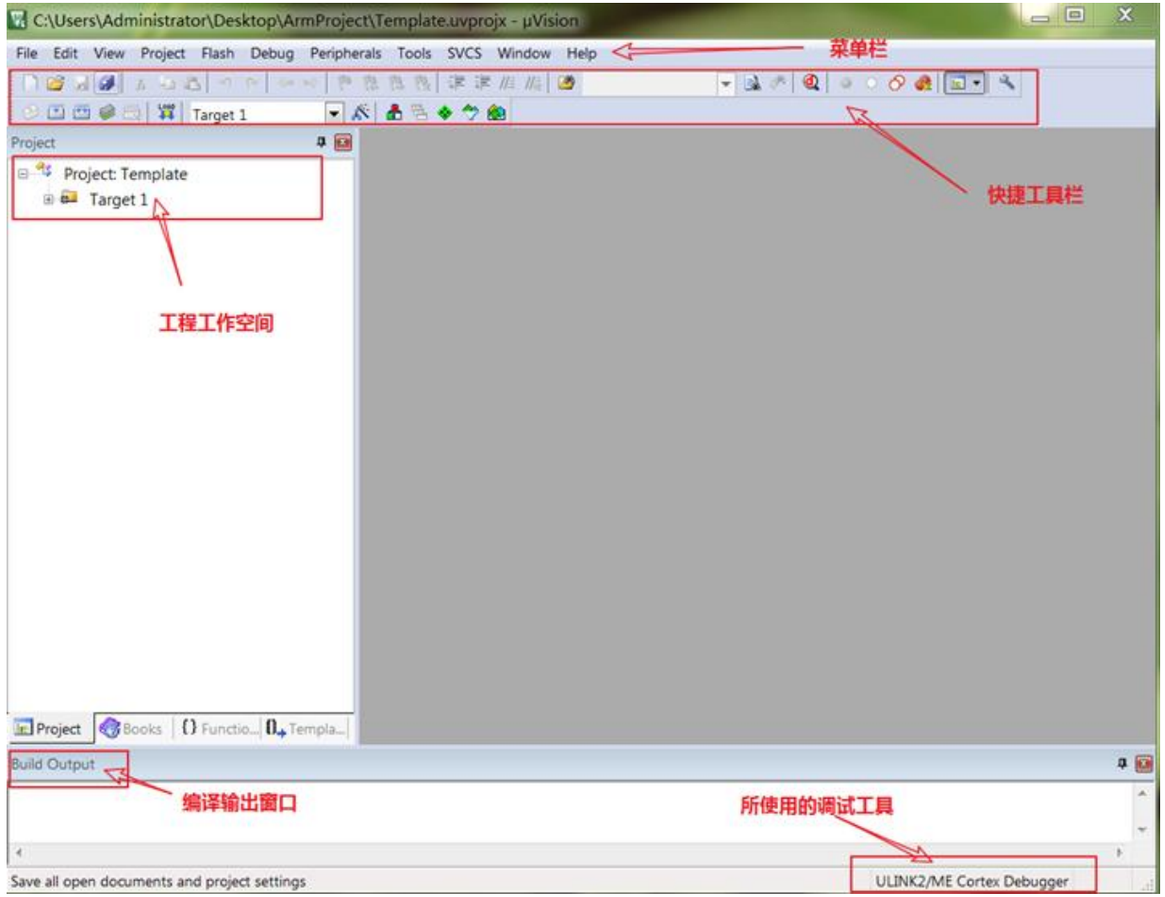
MDK Tools

keil uVision5
IDE集成开发环境

Software Packs

- 在keil 5集成开发软件安装好后，针对具体芯片使用相应设备开发包，安装相应设备的驱动支持程序。

3.3.2 集成开发环境Keil



从Keil官网https://www.keil.com/download/下载最新的Keil软件

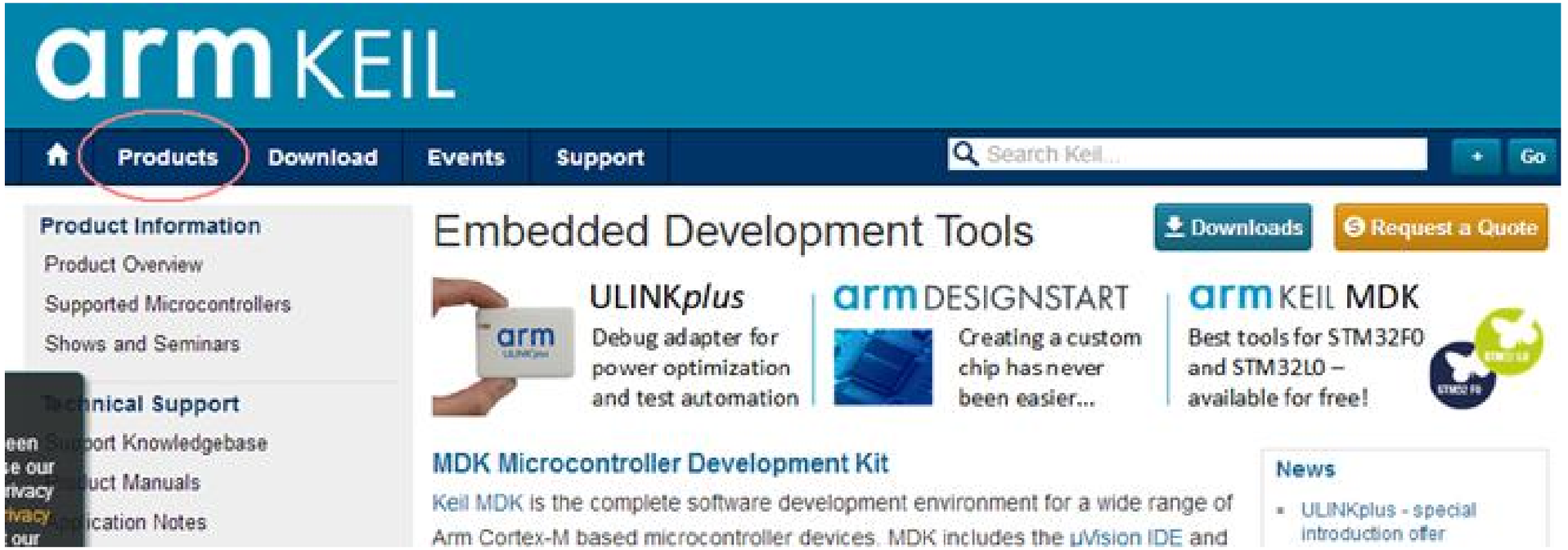
在安装Keil软件时，其安装路径最好不要包含中文

界面由菜单栏、快捷工具栏、工程工作空间和编译输出窗口组成。

3.3.3 下载和安装packs包

1

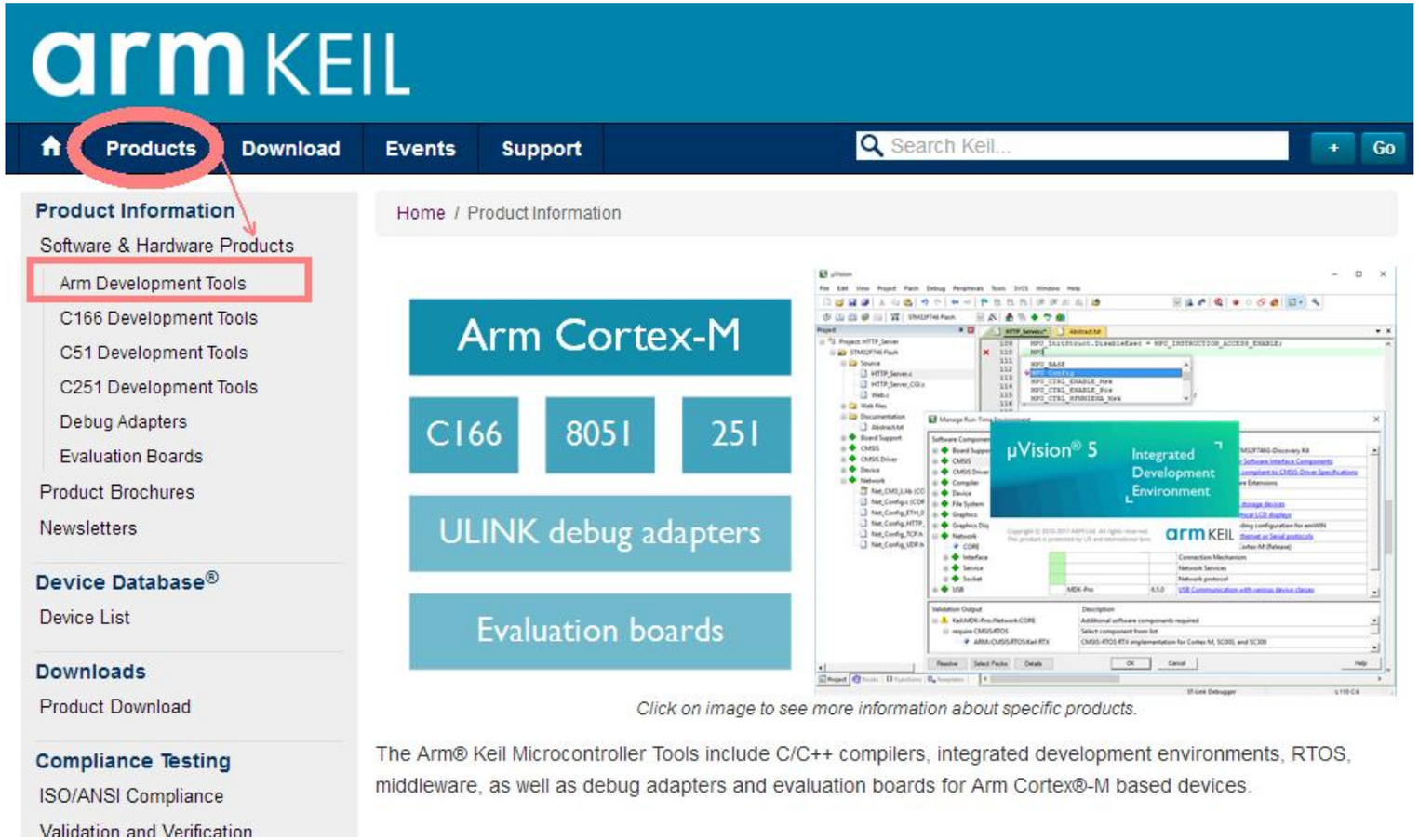
打开Keil官网www.keil.com，在主页上找到“Products”一栏



3.3.3 下载和安装packs包

2

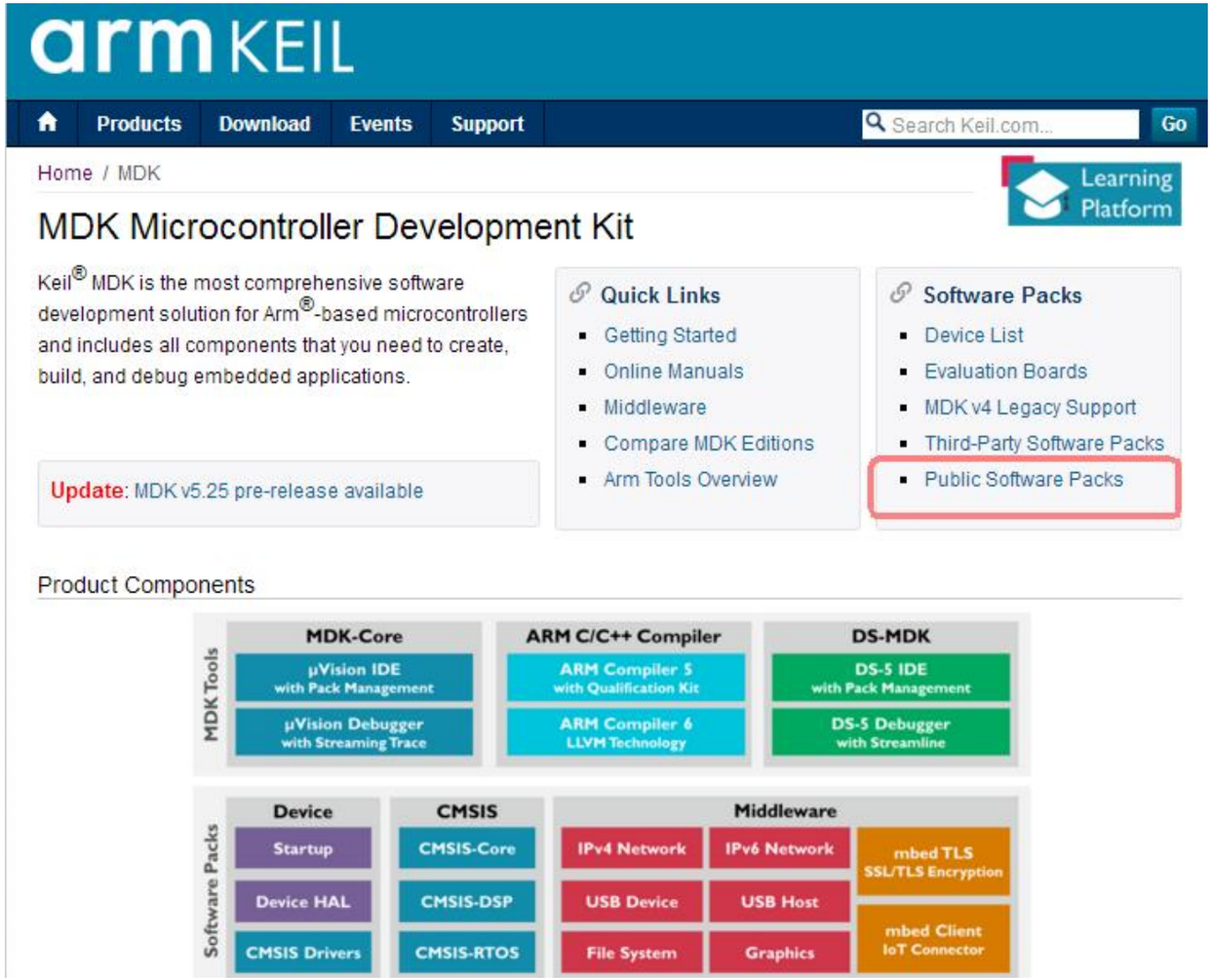
单击“Products”，进入如图所示界面，找到“ARM development tools”(以STM32F1系列为例)



3.3.3 下载和安装packs包

3

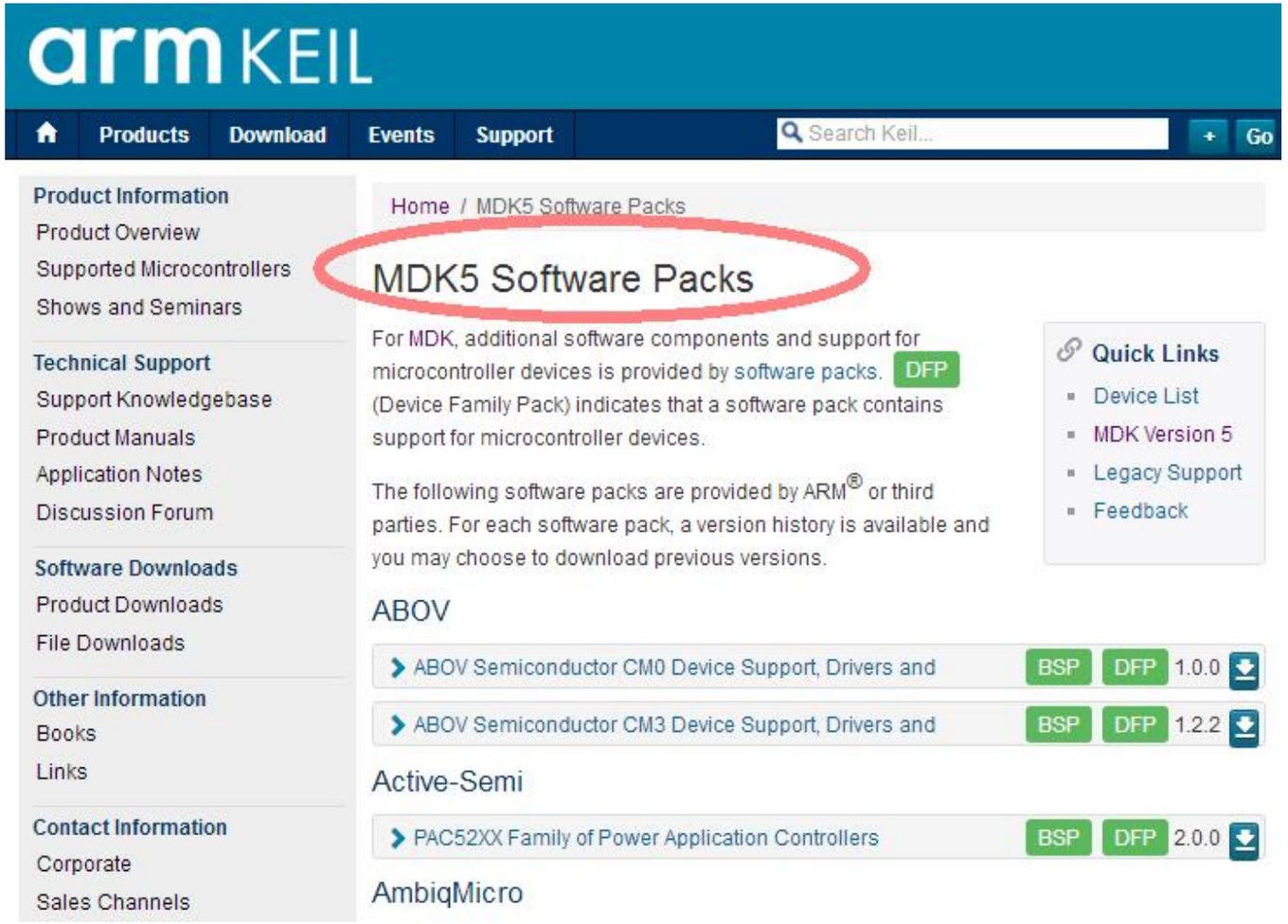
单击“ARM development tools”，进入ARM产品页面，找到“Public software Packs”



3.3.3 下载和安装packs包

4

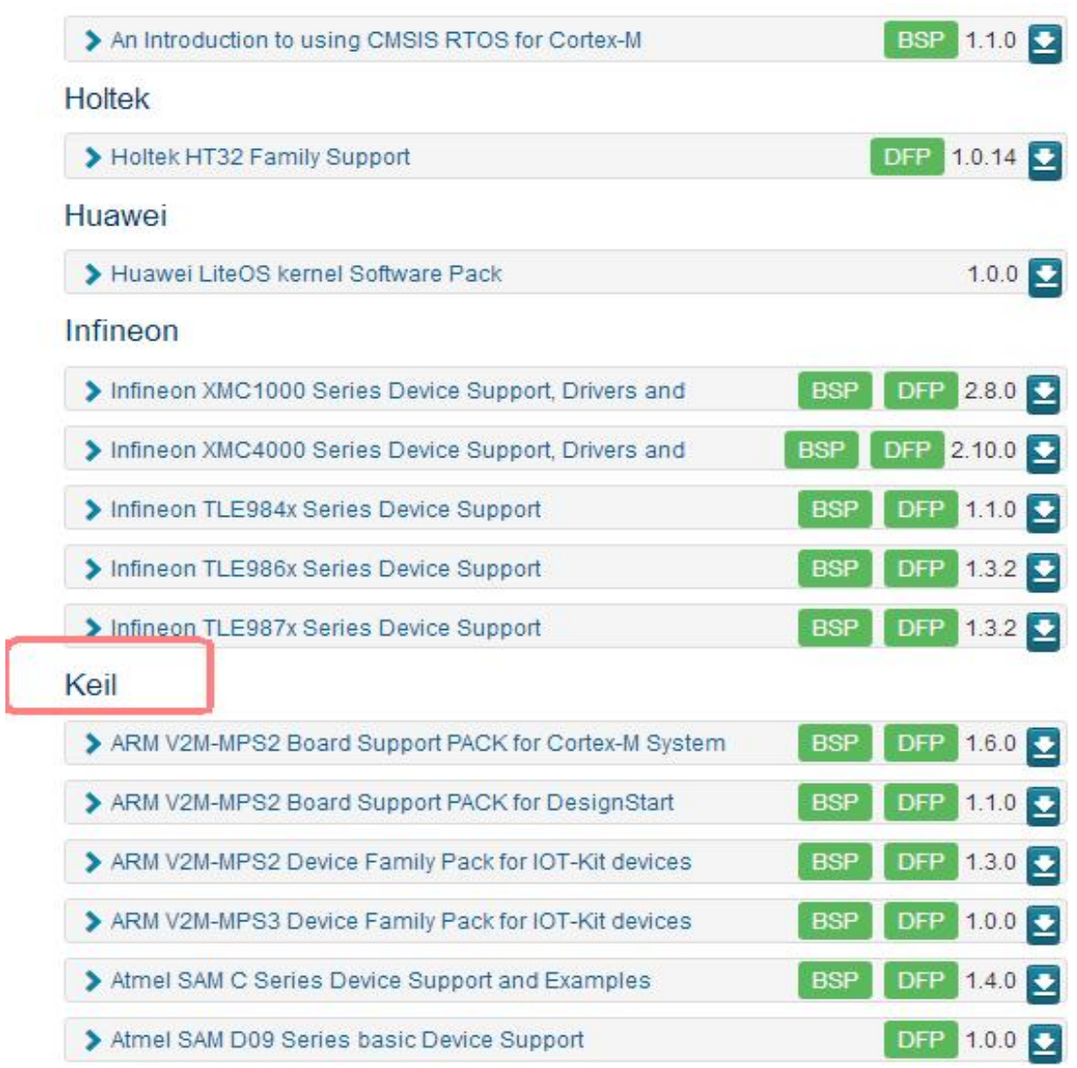
单击“Public software Packs”。可以看到很多packs，找到你想要的下载。



3.3.3 下载和安装packs包

5

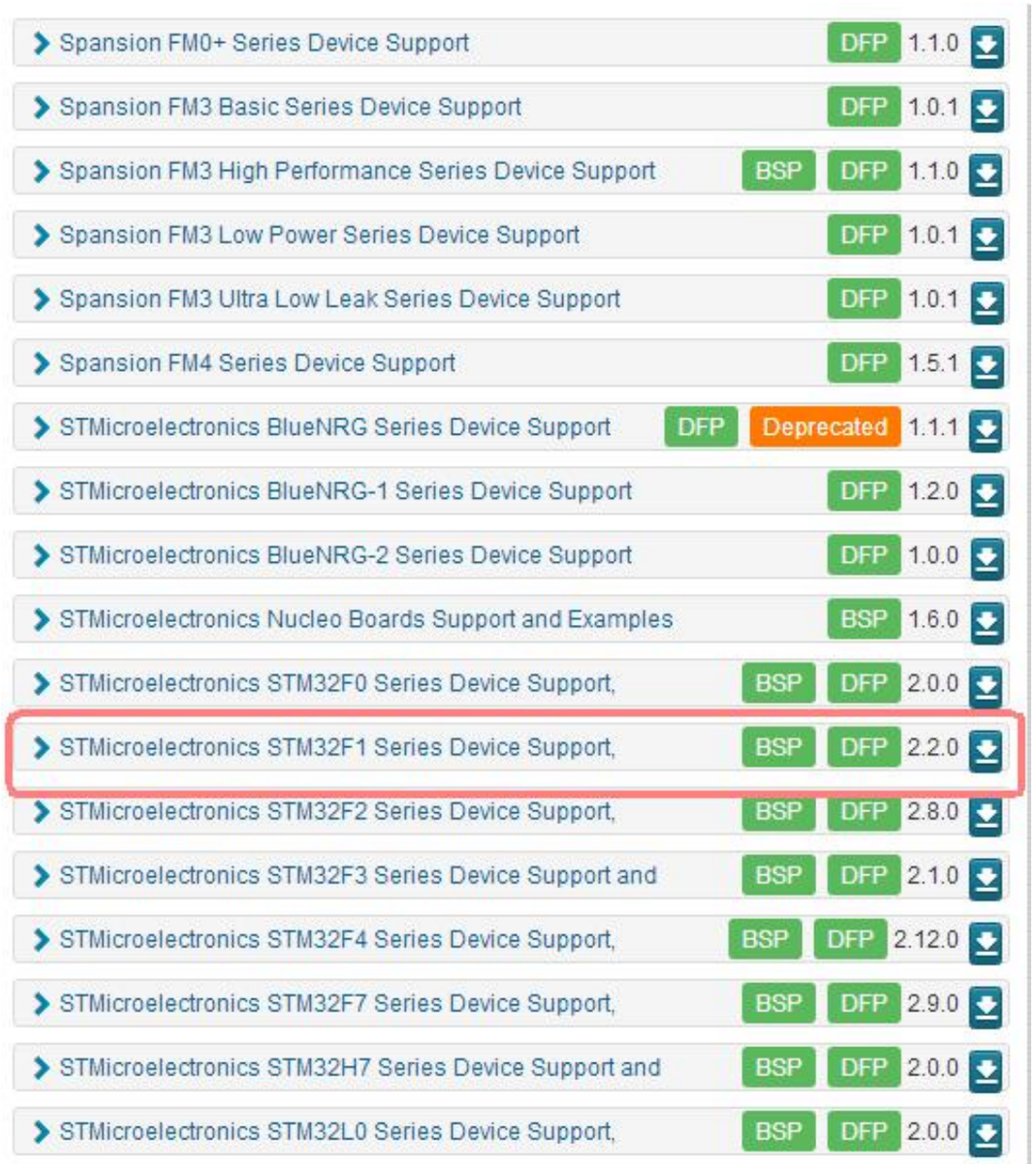
往下拉，找到“Keil”一栏



3.3.3 下载和安装packs包

6

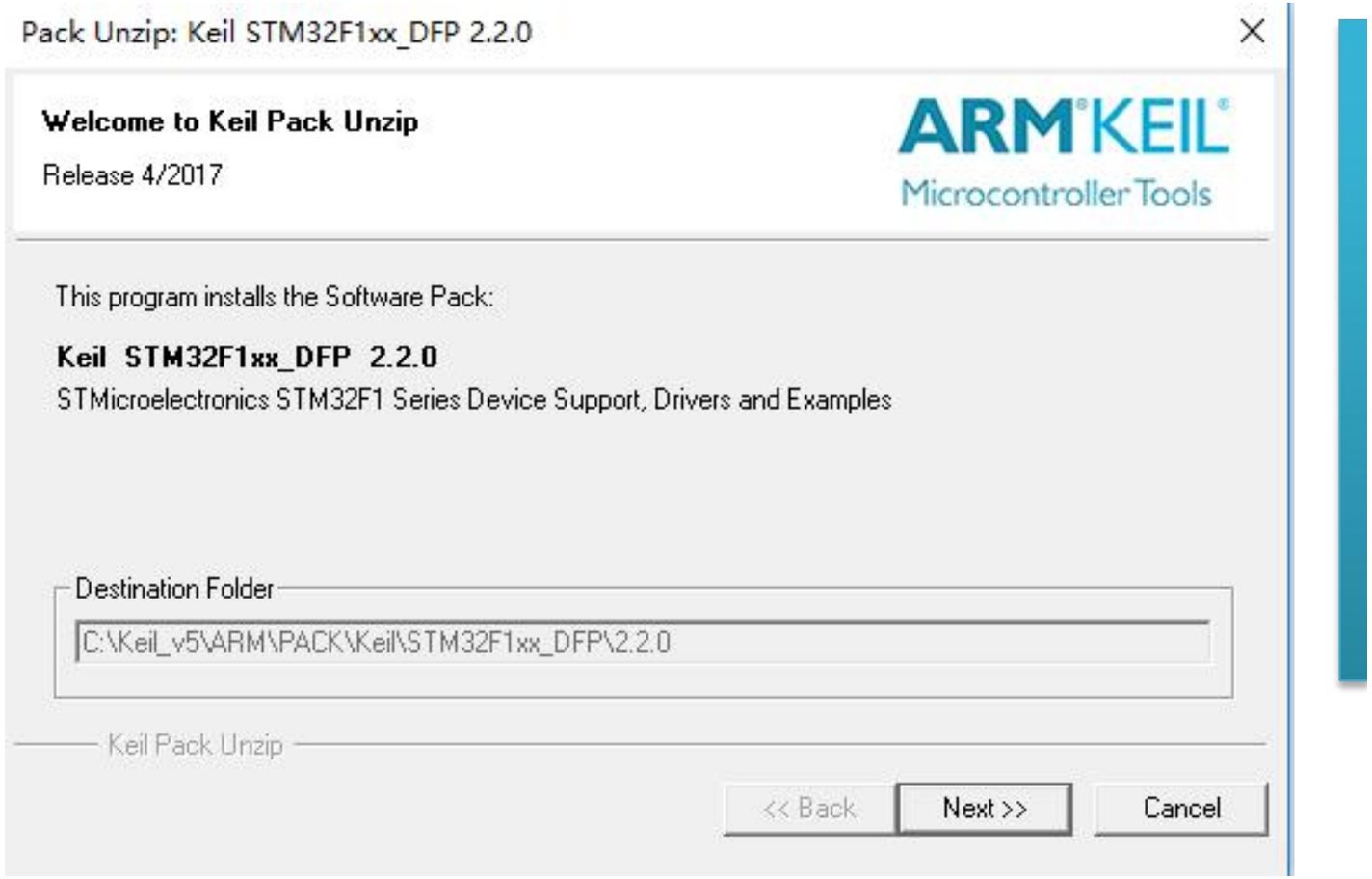
在Keil里面找到“STMicroelectronics STM32F1 Series Device Support”，即是STM32F1xx系列pack，Keil.STM32F1xx_DFP.2.2.0.pack，单击即可下载。



3.3.3 下载和安装packs包

7

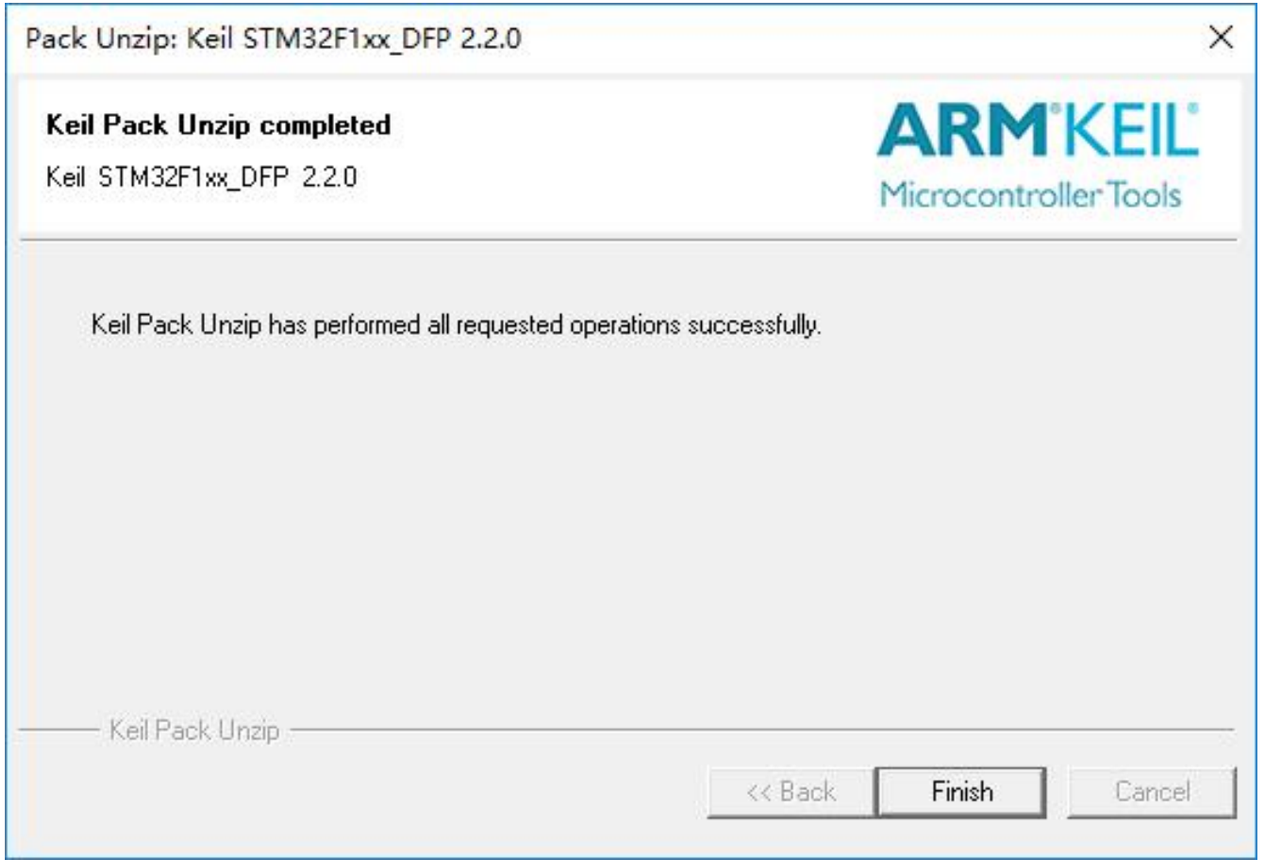
双击下载好的Keil.STM32F1xx_DFP.2.2.0.pack，打开如图所示界面，单击“Next”进行packs包的安装。



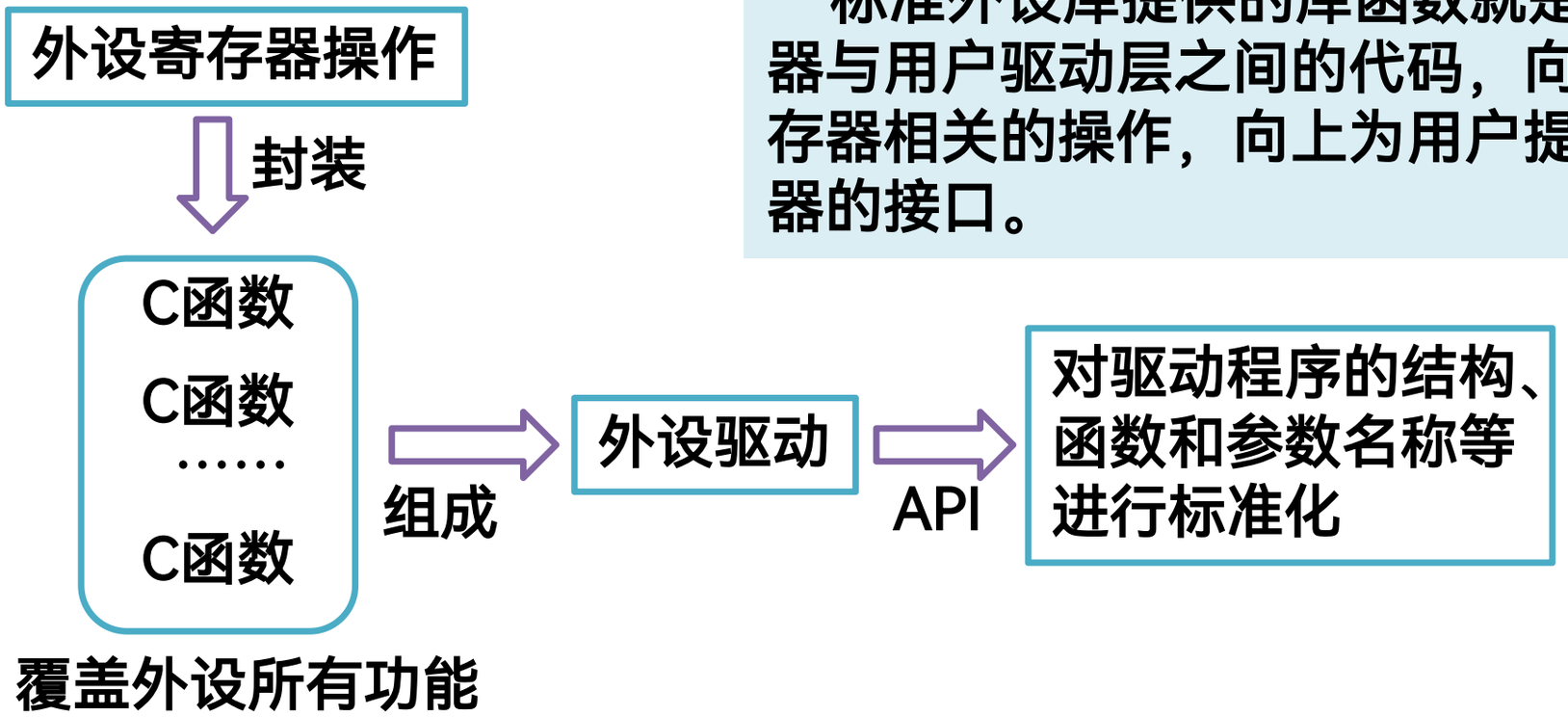
3.3.3 下载和安装packs包

8

当出现如图所示界面，表示packs芯片固件包安装成功，单击“Finish”完成。



3.4 基于标准外设库开发平台搭建

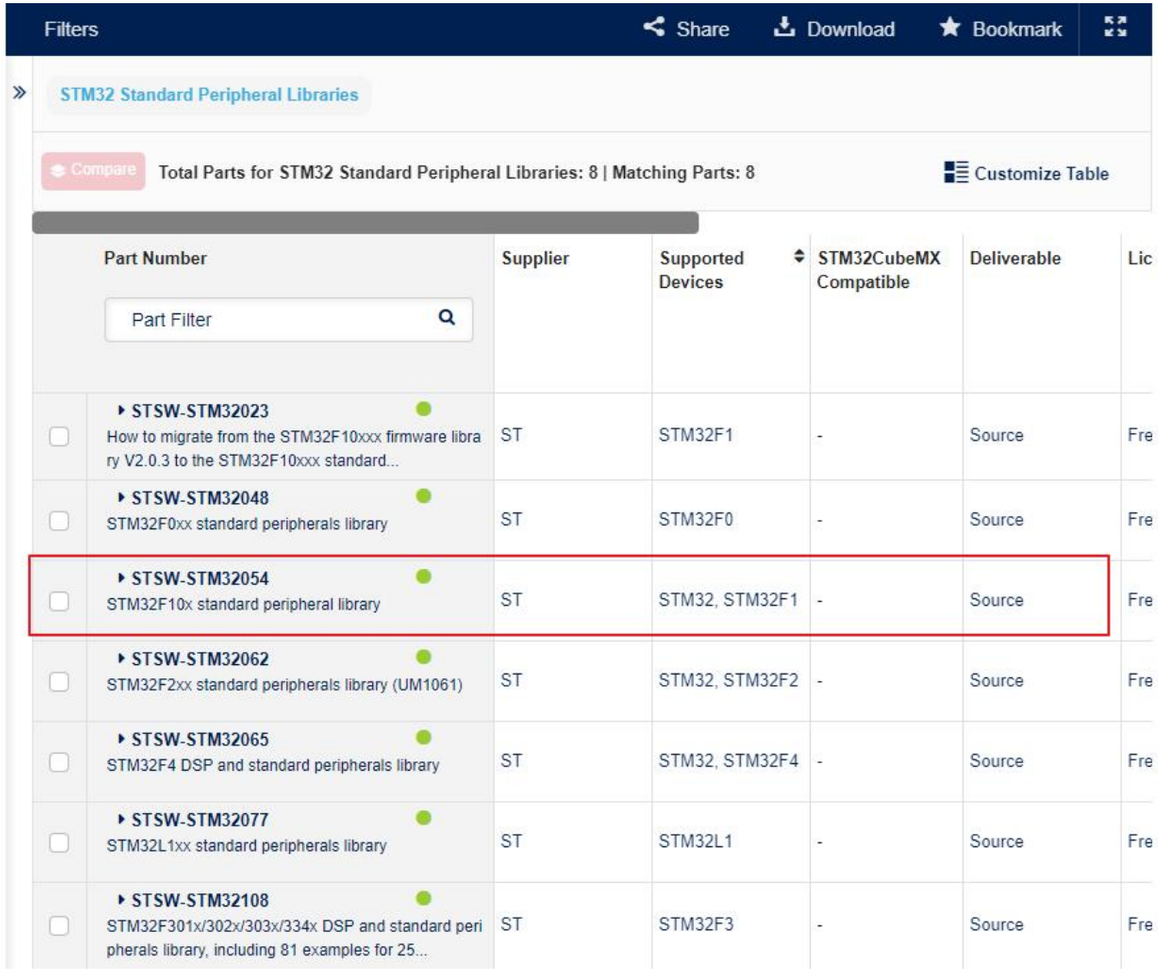


标准外设库提供的库函数就是架设在寄存器与用户驱动层之间的代码，向下处理与寄存器相关的操作，向上为用户提供操作寄存器的接口。

使用标准外设库开发方式最大的优势在于开发者不用深入了解底层硬件细节，就可以灵活规范地使用每一个外设。

3.4.1 STM32标准外设库文件结构

STM32标准外设库的最新版本为v3.5.0，可以在ST的官网上直接下载



3.4.1 STM32标准外设库文件结构

解压后的STM32F10x_StdPeriph_Lib_V3.5.0，包含四个文件夹和一个帮助文



使用手册和帮助文档

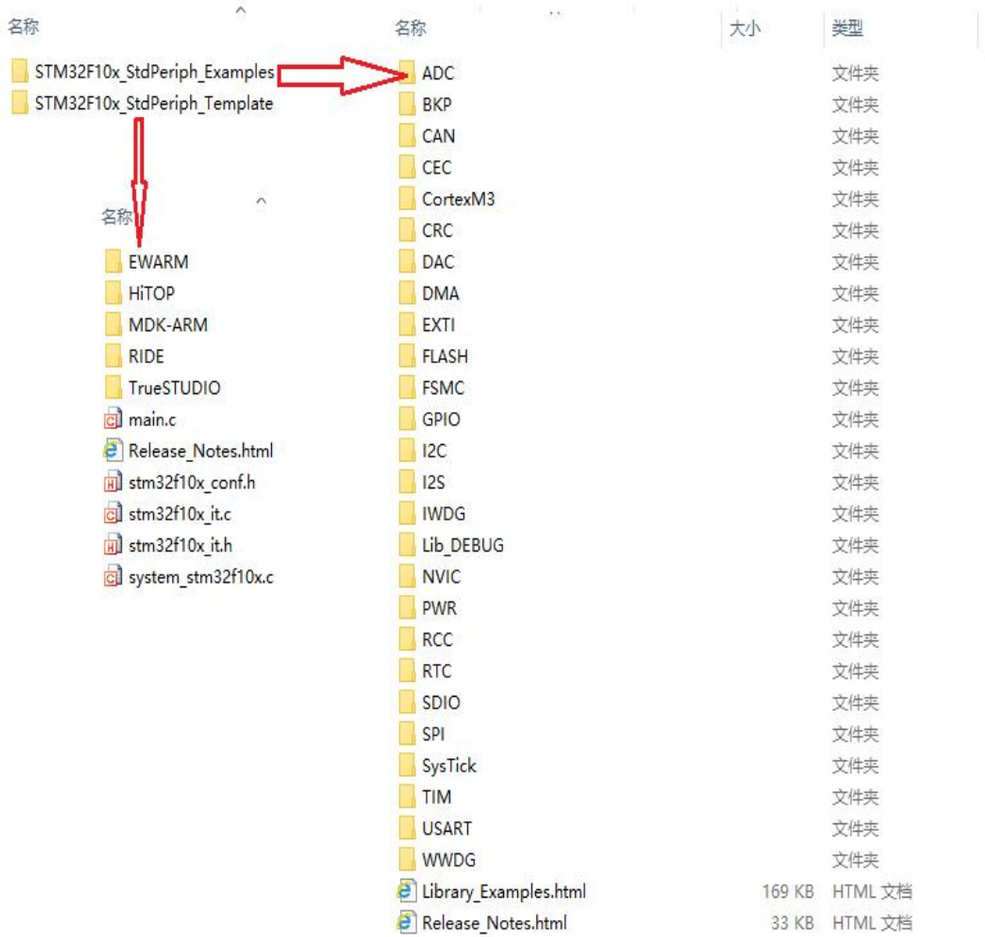
_htmresc：该文件夹中存放的是ST的logo图片，对建立工程没有作用。

Libraries：库函数的源文件，包含STM32的系统文件，如启动文件和大量的头文件。

Project：包含ST官方所有例程和基于不同编译器的项目模板.

Utilities：包含了相关评估板的示例程序和驱动函数

3.4.1 STM32标准外设库文件结构



由STM32F10x_StdPeriph_Examples和STM32F10x_StdPeriph_Template文件夹组成，包含

- ST官方所有例程
- 基于不同编译器的项目模板，如各个外设的例程(如ADC、CAN、DMA、IIC等）和几个不同编译器的工程模板(如MDK-ARM、EWARM等)

3.4.1 STM32标准外设库文件结构

STM32F10x标准外设库文件分析

文件类型	文件	说明
启动文件	startup_stm32f10x_hd.s	汇编语言编写的启动文件，hd代表大容量芯片
内核相关	core_cm3.c	内核文件，包含了内核寄存器的定义和内核外设的函数，Cortex-M3内核及其外设：NVIC，SysTick等
	core_cm3.h	把core_cm3.c中的寄存器封装成相应的函数
	misc.c misc.h	用于中断管理，包含NVIC相关的函数定义和SysTick
外设相关	stm32f10x.h	包含所有外设寄存器和结构体的定义、位定义、中断向量表、存储空间的地址映射等
	system_stm32f10x.c system_stm32f10x.h	系统文件，函数SystemInit用于系统初始化和STM32系统时钟配置
	stm32f10x_ppp.h stm32f10x_ppp.c	外设驱动函数文件，包括相关外设的初始化配置和部分功能应用函数，是编程功能实现的重要组成部分。
	stm32f10x_conf.h	固件库配置文件，通过增加和删除所包含的外设头文件来选择固件库所使用的外设
应用程序	main.c	主函数
中断	stm32f10x_it.c stm32f10x_it.h	存放用户编写的中断服务函数

3.4.1 STM32标准外设库文件结构

1．core_cm3.c和core_cm3.h文件

CMSIS的核心文件

提供进入Cortex-M3内核的接口，由ARM公司提供，对所有基于Cortex-M3内核的芯片都适用

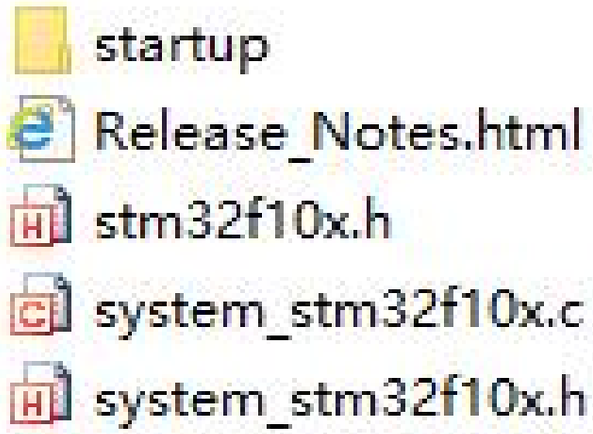
core_cm3.c是内核通用源文件，主要存放的是操作内核外设寄存器的函数

对应的头文件是core_cm3.h，对core_cm3.c中的寄存器进行了映射封装。

3.4.1 STM32标准外设库文件结构

2．system_stm32f10x.c和system_stm32f10x.h文件

DeviceSupport\ST\STM32F10x目录主要存放一些启动文件以及基础的寄存器和中断向量定义文件



主要功能是配置STM32的系统时钟和总线时钟，其中最重要的函数是SystemInit()函数，是在系统启动时调用的初始化系统函数，用来设置整个STM32的系统时钟。

3.4.1 STM32标准外设库文件结构

2．system_stm32f10x.c和system_stm32f10x.h文件

通过变量SystemCoreClock可以获取系统时钟，如果系统时钟为72MHz，则SystemCoreClock=72000000，SystemCoreClockUpdate()函数用于改变系统时钟，让SystemCoreClock变量更新。

```
/**
  ******************************************************************************
  * @file    system_stm32f10x.c
  * @author  MCD Application Team
  * @version V3.5.0
  * @date    11-March-2011
  * @brief   CMSIS Cortex-M3 Device Peripheral Access Layer System Source File.
  *
  * 1.  This file provides two functions and one global variable to be called from
  *     user application:
  *      - SystemInit(): Setups the system clock (System clock source, PLL Multiplier
  *                      factors, AHB/APBx prescalers and Flash settings).
  *                      This function is called at startup just after reset and
  *                      before branch to main program. This call is made inside
  *                      the "startup_stm32f10x_xx.s" file.
  *
  *      - SystemCoreClock variable: Contains the core clock (HCLK), it can be used
  *                                  by the user application to setup the SysTick
  *                                  timer or configure other parameters.
  *
  *      - SystemCoreClockUpdate(): Updates the variable SystemCoreClock and must
  *                                 be called whenever the core clock is changed
  *                                 during program execution.
  *
  * 2. After each device reset the HSI (8 MHz) is used as system clock source.
  *    Then SystemInit() function is called, in "startup_stm32f10x_xx.s" file, to
  *    configure the system clock before to branch to main program.
  *
```

3.4.1 STM32标准外设库文件结构

3．stm32f10x.h头文件

主要包含STM32F10x系列微控制器外设寄存器的定义、位定义、中断向量表以及存储空间的映射等，内含许多结构体和宏定义。

```
1001  typedef struct
1002  {
1003    __IO uint32_t CRL;
1004    __IO uint32_t CRH;
1005    __IO uint32_t IDR;
1006    __IO uint32_t ODR;
1007    __IO uint32_t BSRR;
1008    __IO uint32_t BRR;
1009    __IO uint32_t LCKR;
1010  } GPIO_TypeDef;
```

其中，__IO是宏定义，代表volatile，定义在Libraries\CMSIS\CM3\CoreSupport\core_cm3.h文件中：

```
#define __IO volatile /*!< defines 'read / write' permissions */
```

3.4.1 STM32标准外设库文件结构

4．启动文件startup_stm32f10x_xx.s

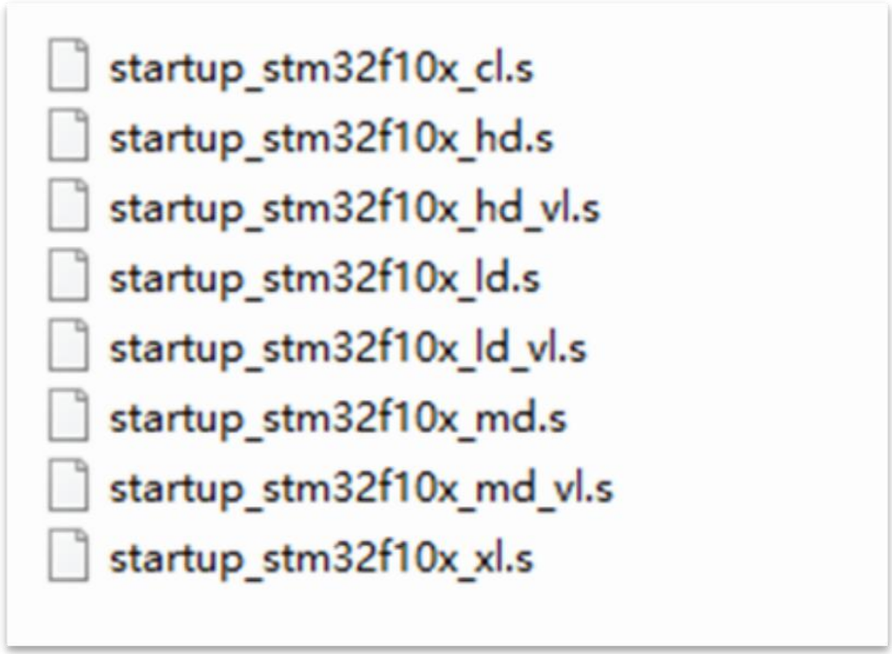
启动文件的作用除了对堆栈等初始化，对中断向量表以及对应的中断处理函数的定义外，还要负责芯片上电后引导程序进入main函数，芯片上电复位后会首先运行一段汇编程序，在进入main函数之前调用SystemInit()函数初始化系统时钟。

```
146  ; Reset handler
147  Reset_Handler    PROC
148                   EXPORT  Reset_Handler                 [WEAK]
149                   IMPORT  __main
150                   IMPORT  SystemInit
151                   LDR     R0, =SystemInit
152                   BLX     R0
153                   LDR     R0, =__main
154                   BX      R0
155                   ENDP
```

3.4.1 STM32标准外设库文件结构

4．启动文件startup_stm32f10x_xx.s

在startup\arm文件夹下，有8个以startup打头的.s文件，用于针对不同容量的STM32芯片选择不同的启动文件，这里的容量是指片内Flash的大小。



startup_stm32f10x_hd.s：适用于大容量的STM32，Flash容量大于等于256K。
startup_stm32f10x_md.s：适用于中等容量的STM32，Flash容量64K~256K之间。

startup_stm32f10x_ld.s：适用于小容量的STM32，Flash容量小于32K。

3.4.1 STM32标准外设库文件结构

5．stm32f10x_xxx.c和stm32f10x_xxx.h

xxx代表不同的外设，包含了相关外设的初始化配置和部分功能应用函数。

6．misc.c和misc.h

misc.c和misc.h提供了外设对内核的中断向量控制器NIVC的访问函数，在配置中断时，必须把这个文件添加进工程。

7．stm32f10x_it.c和stm32f10x_it.h

用于编写中断服务函数，相应的中断服务函数接口已在启动文件中定义。

8．stm32f10x_conf.h

用于配置是否使用某外设的头文件，利用这个头文件可以方便地增加和删除外设的驱动函数。

3.4.1 STM32标准外设库文件结构

标准外设库遵从的命名规则

1

PPP表示任一外设缩写

缩写	外设名称	缩写	外设名称
ADC	模数转换器	IIS	IIS总线接口
BKP	备份寄存器	IWDG	独立看门狗
CAN	控制器局域网	NVIC	嵌套向量中断控制器
CRC	CRC计算单元	PWR	电源控制
DAC	数模转换器	RCC	复位和时钟控制
DBGMCU	MCU调试模块	RTC	实时时钟
DMA	DMA控制器	SDIO	SDIO接口
EXTI	外部中断/事件控制器	SPI	SPI接口
SysTick	系统定时器	FSMC	灵活的静态存储器控制器
Flash	Flash存储器	TIM	定时器
GPIO	通用输入输出	USART	通用同步异步收发器
IIC	IIC总线接口	WWDG	窗口看门狗

3.4.1 STM32标准外设库文件结构

标准外设库遵从的命名规则

2

源程序文件和头文件的命名都以“stm32f10x_”作为开头

3

常量仅被应用于一个文件的，定义于该文件中；被应用于多个文件的，在对应头文件中定义，所有常量用大写英文字母命名

4

寄存器作为常量处理，用大写英文字母命名，一般与缩写规范一致

3.4.1 STM32标准外设库文件结构

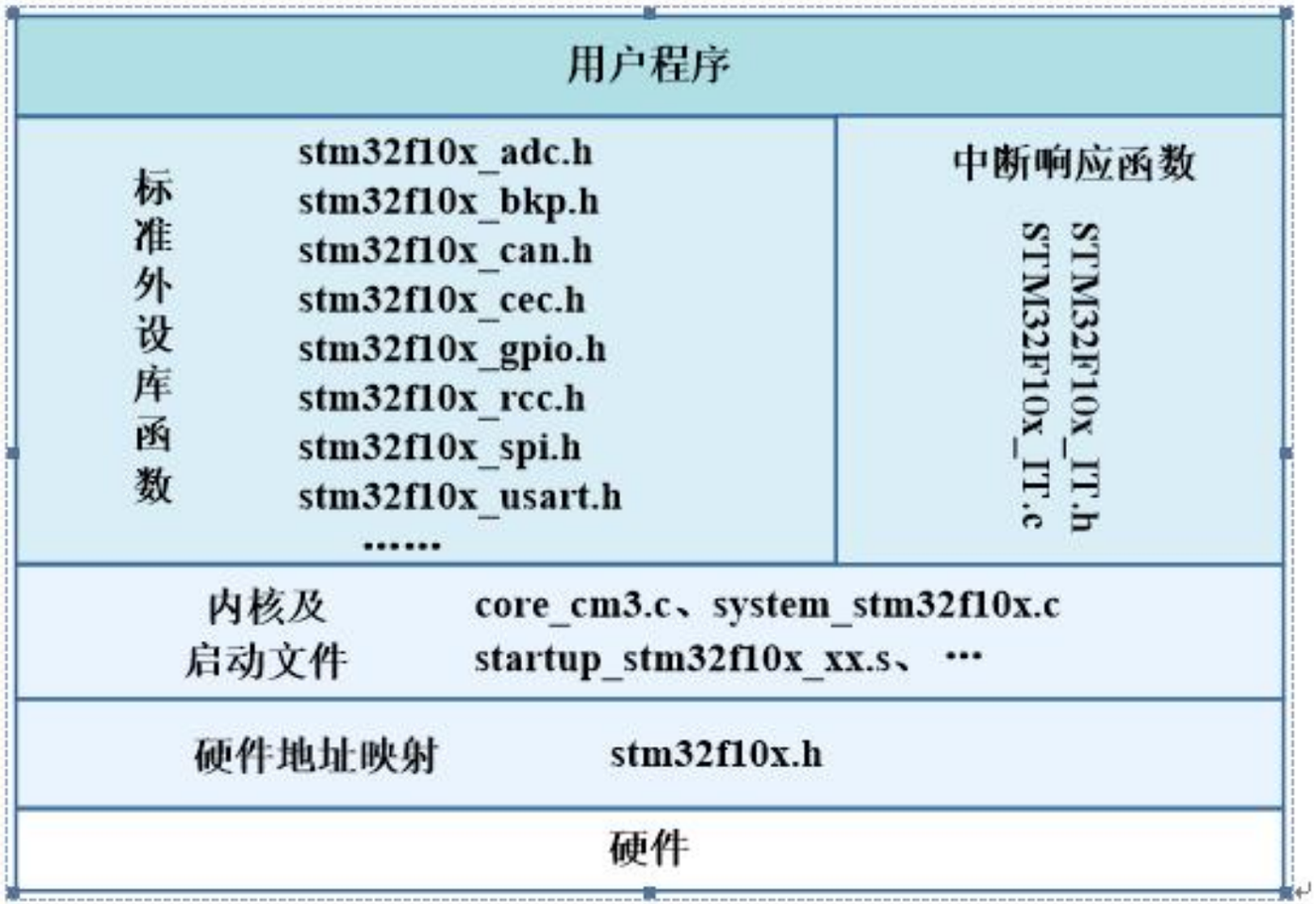
5 外设函数的命名以该外设的缩写加下划线为开头

函数命名	功能	举例
名为PPP_Init的函数	根据PPP_InitTypeDef中指定的参数，初始化外设PPP	TIM_Init
名为PPP_DeInit的函数	复位外设PPP的所有寄存器至缺省值	TIM_DeInit
名为PPP_Cmd的函数	使能或者失能外设PPP	SPI_Cmd
名为PPP_ITConfig的函数	使能或者失能来自外设PPP的中断源	RCC_ITConfig
名为PPP_DMAConfig的函数	使能或者失能外设PPP的DMA接口	TIM1_DMAConfig
名为PPP_GetFlagStatus的函数	检查外设PPP某标志位被设置与否	I2C_GetFlagStatus
名为PPP_ClearFlag的函数	清除外设PPP标志位	I2C_ClearFlag
名为PPP_GetITStatus的函数	判断来自外设PPP的中断发生与否	I2C_GetITStatus
名为PPP_ClearITPendingBit的函数	清除外设PPP中断待处理标志位	I2C_ClearITPendingBit。

3.4.1 STM32标准外设库文件结构

STM32标准外设库结构是符合CMSIS标准的软件架构，其整体框架结构包括：

- 内核启动文件
- 中断响应文件
- 硬件地址映射文件
- 标准外设库函数
- 用户程序



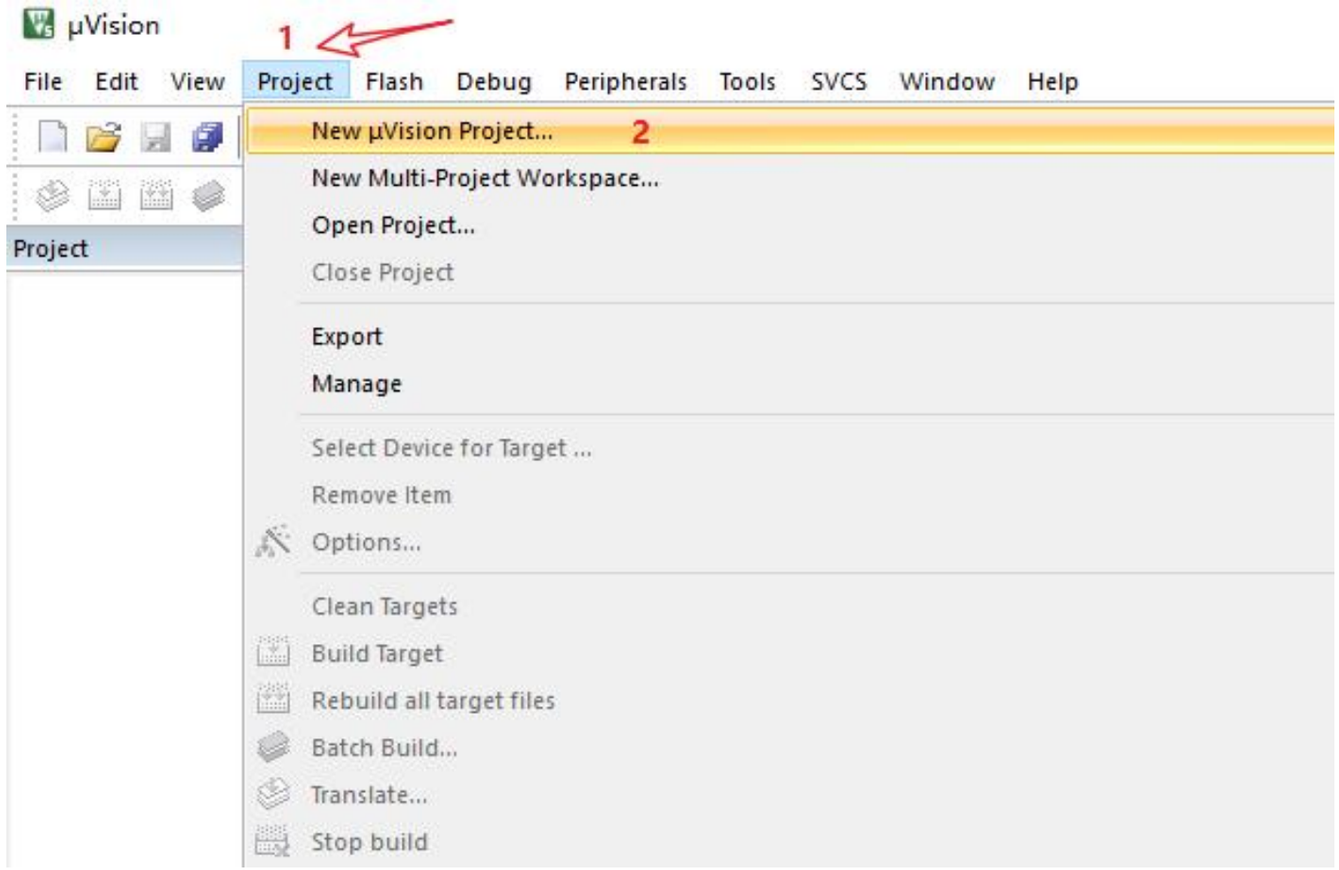
STM32标准外设库框架结构图

3.4.2 基于标准外设库新建工程模板

步骤1：打开keil 5软件新建一个工程

1）新建一个文件夹MyTemplates(工程文件夹的名字随意定义，但最好是英文的）。

2）打开keil 5软件，在菜单栏的Project中单击“New uVision Project”，选择本地新建的MyTemplates文件

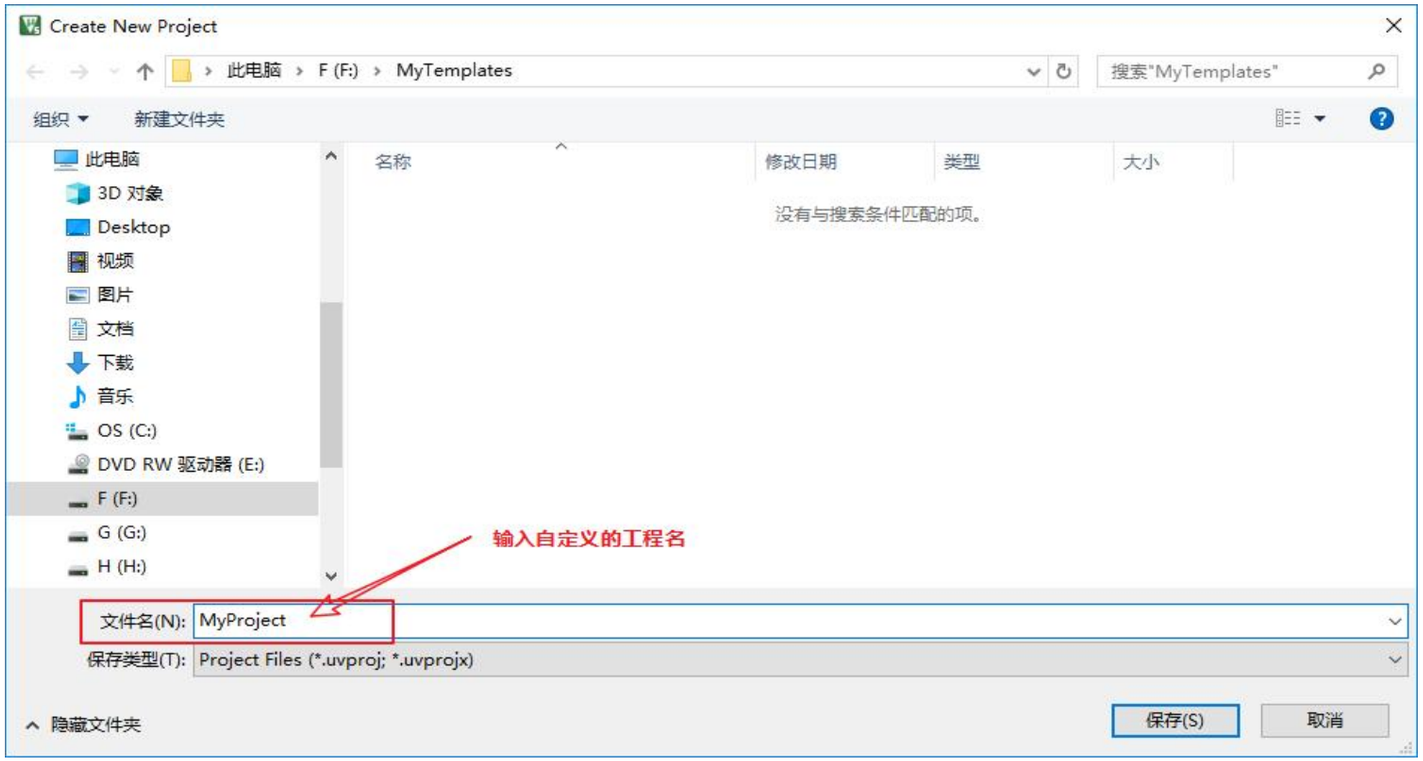


新建一个工程文件

3.4.2 基于标准外设库新建工程模板

步骤1：开keil 5软件新建一个工程

3）在出现的提示框中，输入工程文件名MyProject，创建一个新的工程

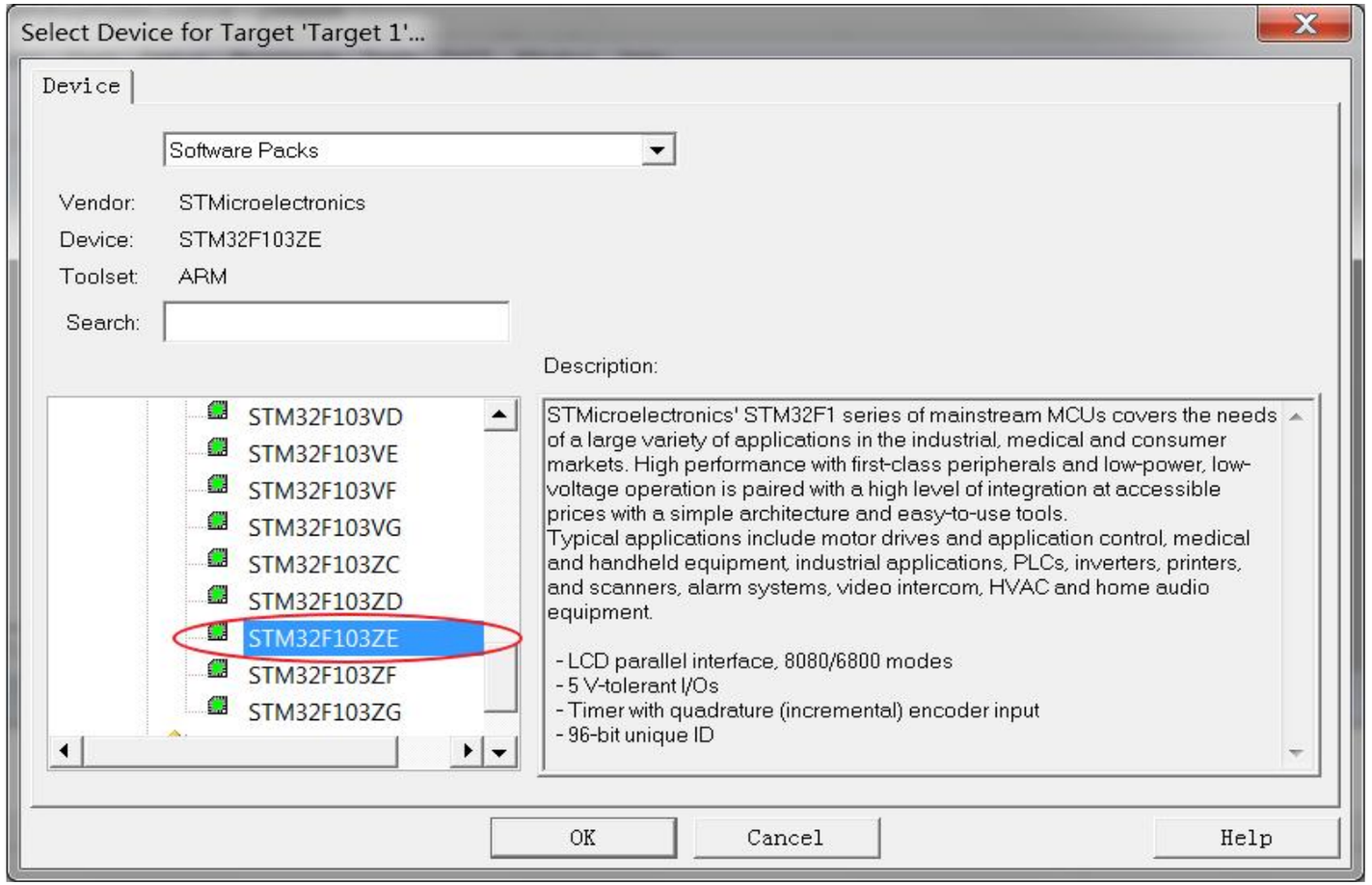


输入工程名

3.4.2 基于标准外设库新建工程模板

步骤1：开keil 5软件新建一个工程

4）单击保存，在新出现的界面中选择MCU芯片型号，这里选择STM32F103ZE

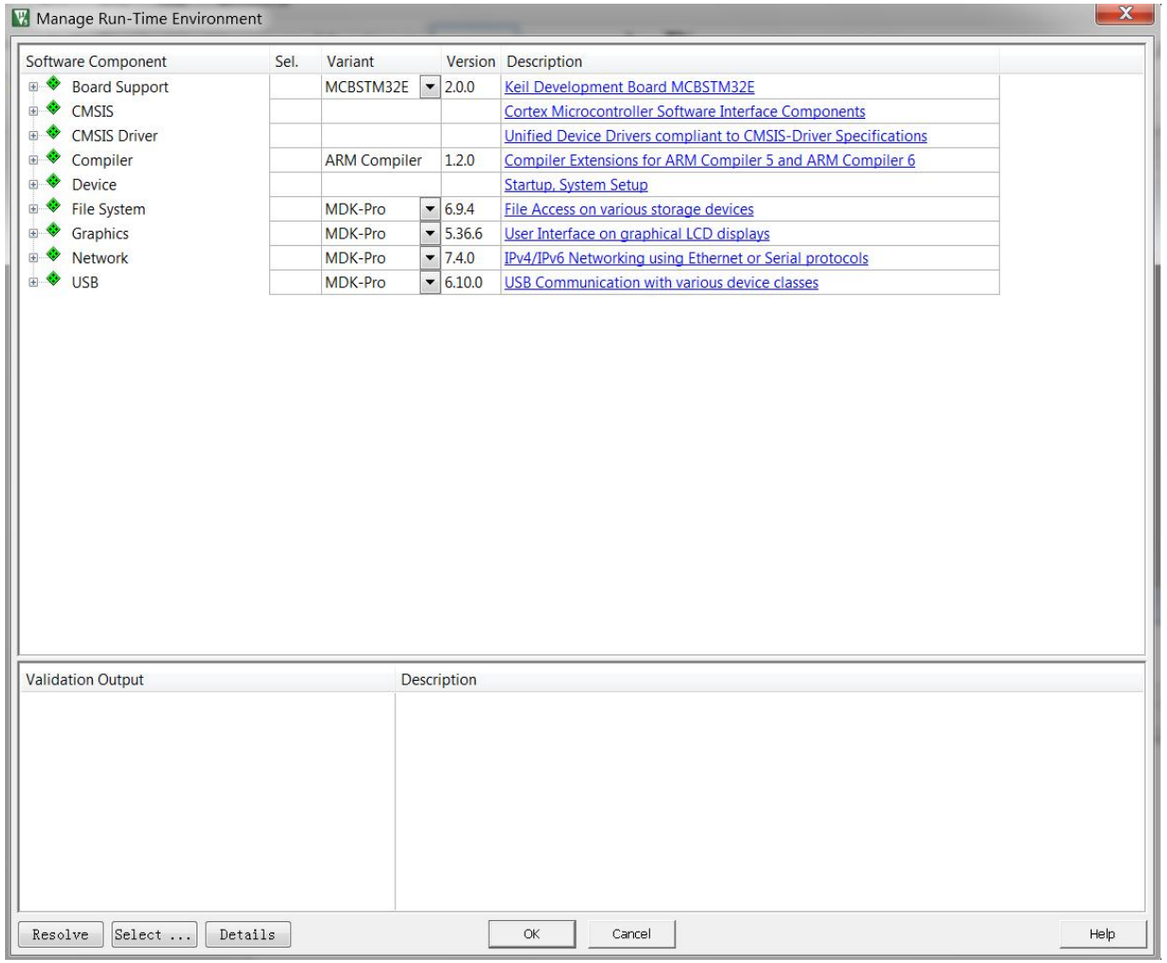


选择MCU芯片型号

3.4.2 基于标准外设库新建工程模板

步骤1：开keil 5软件新建一个工程

5）单击“OK”，出现添加库文件界面，这里直接关掉或单击“OK”，后面可以通过手动方式添加相应的库文件



添加库文件界面

3.4.2 基于标准外设库新建工程模板

步骤2：复制标准外设库文件到相应文件夹中

在MyTemplates文件夹下新建四个文件夹

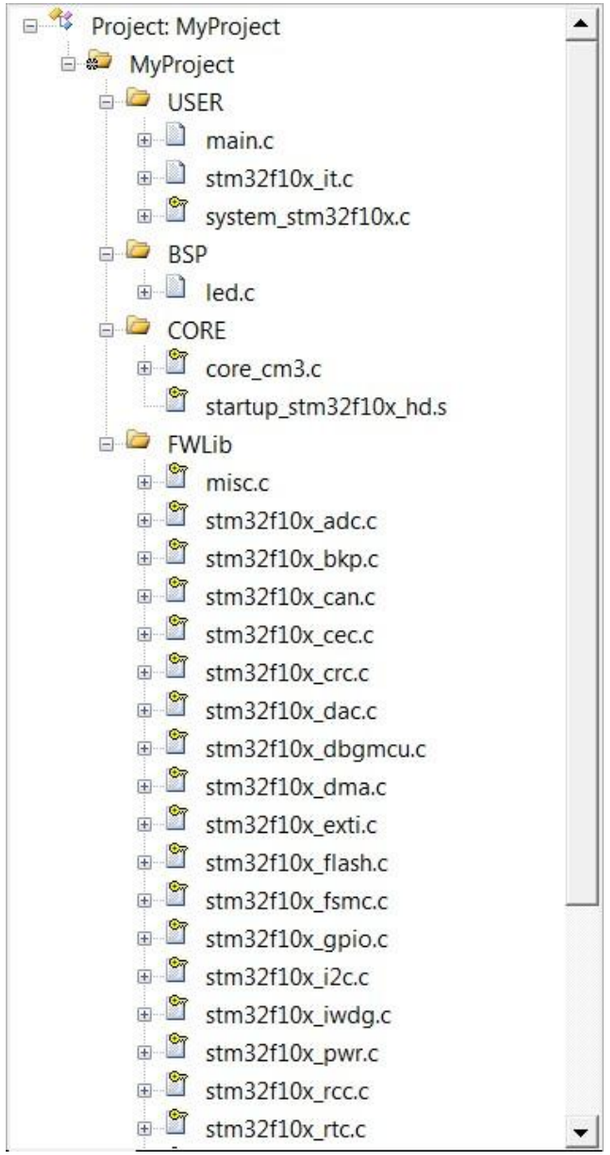
- **User**：用来存放main函数和中断程序。复制stm32f10x_conf.h、stm32f10x_it.c、stm32f10x_it.h和main.c放在新建的User文件夹下
- **BSP**：存放用户自己编写的硬件驱动文件，比如LED.c、LED.h等
- **System s**：存放系统文件、启动文件等，从标准外设库中复制core_cm3.c、core_cm3.h、startup_stm32f10x_hd.s、system_stm32f10x.c四个文件存放进去
- **FWLib**：存放STM32标准外设库的src和inc两个文件，这两个文件包含了芯片上的所有驱动。从标准外设库中直接把src和inc两个文件夹复制过去即可

3.4.2 基于标准外设库新建工程模板

步骤2：复制标准外设库文件到相应文件夹中

新建工程模板的结构如图所示

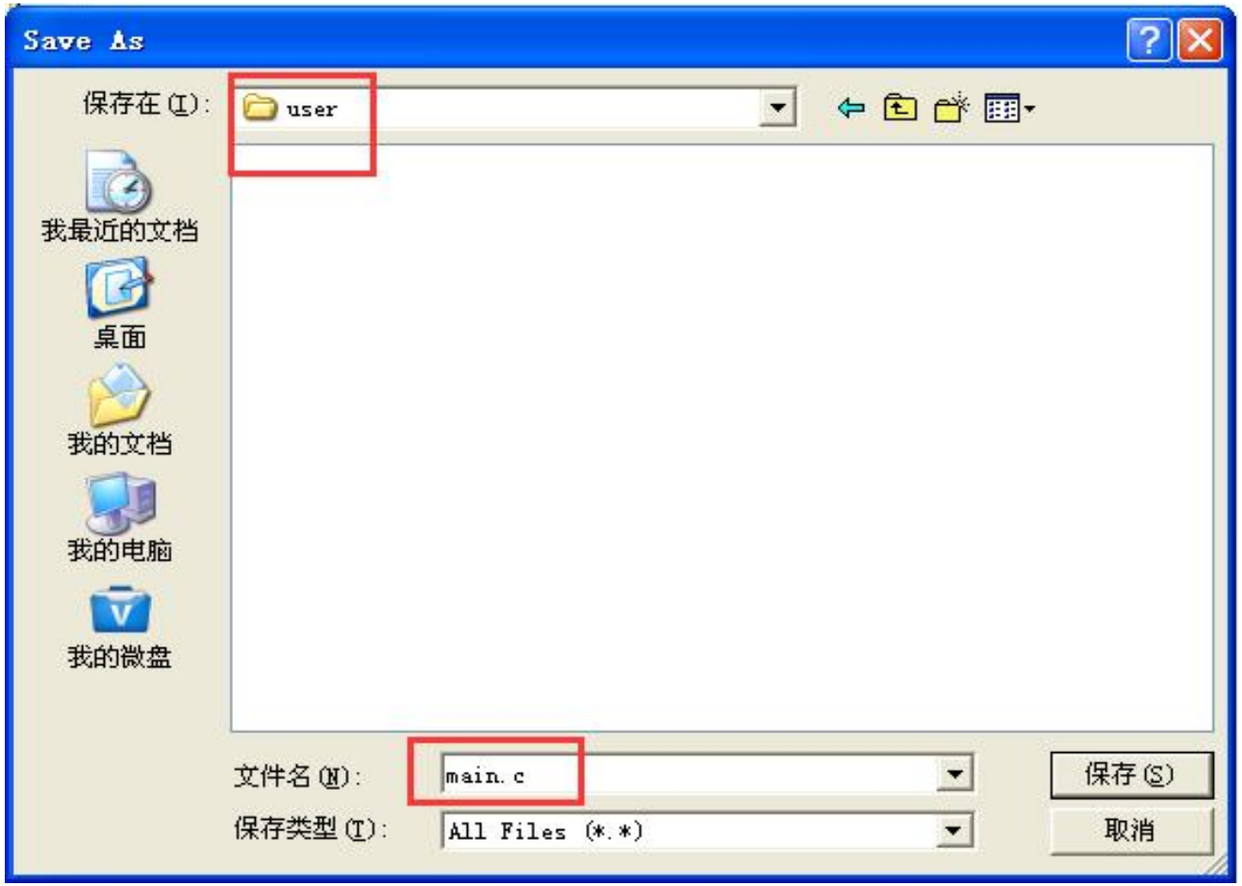




3.4.2 基于标准外设库新建工程模板

步骤3：新建一个main.c源文件

单击keil 5编译器的新建文件按钮，新建一个文件，命名为main.c，然后保存到user文件夹，以后只要是自己新建的.c文件和.h文件都放在这个文件夹下。



3.4.2 基于标准外设库新建工程模板

步骤4：把内核、启动文件、外设等相关的库文件添加进Keil工程

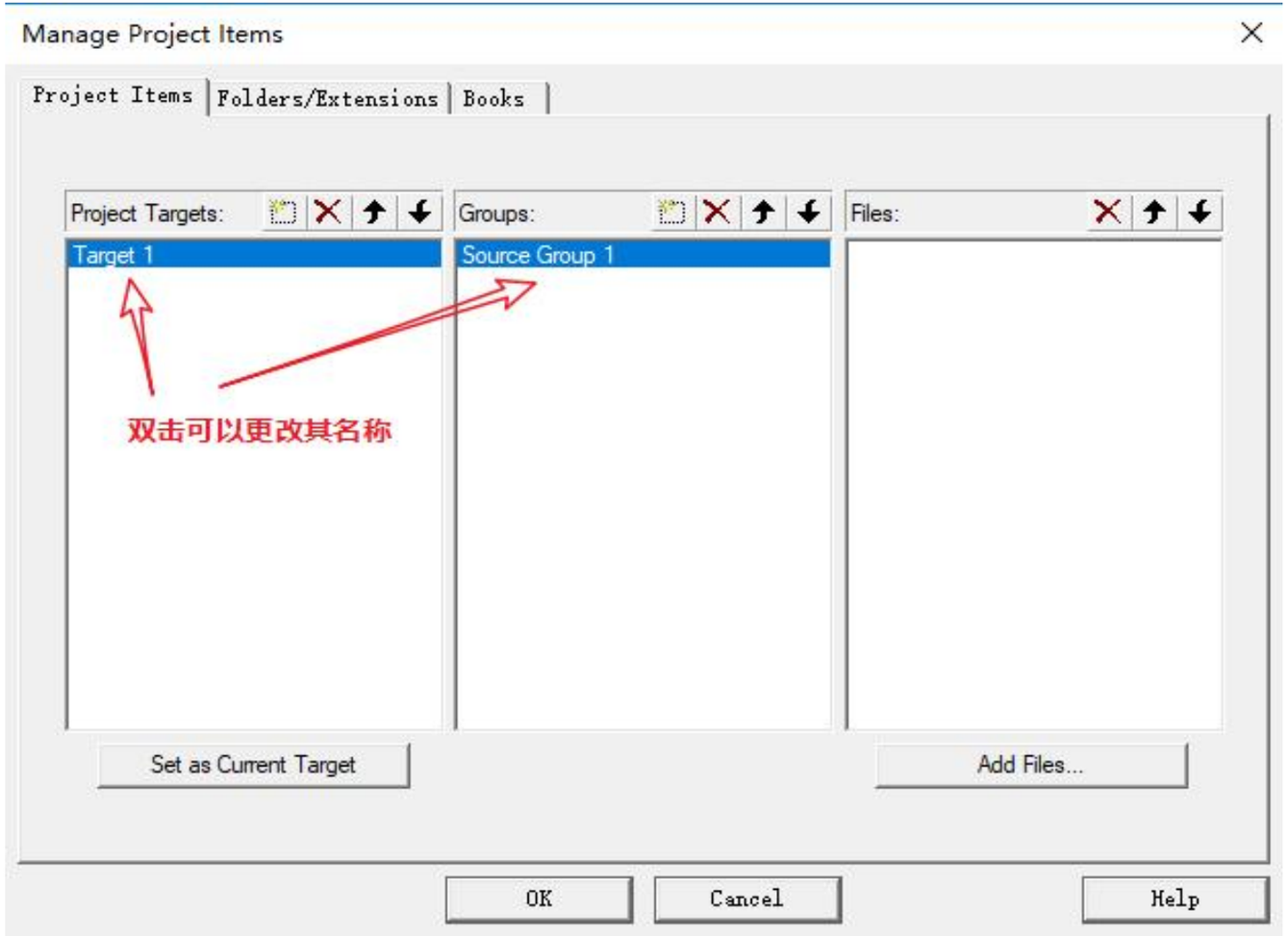
右键单击左边的工程“Target1”，选择“Manage Project Items”



3.4.2 基于标准外设库新建工程模板

步骤4： 把内核、启动文件、外设等相关的库文件添加进Keil工程

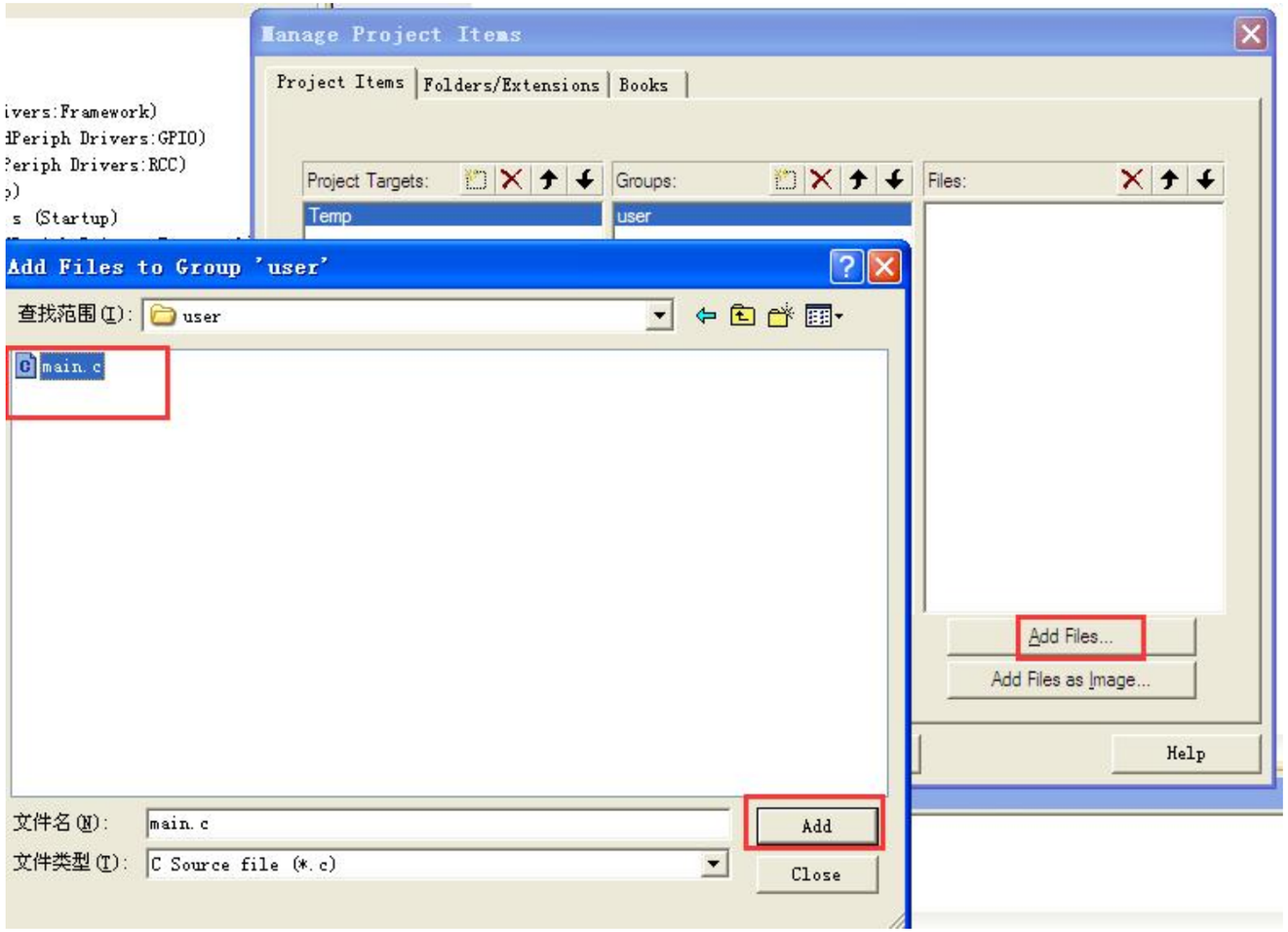
在弹出的界面中，更改工程名和源文件组名，分别双击“Target1”和“source groups”即可更改其名字，其中Project Targets为工程目标的名字，可以根据项目名称定义一个有意义的名称，Groups为项目组名称，更改或添加工程模板里的USER、BSP、FWLib、CORE等



3.4.2 基于标准外设库新建工程模板

步骤4： 把内核、启动文件、外设等相关的库文件添加进Keil工程

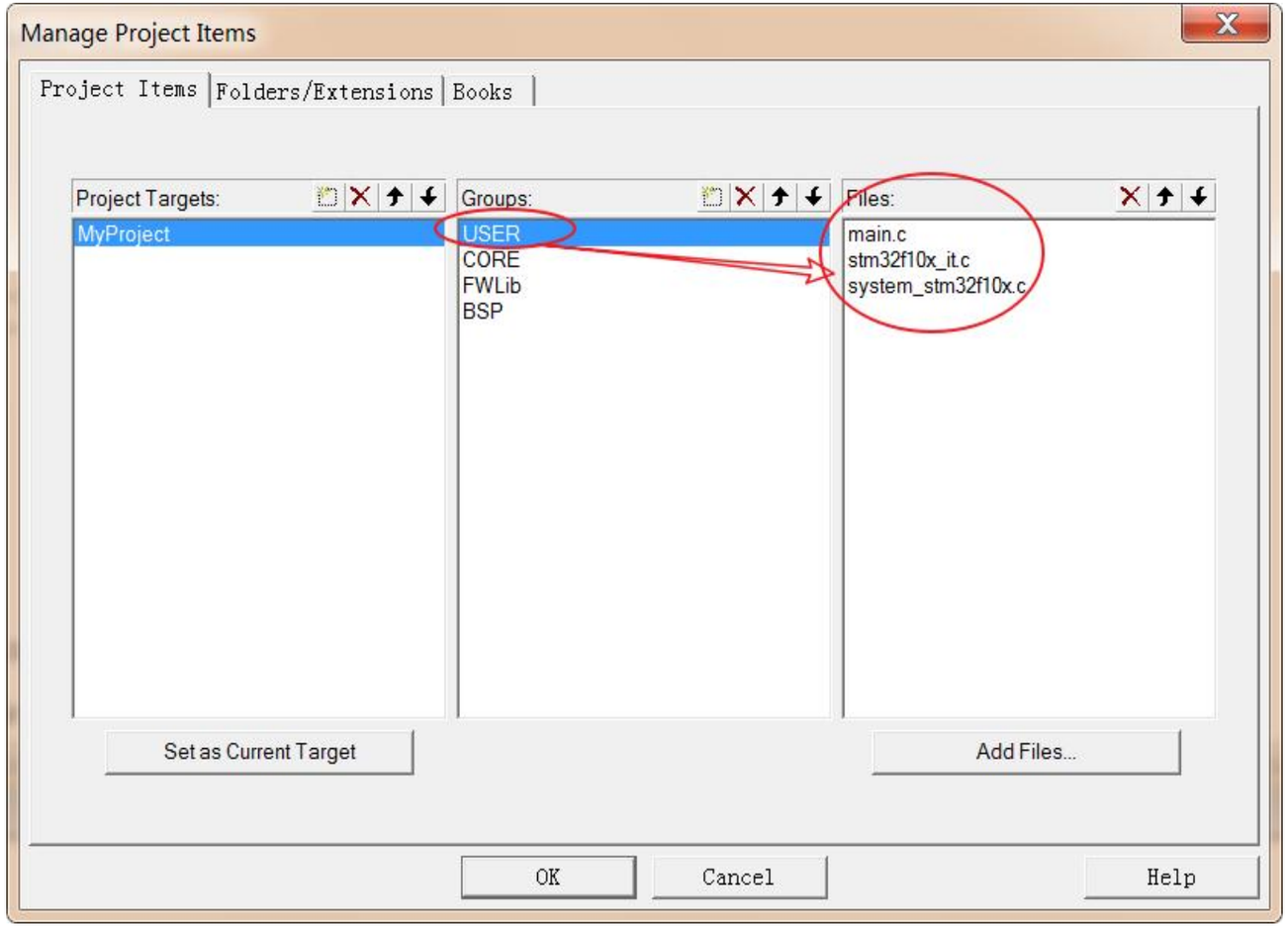
然后单击“Add Files”，添加新文件到源文件组。如添加之前新建的main.c



3.4.2 基于标准外设库新建工程模板

步骤4： 把内核、启动文件、外设等相关的库文件添加进Keil工程

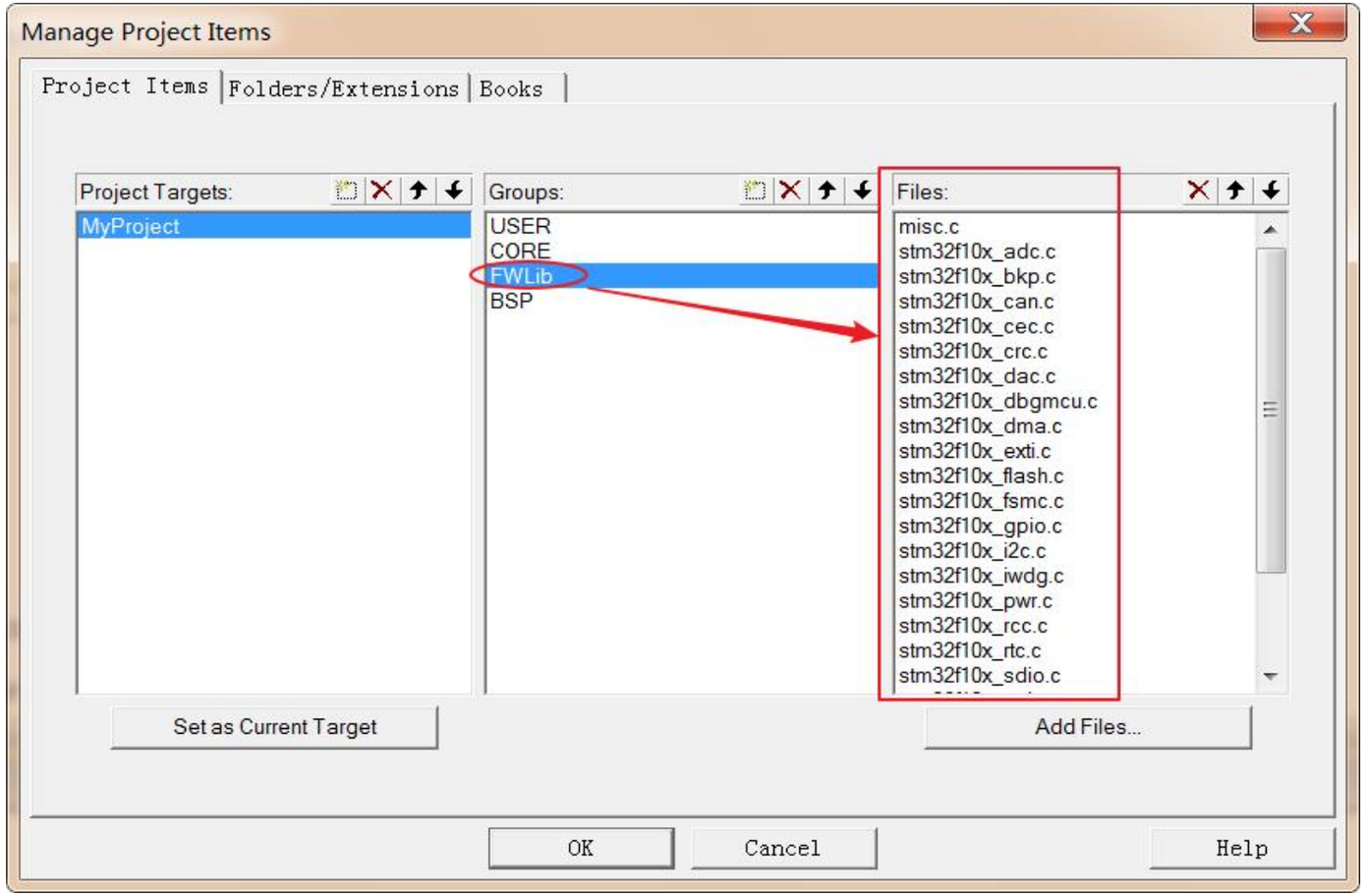
依次将FWLib、USER中的文件添加进工程。添加完成之后的结构如图所示，单击“OK”按钮，完成添加。



3.4.2 基于标准外设库新建工程模板

步骤4： 把内核、启动文件、外设等相关的库文件添加进Keil工程

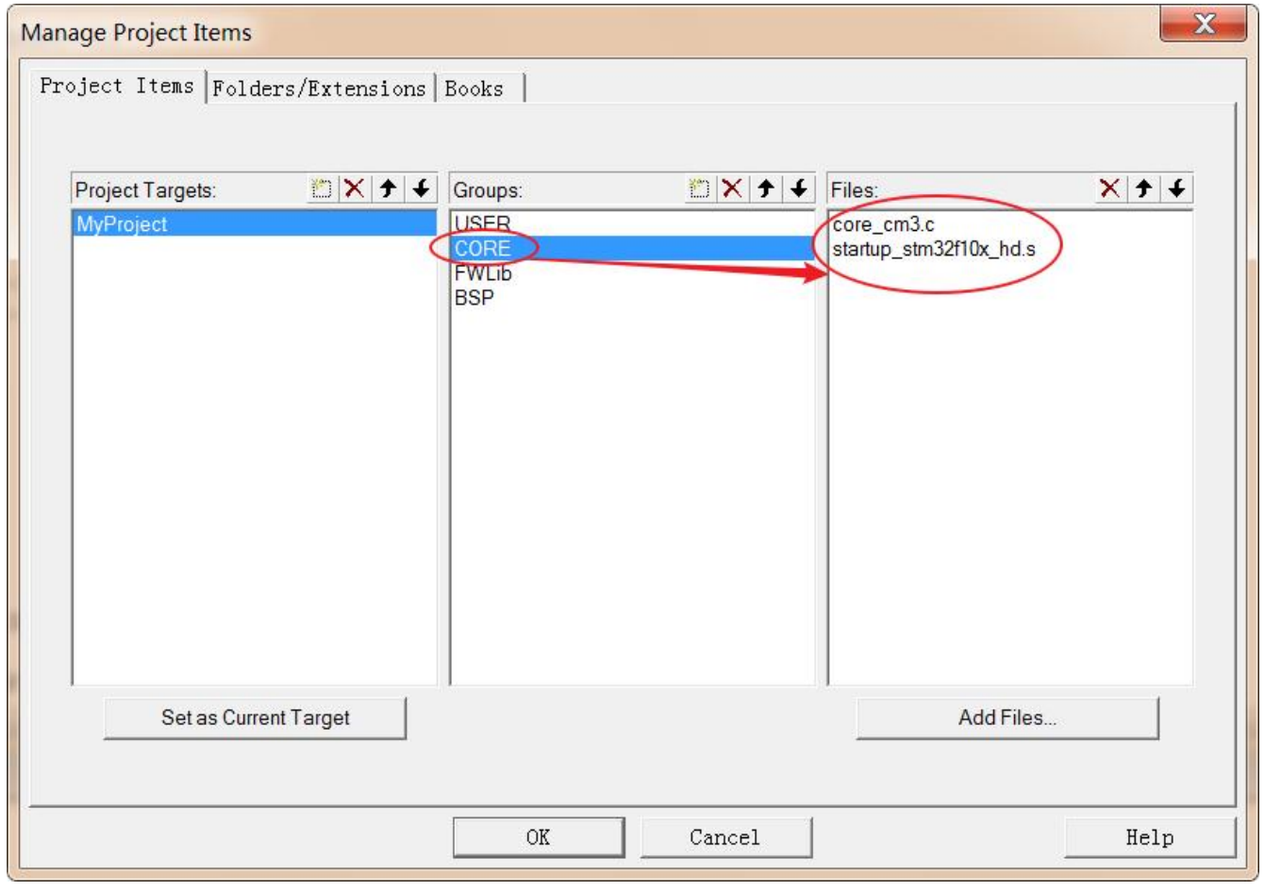
添加完成后的FWLib文件组的结构



3.4.2 基于标准外设库新建工程模板

步骤4： 把内核、启动文件、外设等相关的库文件添加进Keil工程

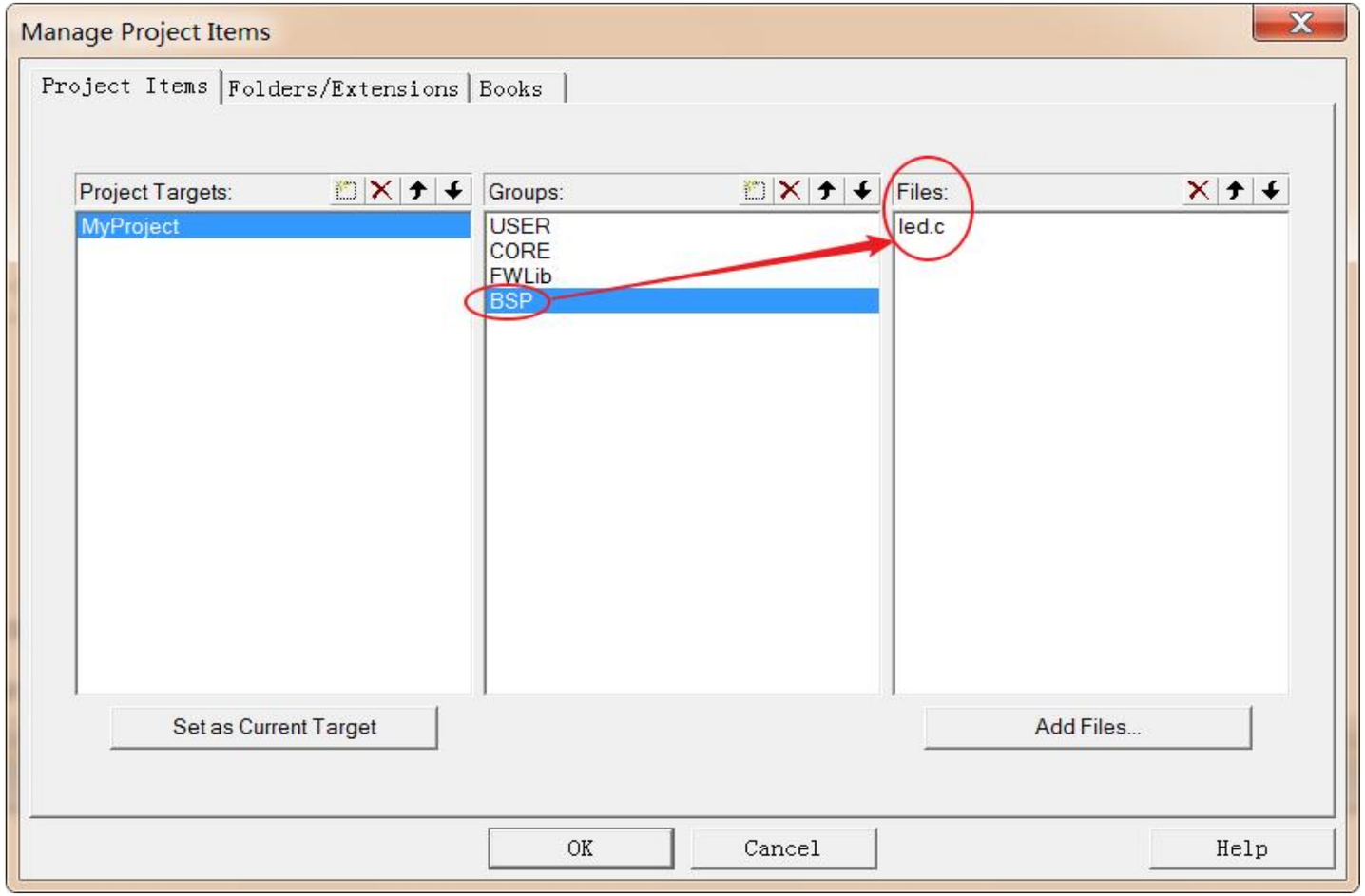
添加完成后的CORE文件组的结构



3.4.2 基于标准外设库新建工程模板

步骤4：把内核、启动文件、外设等相关的库文件添加进Keil工程

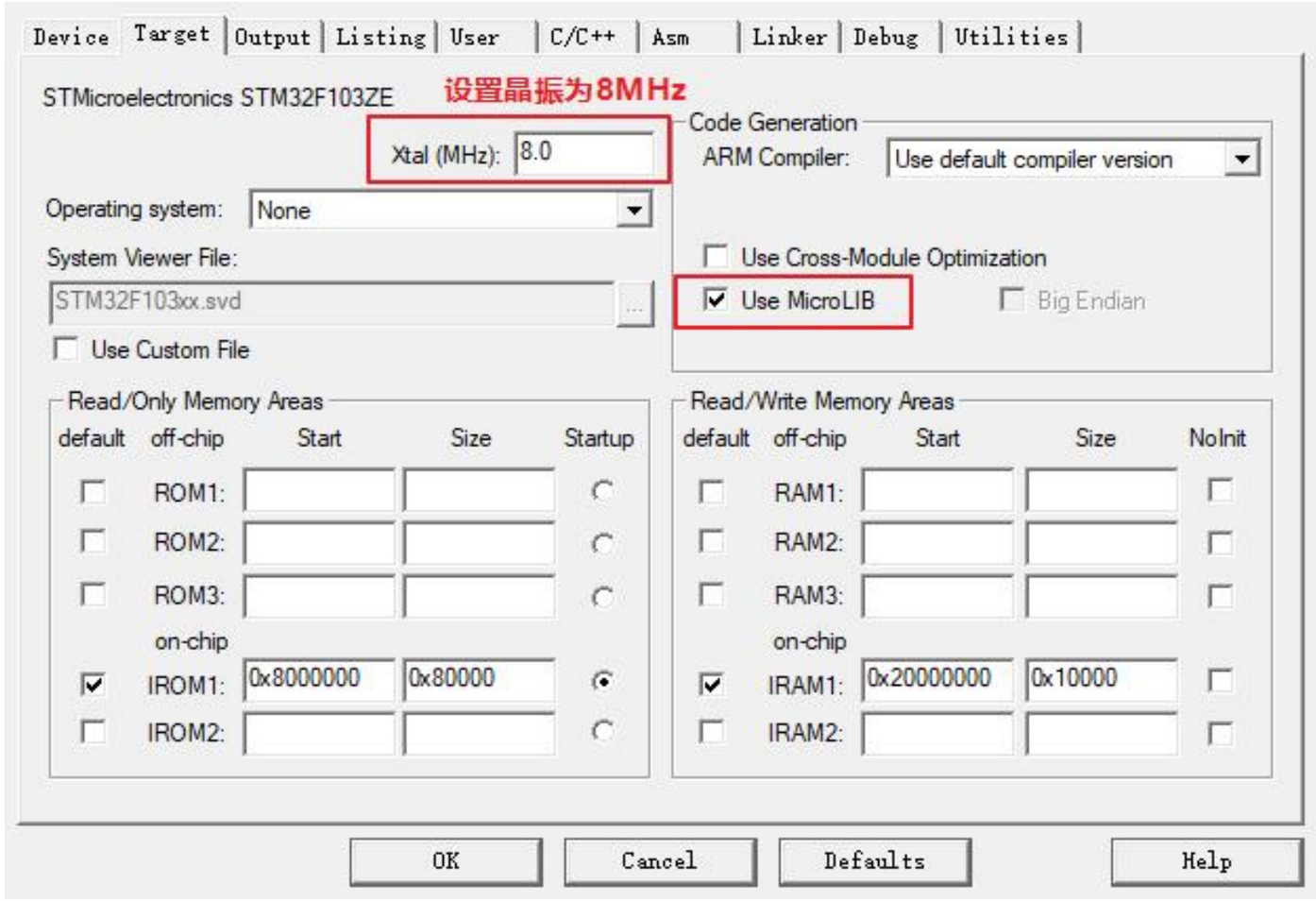
添加完成后的BSP文件组的结构



3.4.2 基于标准外设库新建工程模板

步骤5：配置Keil MDK

单击“options for target”，在“Target”选项页中设置晶振为8MHz，并勾选“Use MicroLIB”(使用Micro库)

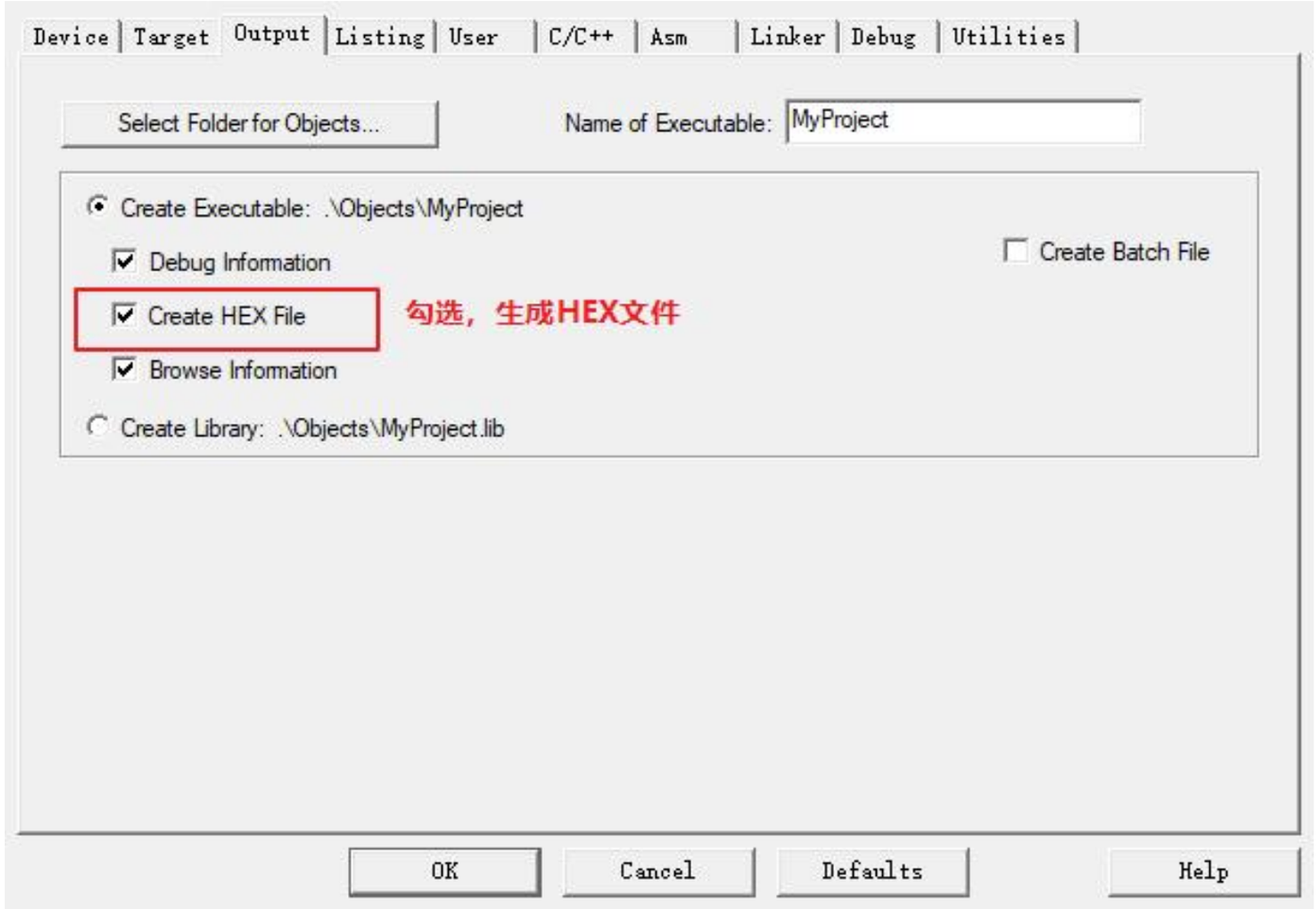


设置晶振大小，勾选“Use MicroLIB”

3.4.2 基于标准外设库新建工程模板

步骤5：配置Keil MDK

在“Output”选项页中勾选“Create HEX File”，生成HEX文件

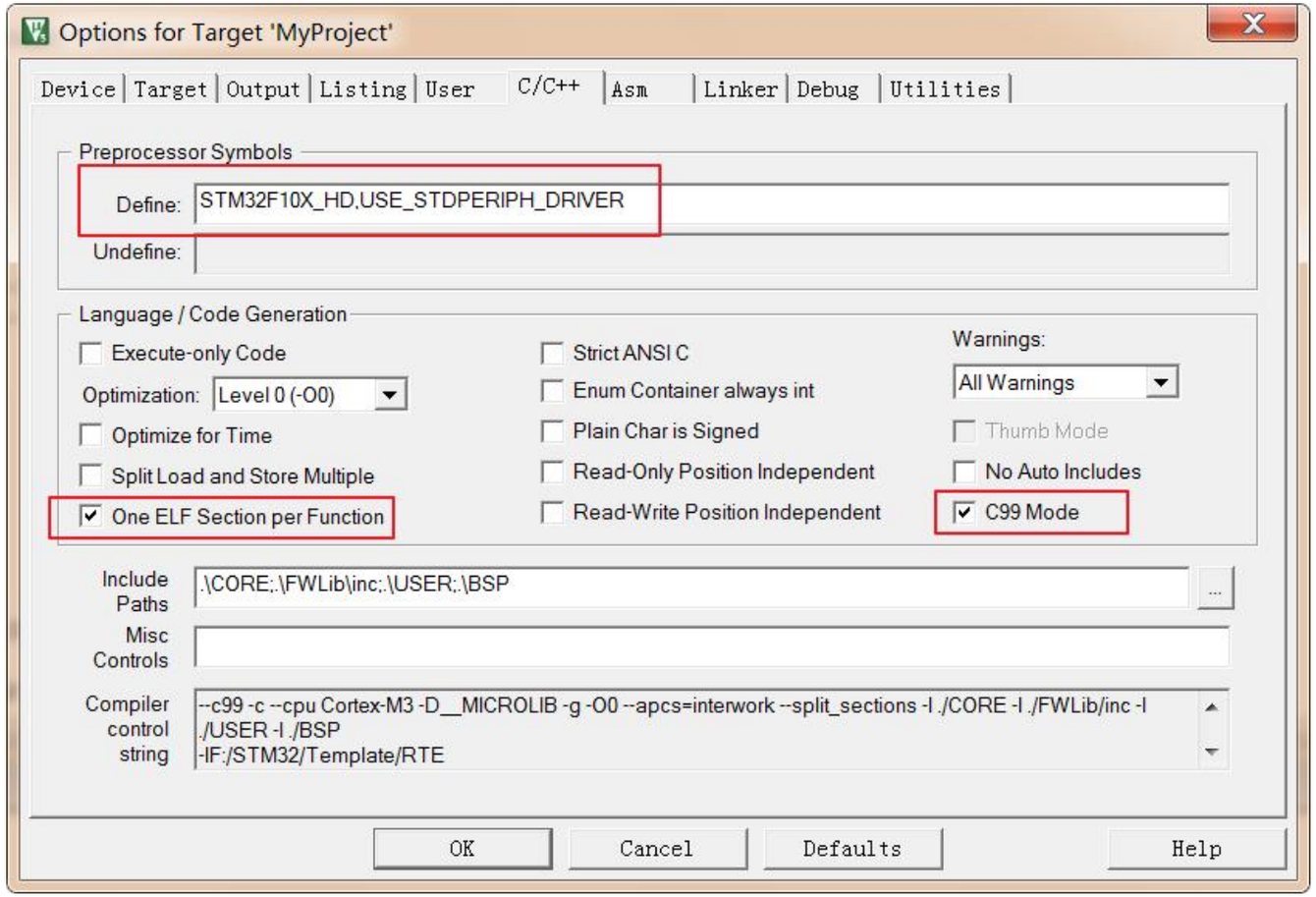


勾选“Create HEX File”，生成HEX文件

3.4.2 基于标准外设库新建工程模板

步骤6： 在工程设置窗口的“C/C++”选项页的“Define”框中输入两个宏

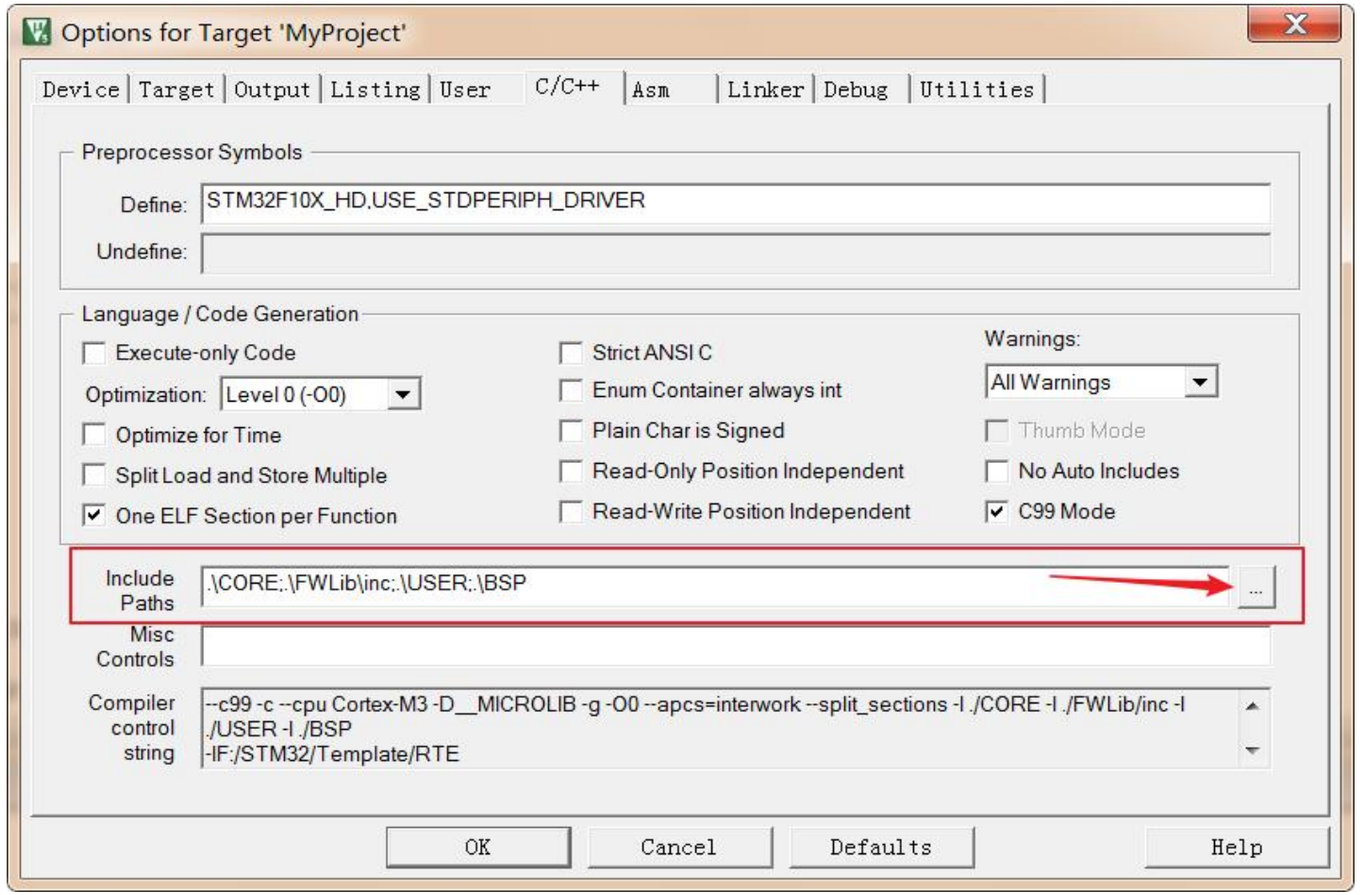
目的： 为了屏蔽编译器的默认搜索路径，转而使用添加到工程中的STM32F10x标准外设库，即使用STM32F10x标准外设库进行STM32的开发



添加宏

3.4.2 基于标准外设库新建工程模板

步骤7：在工程设置窗口的“C/C++”选项页的“Include Paths”中，将工程头文件.h添加进搜索路径中

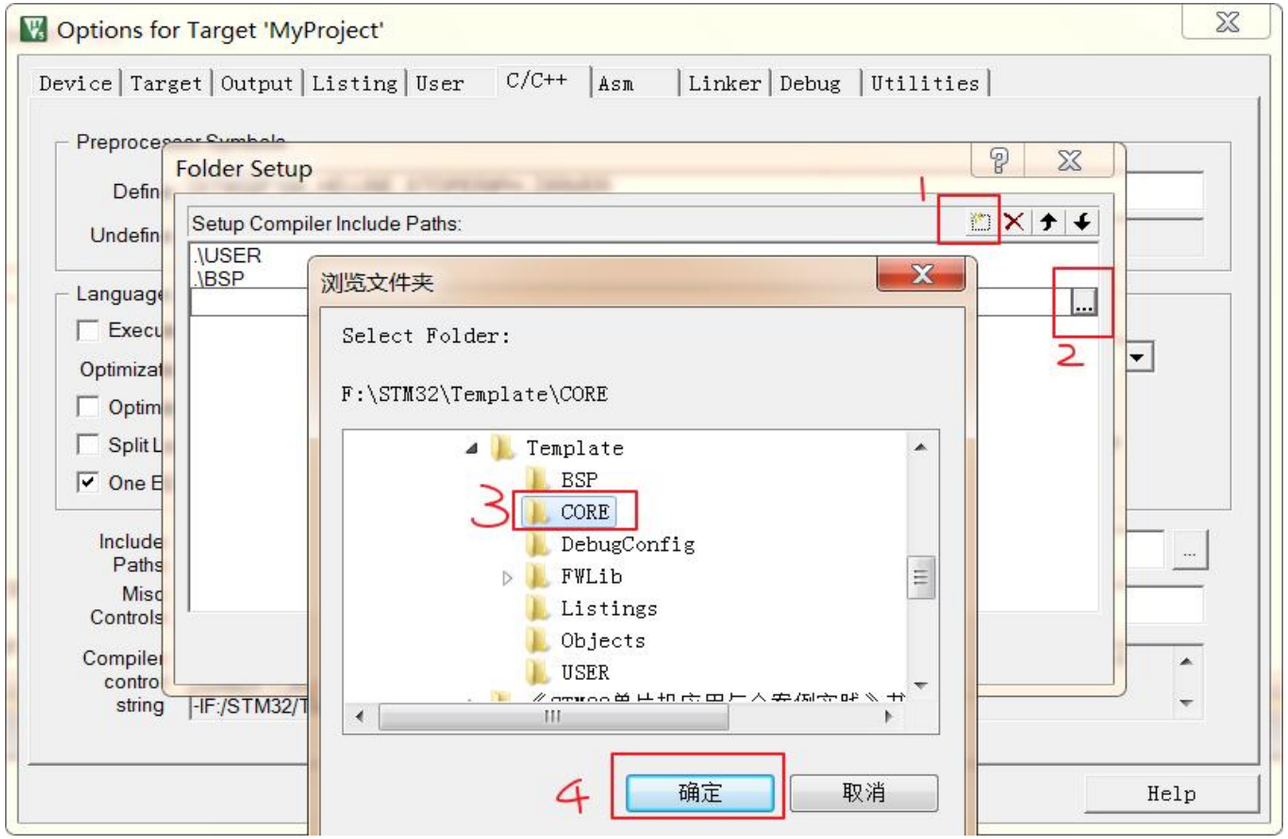


添加搜索路径

3.4.2 基于标准外设库新建工程模板

步骤7：在工程设置窗口的C/C++选项页的Include Paths中，将工程头文件.h添加进搜索路径中

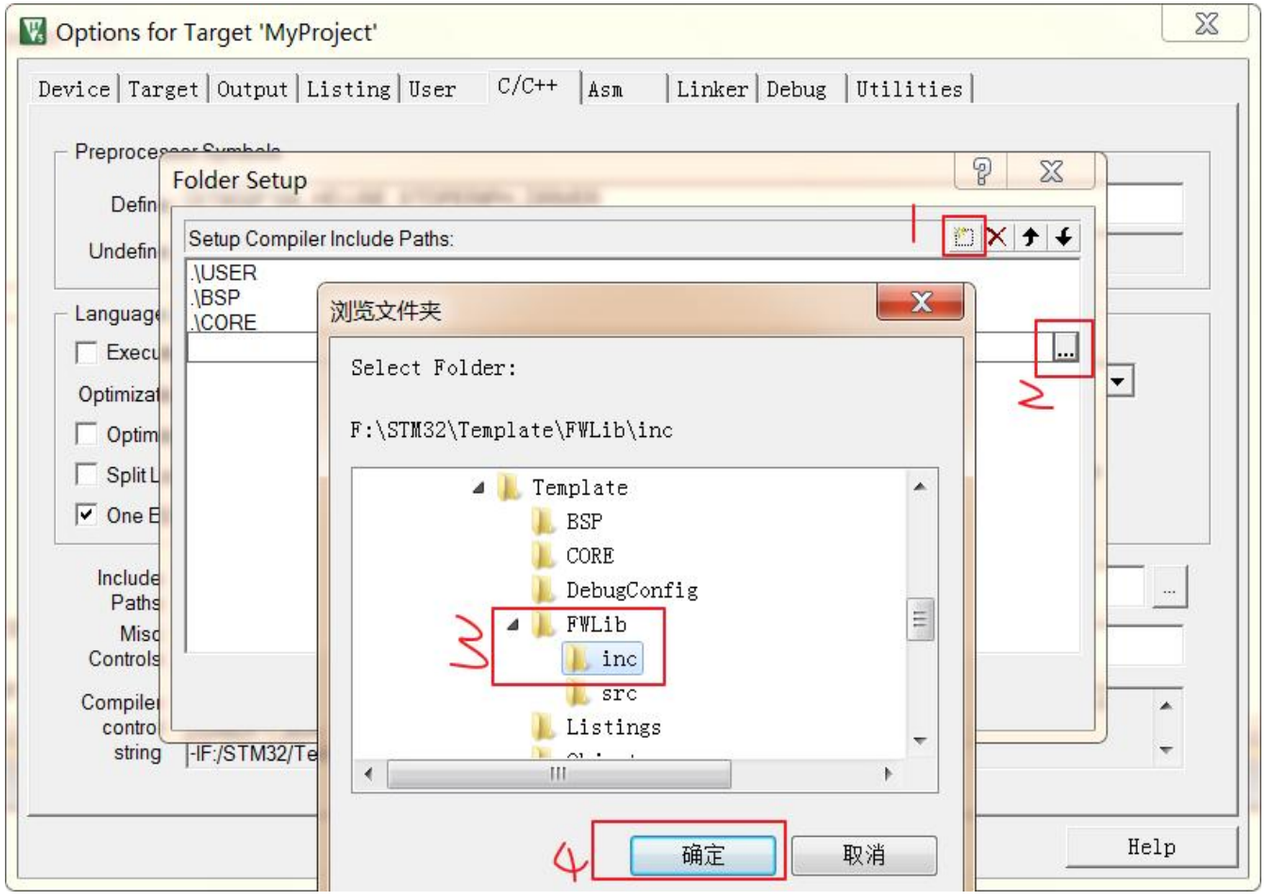
将CORE中的头文件添加进工程搜索路径



3.4.2 基于标准外设库新建工程模板

步骤7：在工程设置窗口的C/C++选项页的Include Paths中，将工程头文件.h添加进搜索路径中

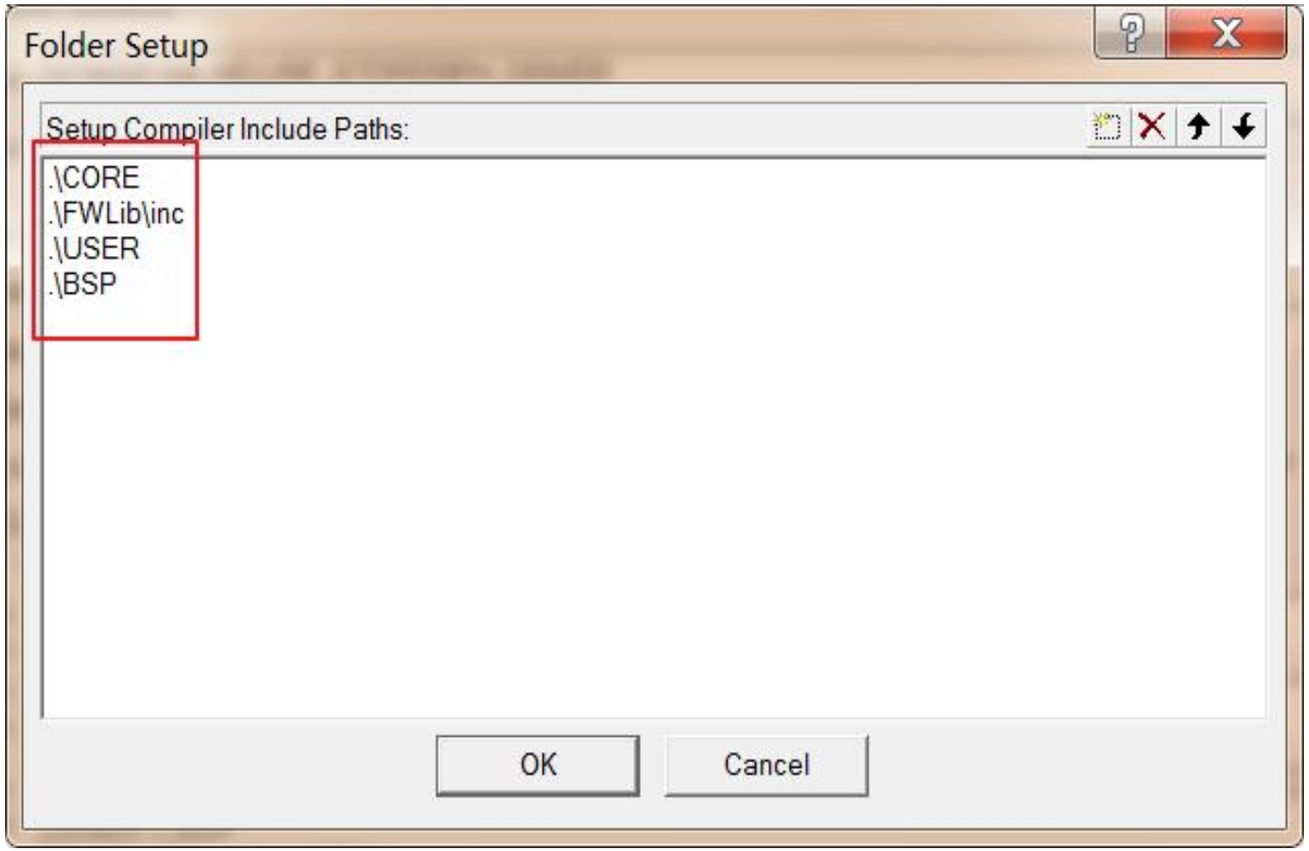
将FWLib中的头文件添加进工程搜索路径



3.4.2 基于标准外设库新建工程模板

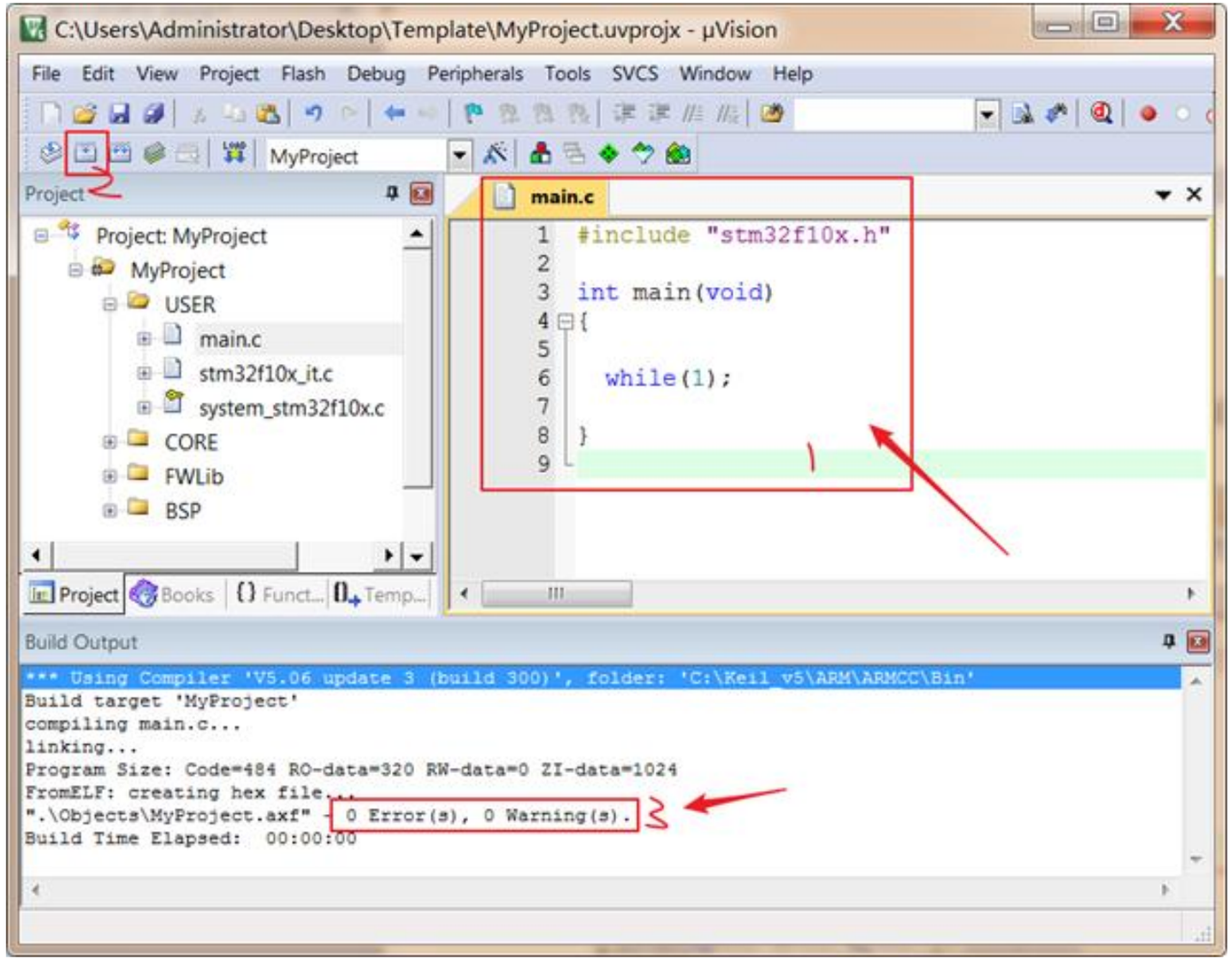
步骤7：在工程设置窗口的C/C++选项页的Include Paths中，将工程头文件.h添加进搜索路径中

将CORE、FWLib、USER、BSP等头文件都添加进工程搜索路径后的结构如图



3.4.2 基于标准外设库新建工程模板

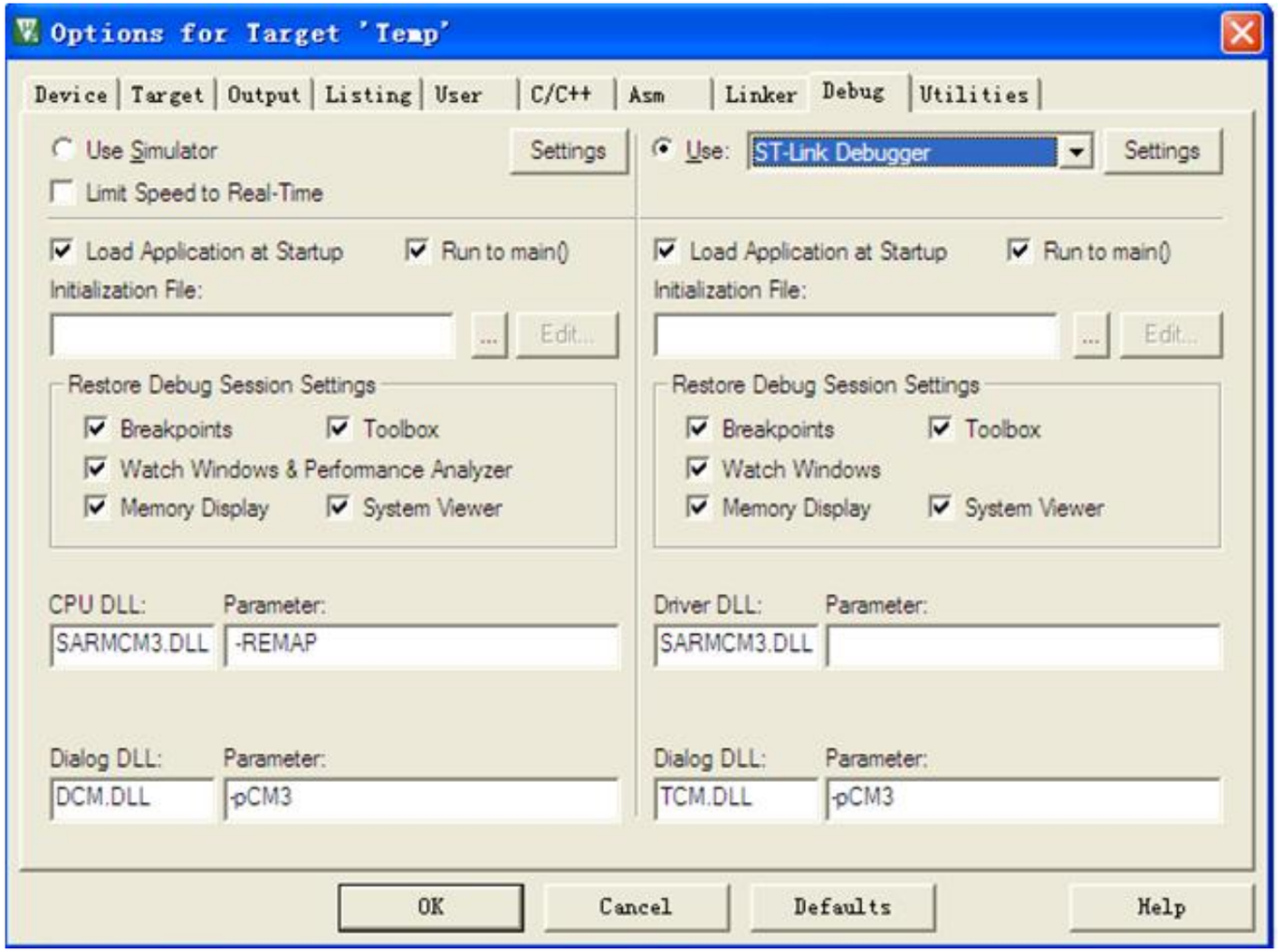
步骤8： 编写main.c代码，点击编译按钮，如果结果为“0 Error(s)”，则新建工程完成



3.4.2 基于标准外设库新建工程模板

步骤9：调试和下载

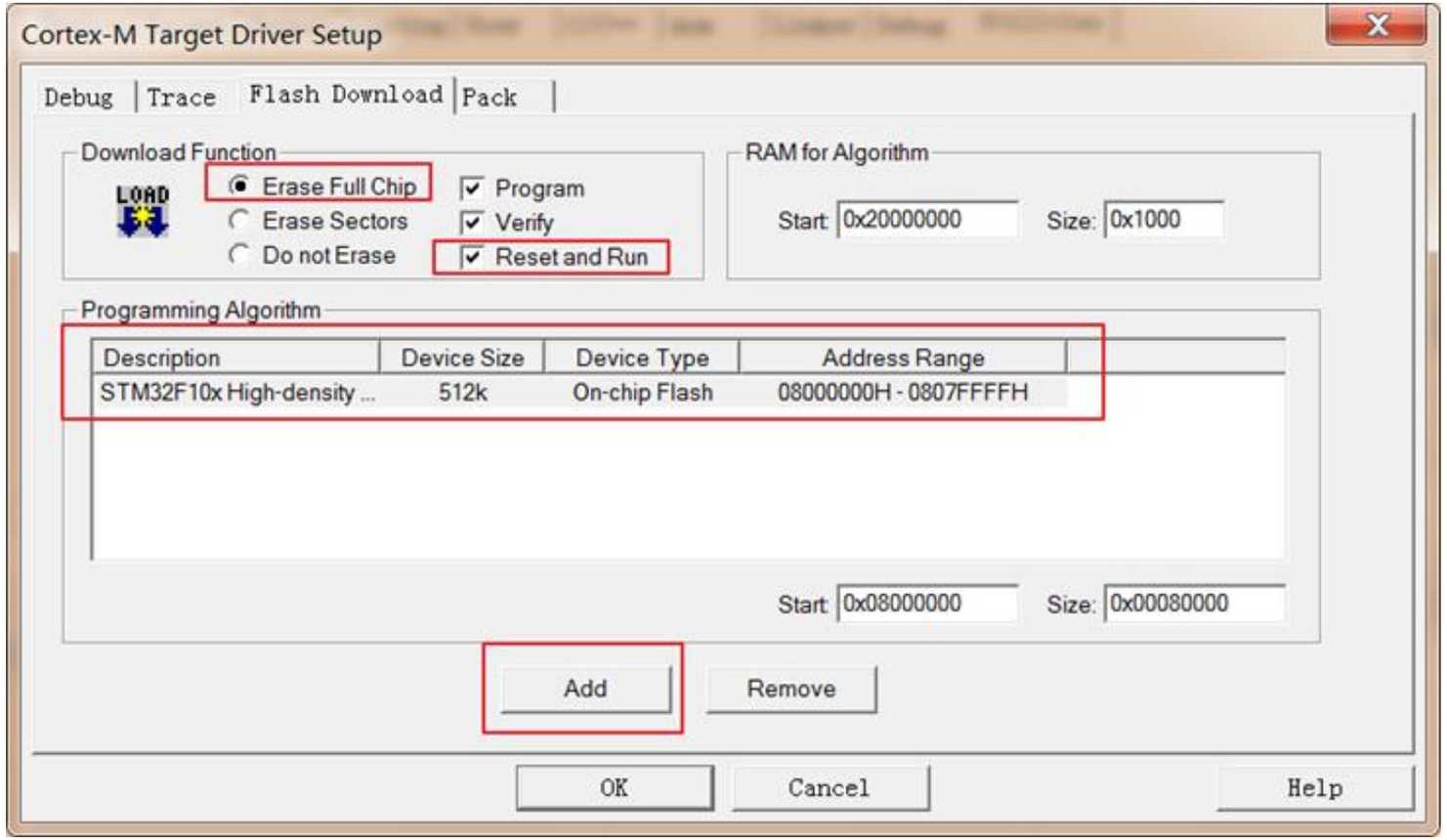
采用ST-Link可以进行代码的在线仿真、烧写程序等，需要安装ST-Link驱动。驱动安装完毕后，连接开发板，在“Debug”选项卡中选择“ST-Link Debugger”



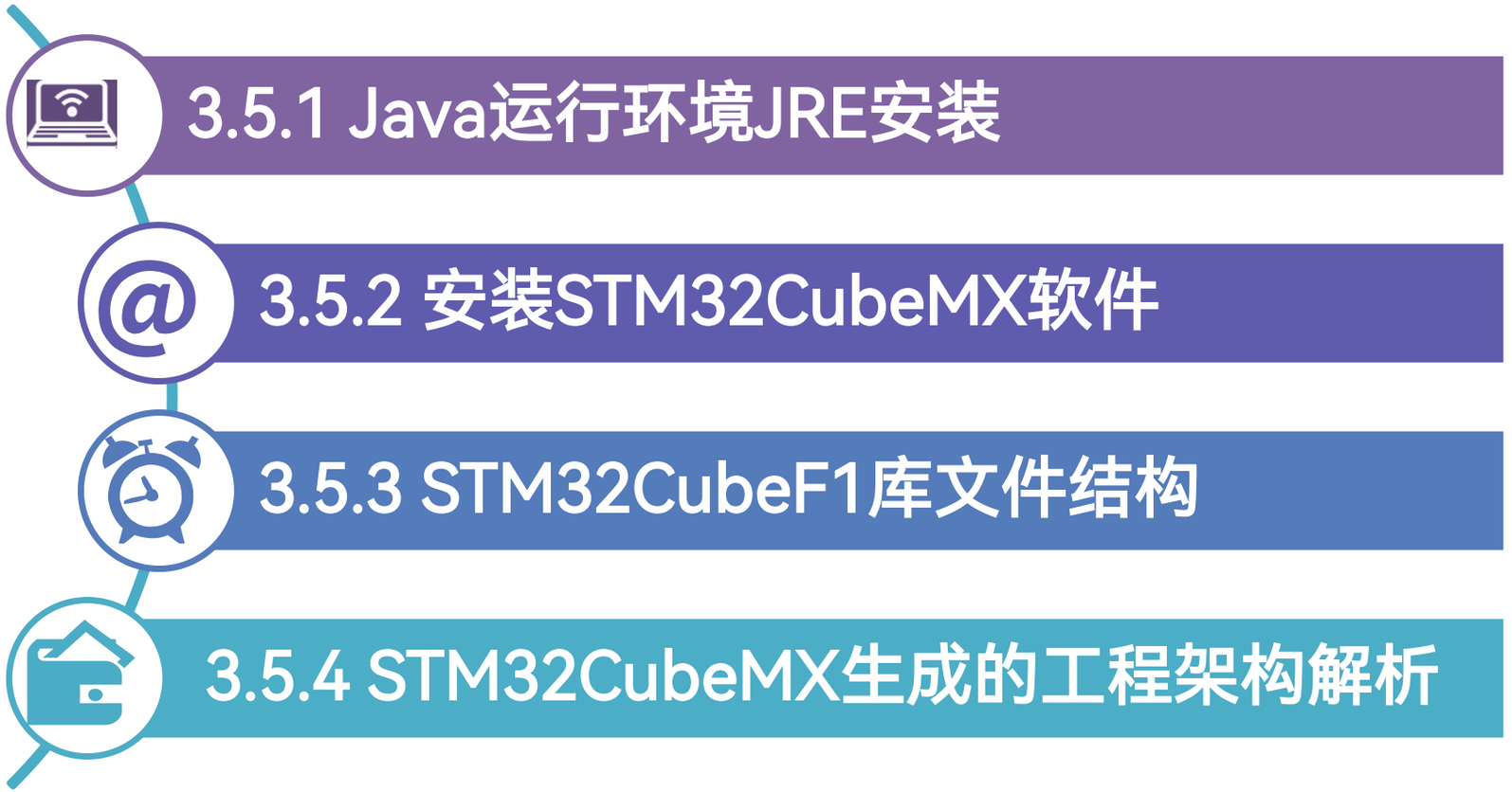
3.4.2 基于标准外设库新建工程模板

步骤9：调试和下载

单击Settings按钮，在弹出的界面中勾选中“Reset and Run”，选择“Erase Full Chip”，每次烧写都将擦除掉芯片上已有的内容

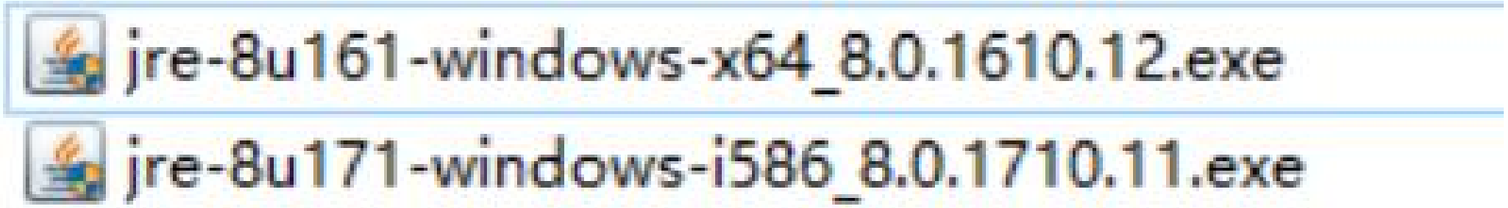


3.5 基于HAL库开发平台的搭建



3.5.1 Java运行环境JRE安装

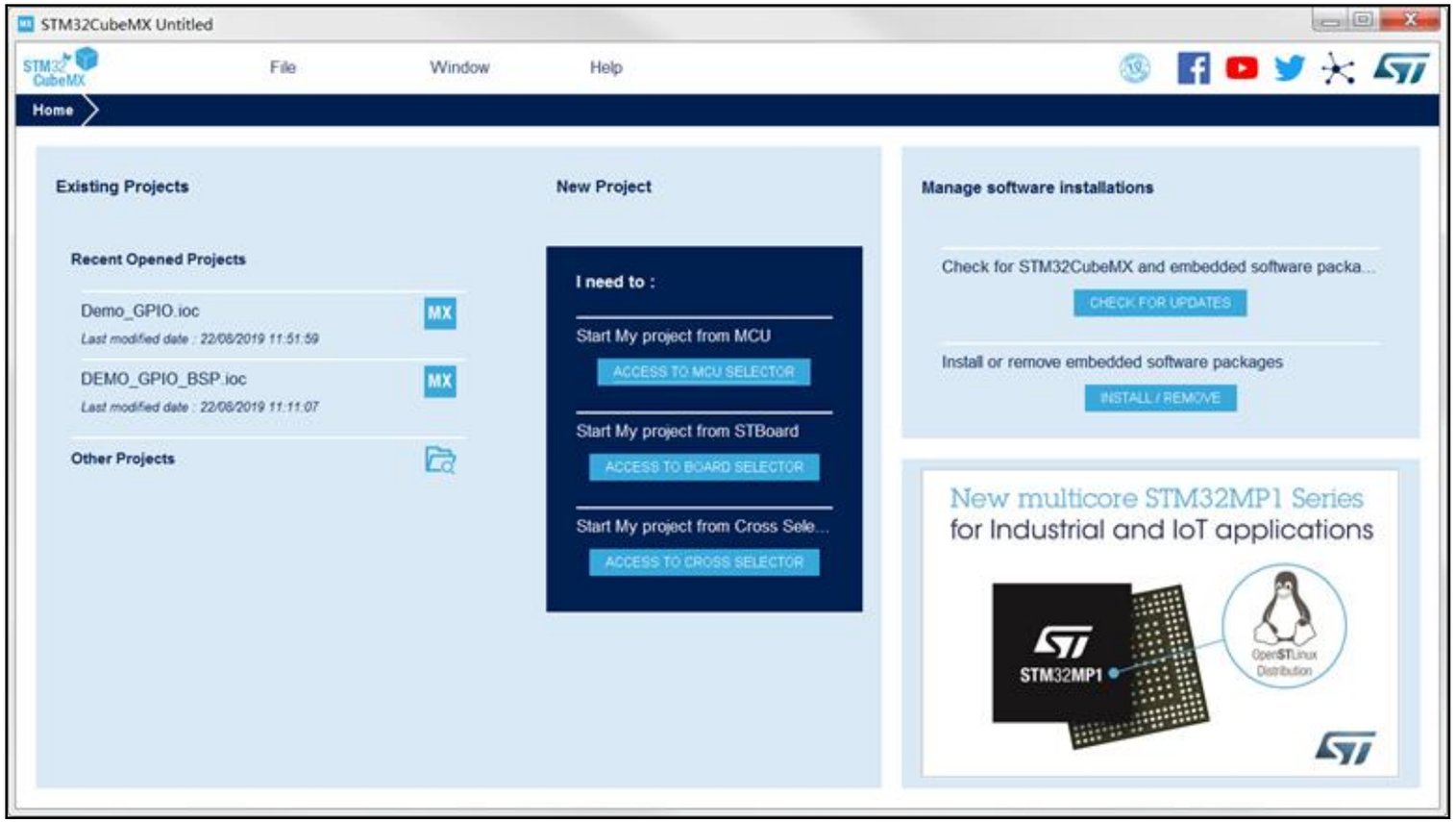
STM32CubeMX软件是基于Java环境运行的，需要先安装JRE（Java Run Time Environment），可通过Java官网下载。用户可根据操作系统选择64位的还是32位的，如图所示。



3.5.2 安装STM32CubeMX软件

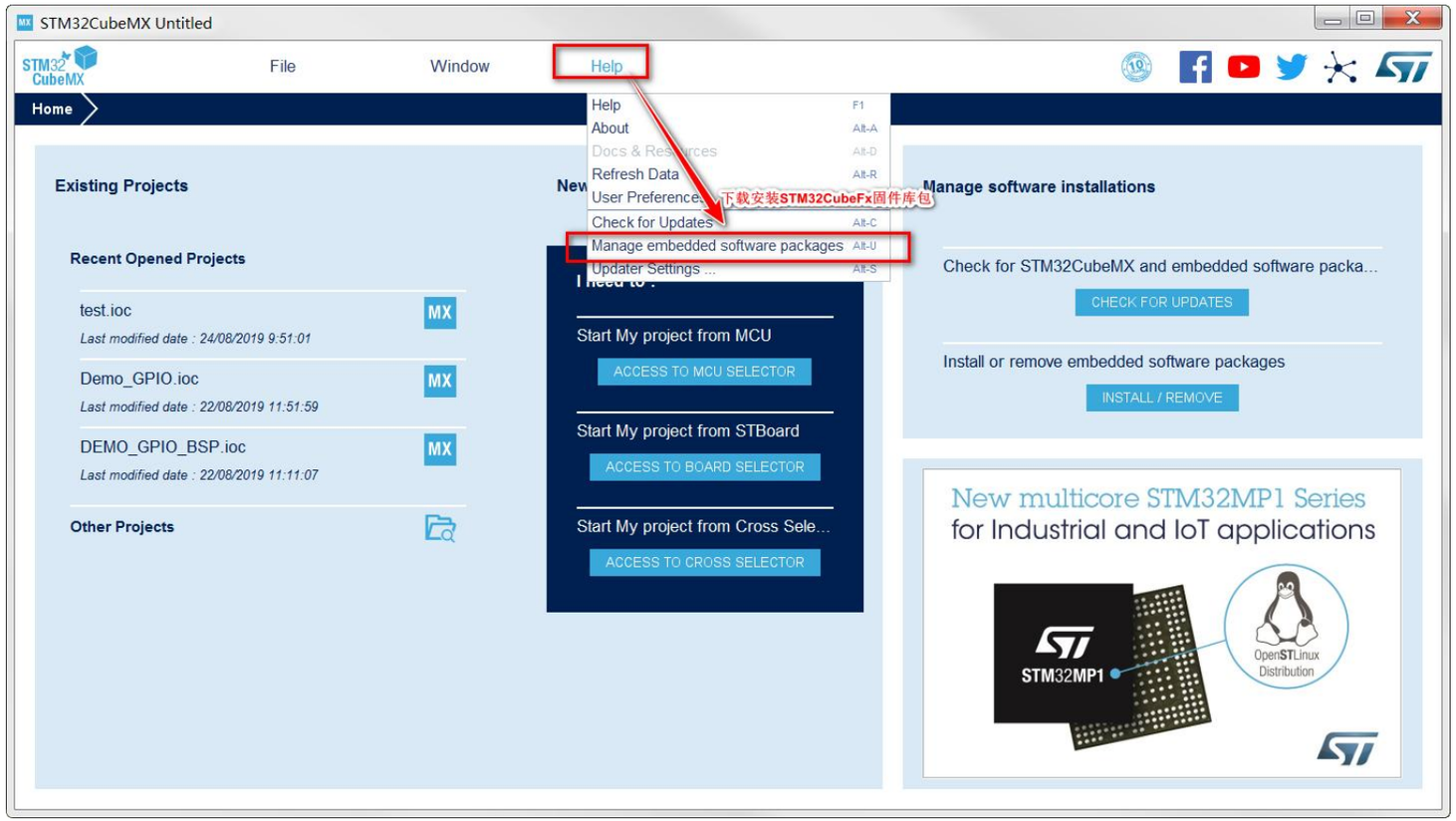
STM32CubeMX软件可从ST官网下载，当前版本是5.3.0。

安装STM32CubeMX成功后，打开软件，显示如图所示界面。



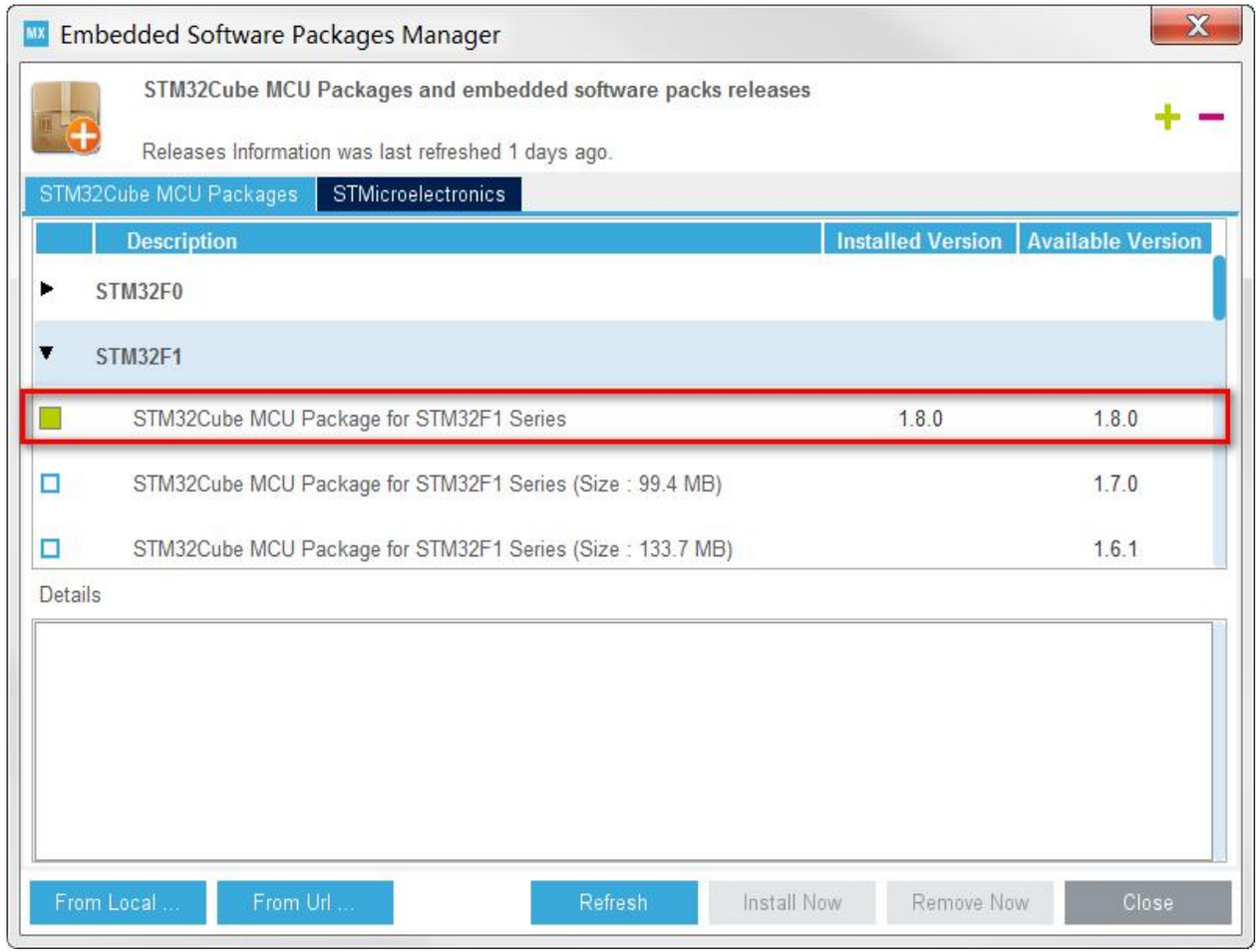
3.5.2 安装STM32CubeMX软件

安装相应的STM32Cube软件固件包，通过STM32CubeMX的“Help”菜单栏，找到“Manage embedded software packages”，如图所示。



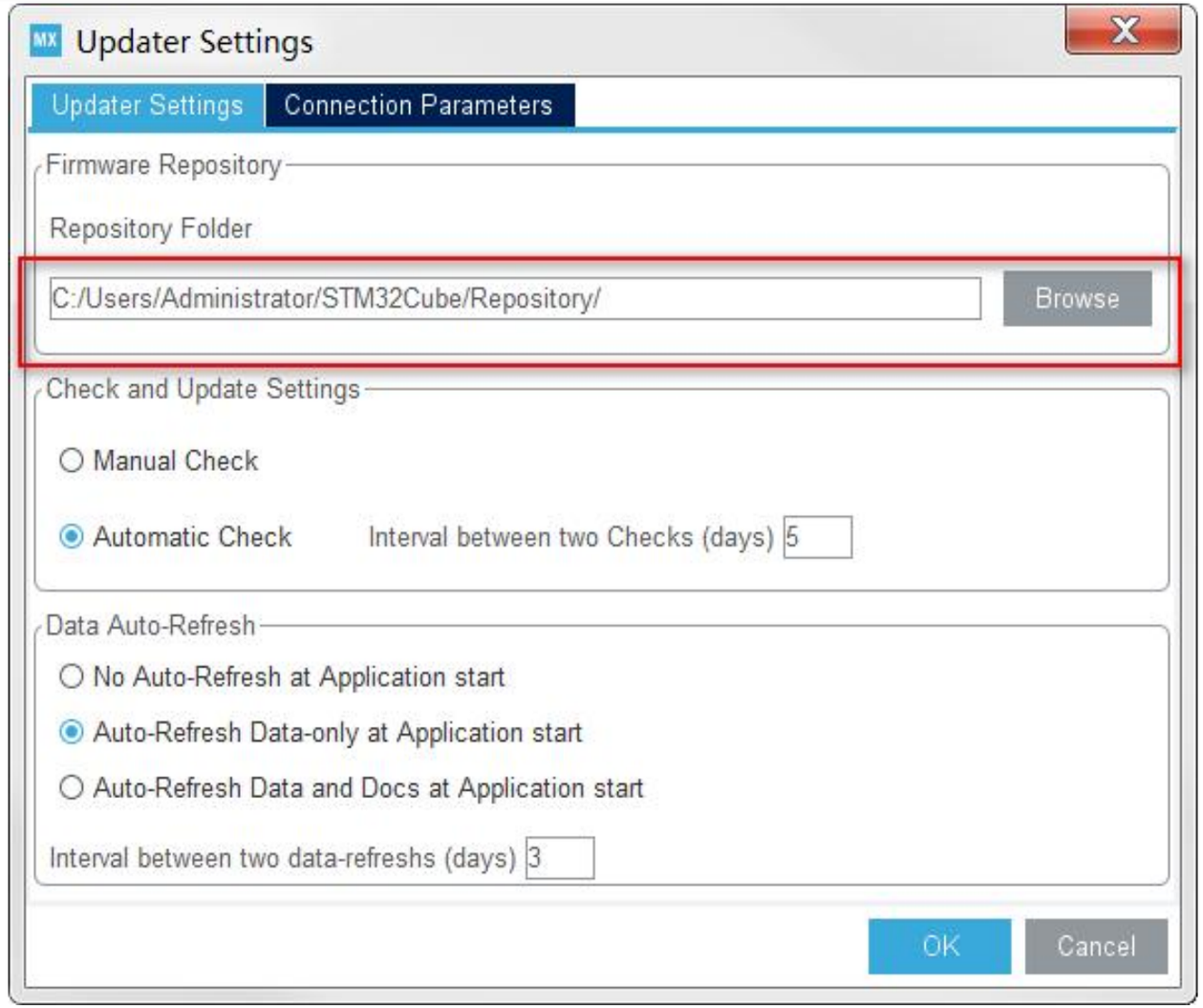
3.5.2 安装STM32CubeMX软件

选择进行在线下载STM32CubeFx库，这里选择STM32CubeF1下载



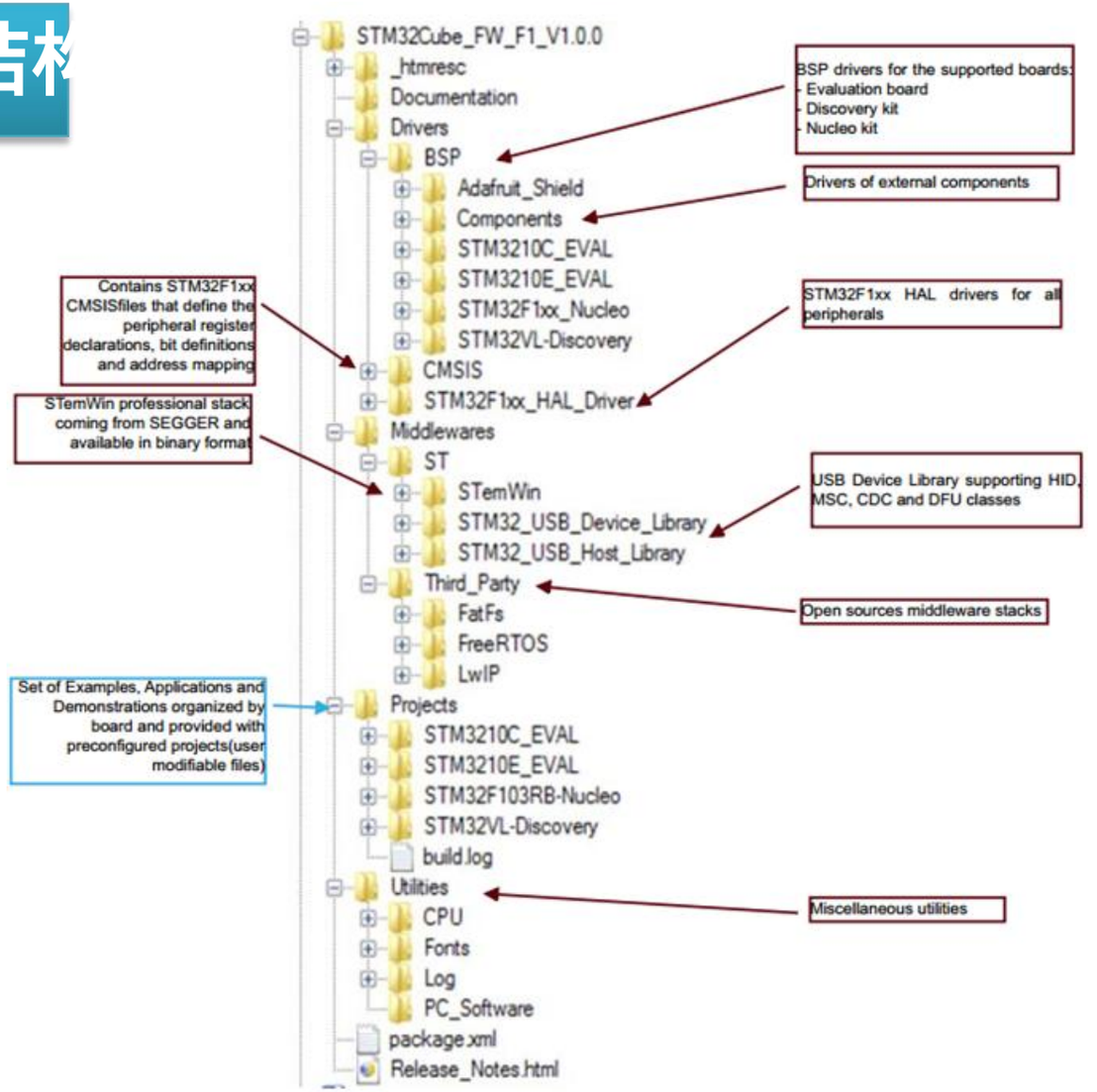
3.5.2 安装STM32CubeMX软件

查看STM32CubeF1库的文件结构，在“Help”→“Updater Settings”菜单栏下的“Repository Folder”可查看到在线下载的HAL库存放的位置，这个位置可以更改



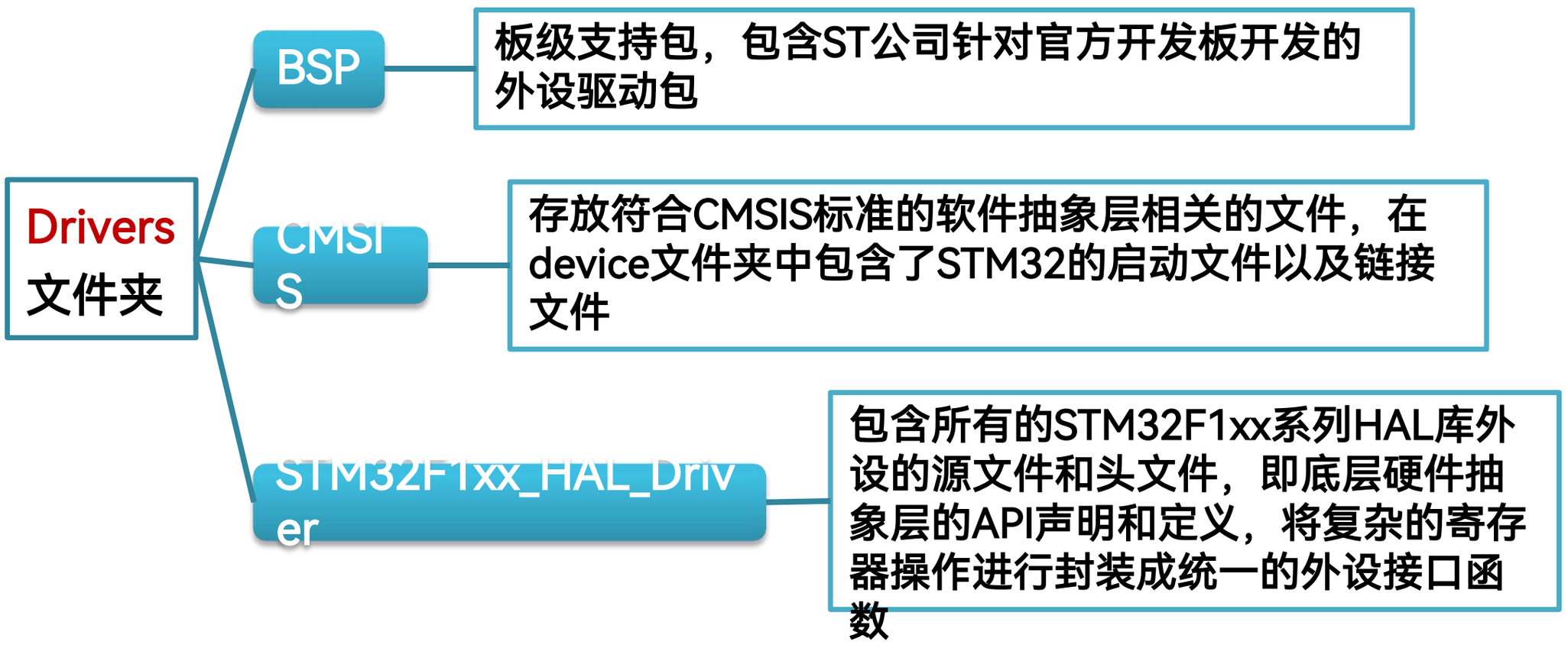
3.5.3 STM32CubeF1库文件结构

Documentation文件夹包含了一个PDF格式的STM32CubeF1用户说明手册，对STM32CubeF1的固件库包进行了介绍和使用说明



STM32CubeF1库的文件结构图

3.5.3 STM32CubeF1库文件结构



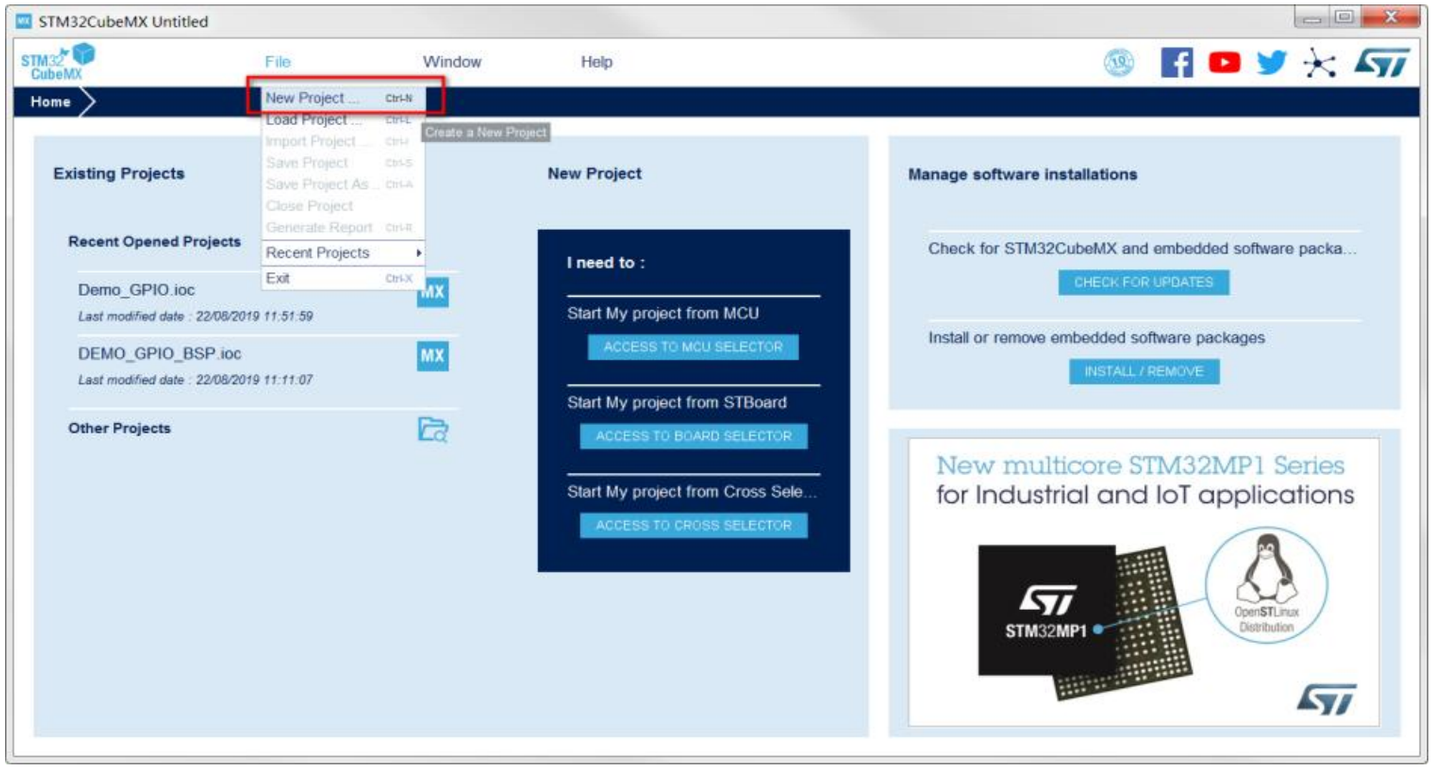
3.5.3 STM32CubeF1库文件结构

Middlewares文件夹	Projects文件夹	Utilities文件夹
包含ST和Third_Party两个子文件夹：ST文件夹下存放的是STM32相关的一些文件；Third_Party文件夹为第三方中间件	ST官方推出的基于官方开发板的可以直接编译的工程实例	一些实用的其他功能函数库

3.5.4 STM32CubeMX生成的工程架构解析

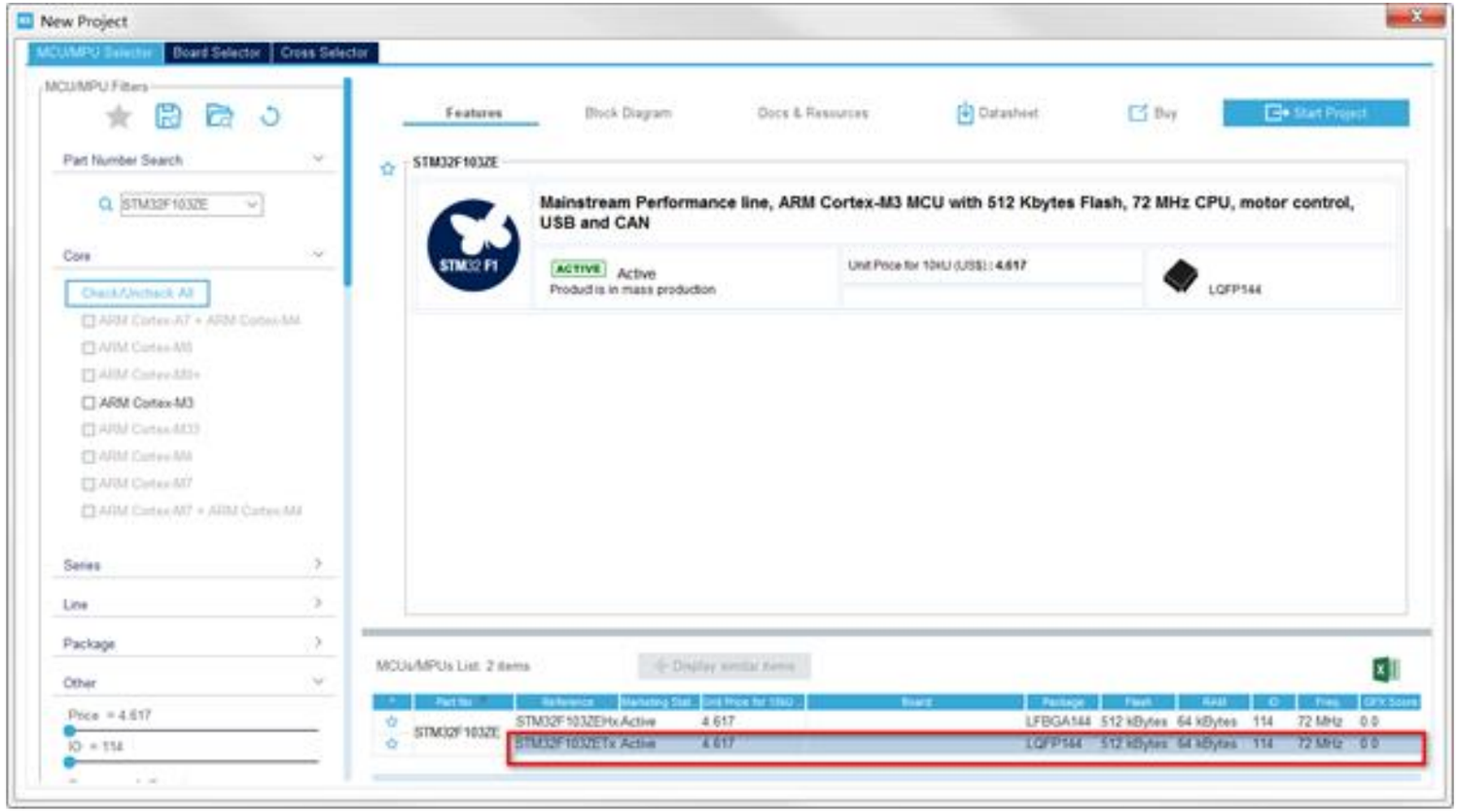
利用STM32CubeMX生成一个MDK-Keil 工程

步骤1：打开STM32CubeMX软件，新建一个Project工程



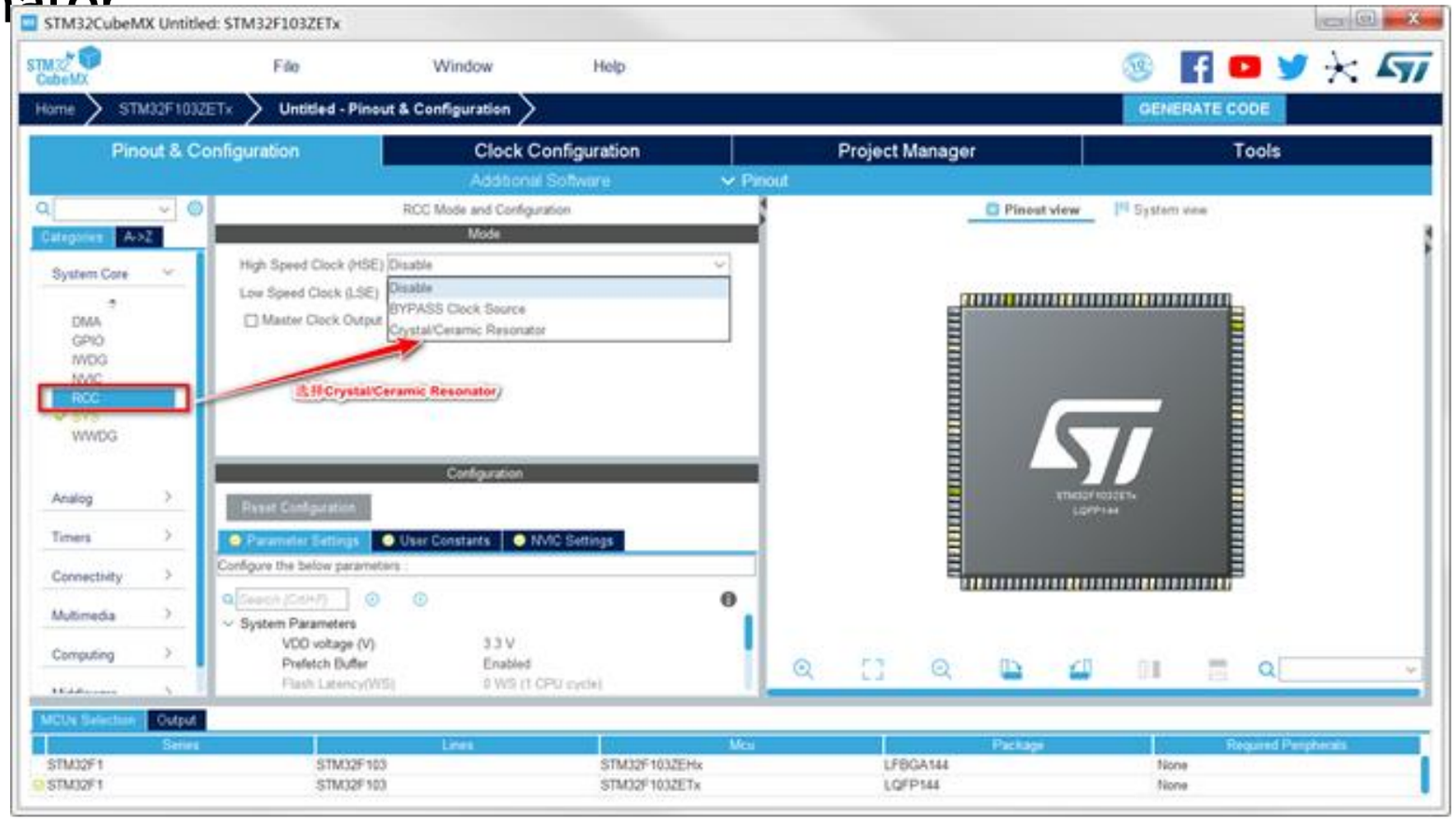
3.5.4 STM32CubeMX生成的工程架构解析

步骤2： 选择MCU。本实例采用STM32F103ZET6



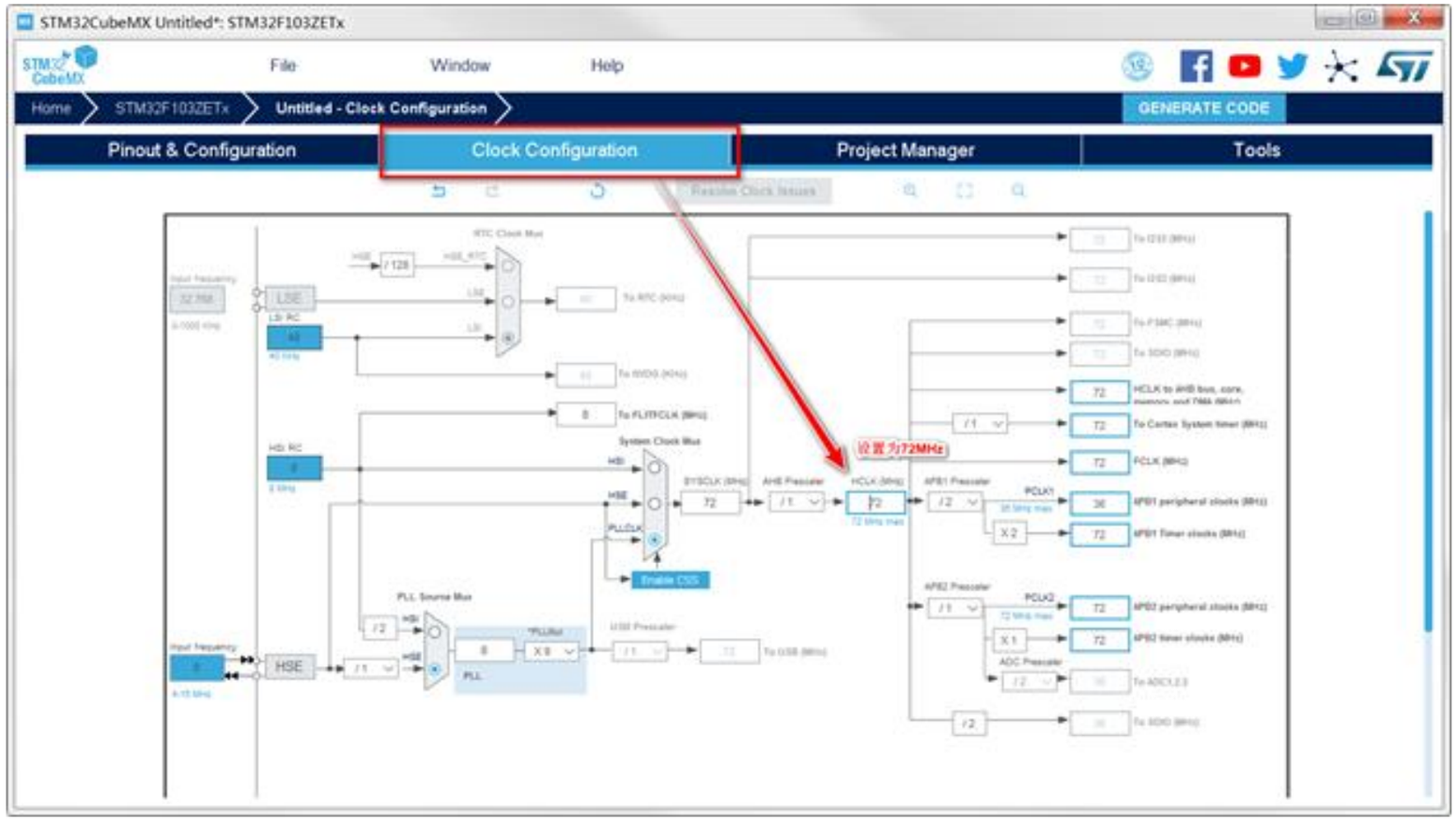
3.5.4 STM32CubeMX生成的工程架构解析

步骤3： 配置RCC，选择高速外部时钟HSE为“Crystal/Ceramic Resonator”



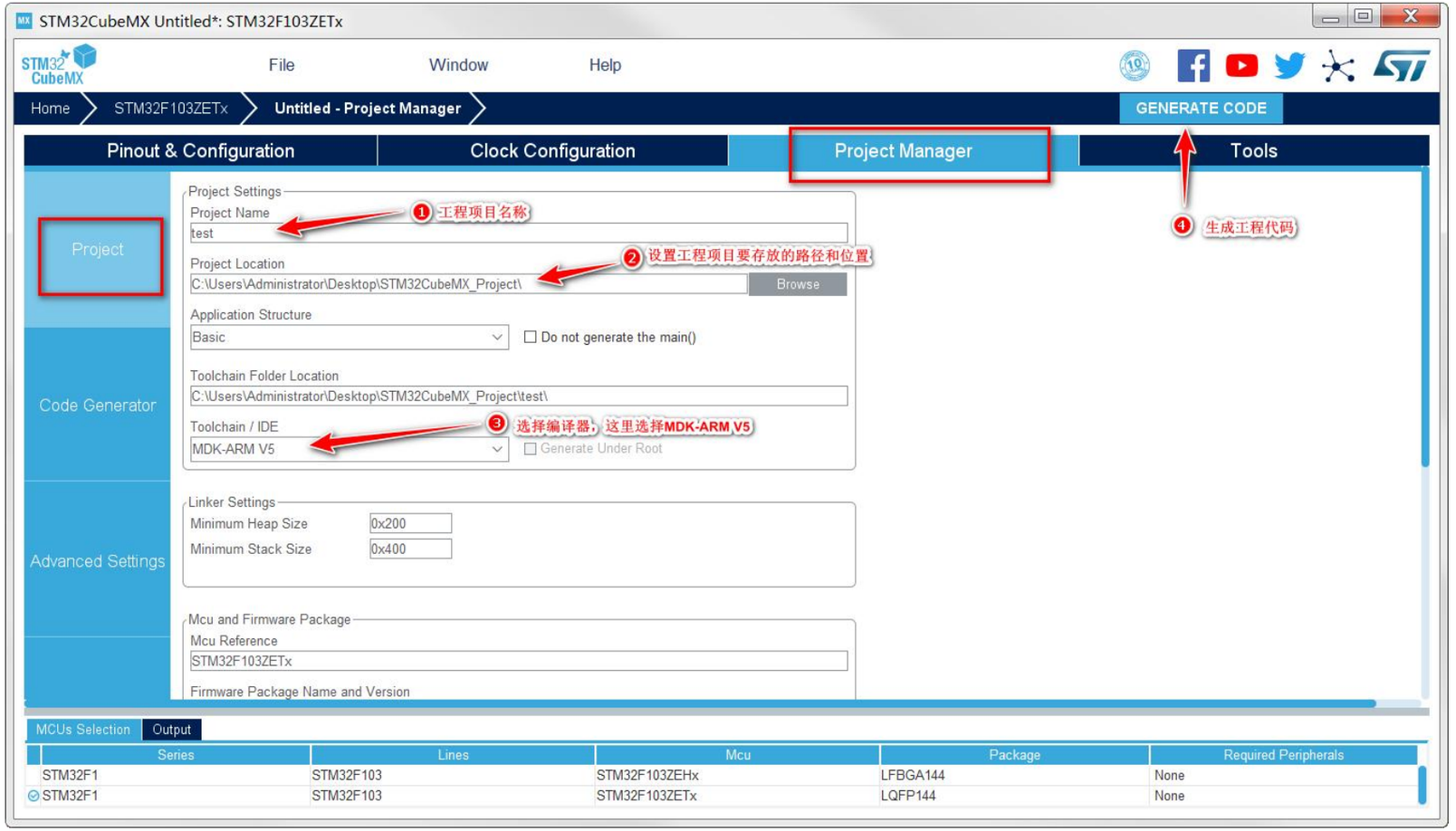
3.5.4 STM32CubeMX生成的工程架构解析

步骤4：配置时钟树



3.5.4 STM32CubeMX生成的工程架构解析

步骤5：设置工程相关参数，如工程名称、工程存放位置

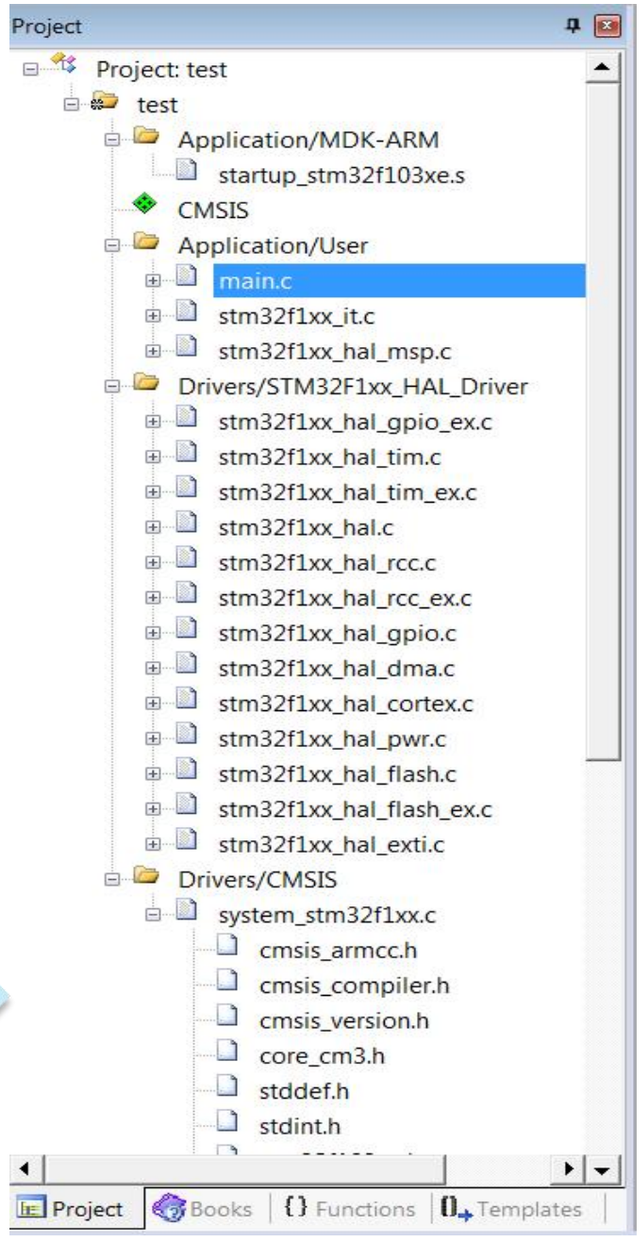


3.5.4 STM32CubeMX生成的工程架构解析

步骤6： 单击“GENERATE CODE”生成工程代码



使用MDK-Keil v5打开工程后，先进行编译。在Keil v5左侧显示Project工程目录架构



3.5.4 STM32CubeMX生成的工程架构解析

STM32CubeMX生成的工程架构分析



四个模块

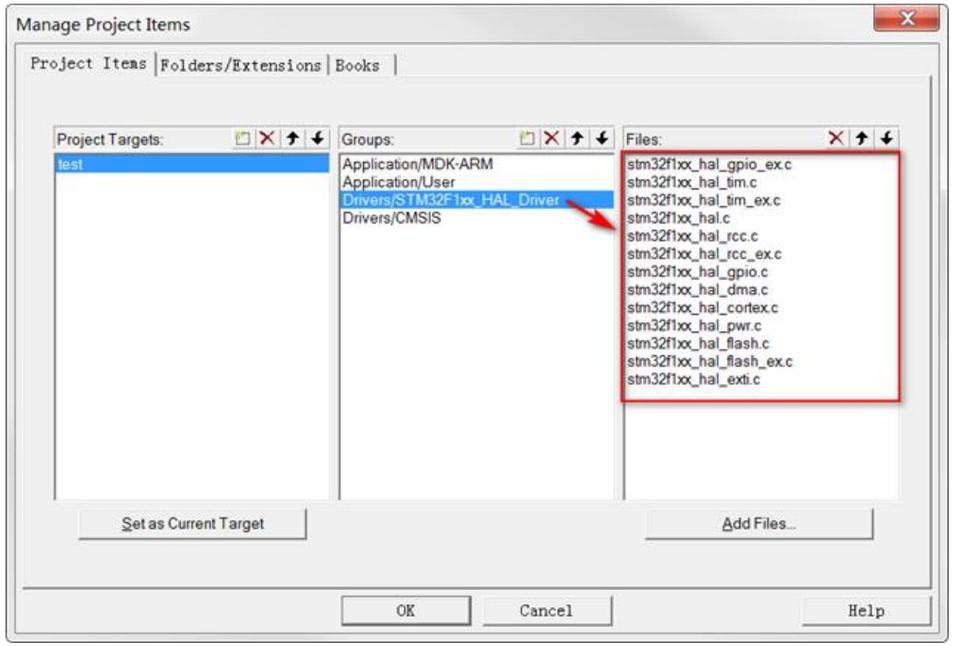
- Application/MDK-ARM
- Drivers/STM32F1xx_HAL_Driver
- Drivers/CMSIS
- Application/User

3.5.4 STM32CubeMX生成的工程架构解析

Drivers文件夹

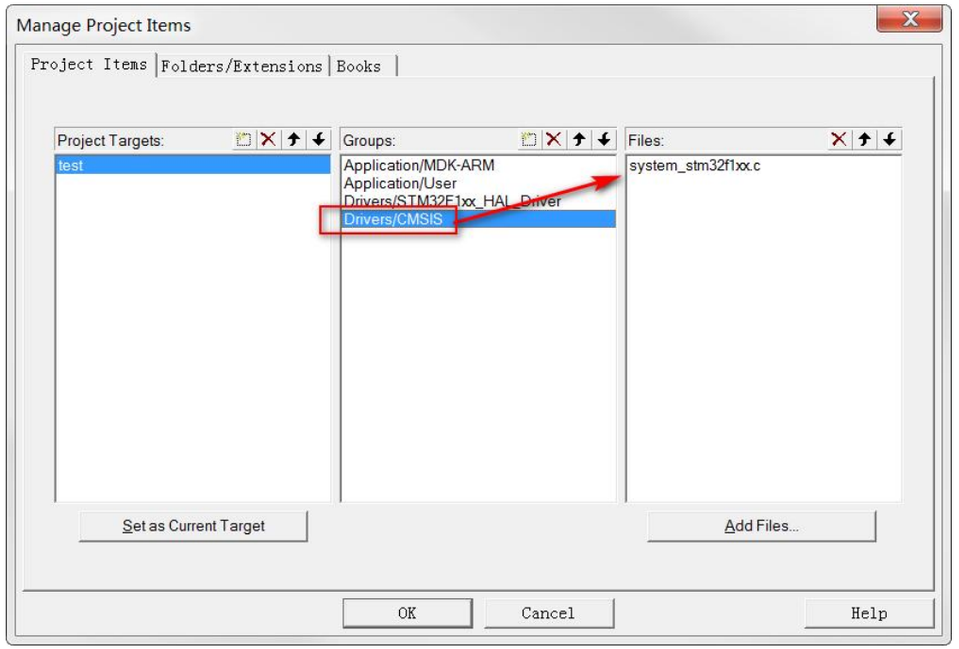
STM32F1xx_HAL_Driver

存放的是STM32F1系列HAL库的头文件和源文件



CMSIS

放置的是ARM提供的芯片内核相关的文件，还有一个Device用于存放ST的STM32F1xx的系统文件



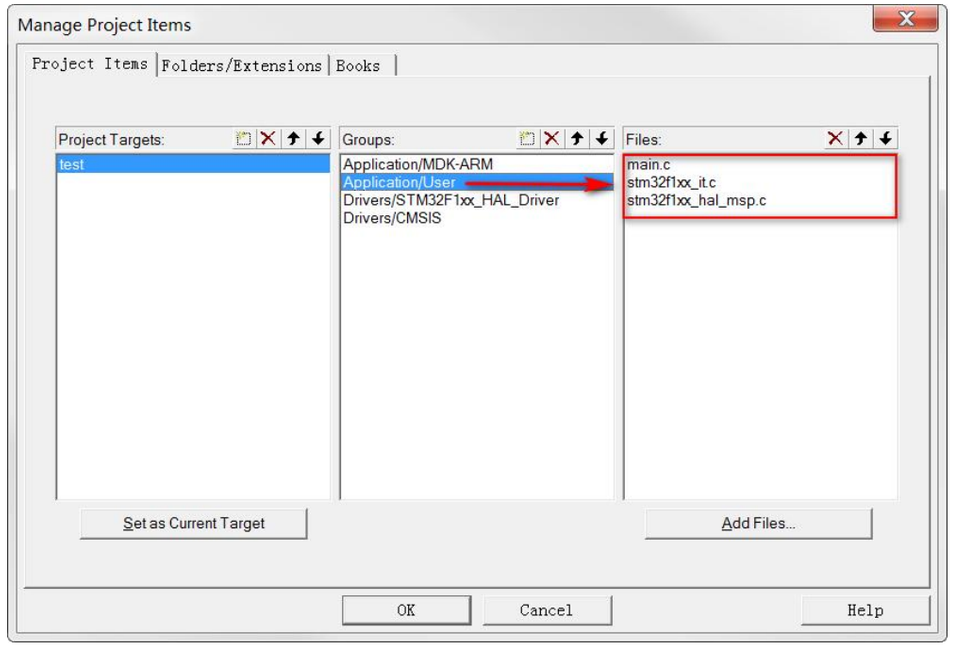
3.5.4 STM32CubeMX生成的工程架构解析

Inc和Src存放工程所用到的头文件、源文件



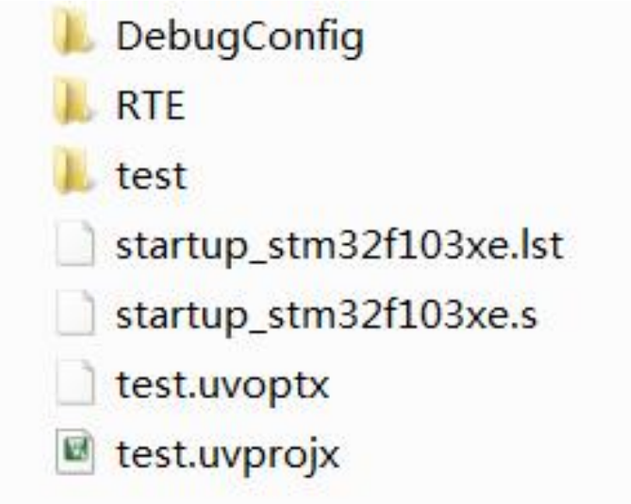
- stm32f1xx_hal_conf.h头文件为STM32的HAL库的工程模块配置文件，内部包含了STM32的所有模块，项目开发时注释掉工程不用的功能模块，在编译时就不会产生相应代码
- stm32f1xx_it.c和stm32f1xx_it.h是中断处理的相关文件

Src文件夹对应的ARM-Keil v5中的Application/User工程组



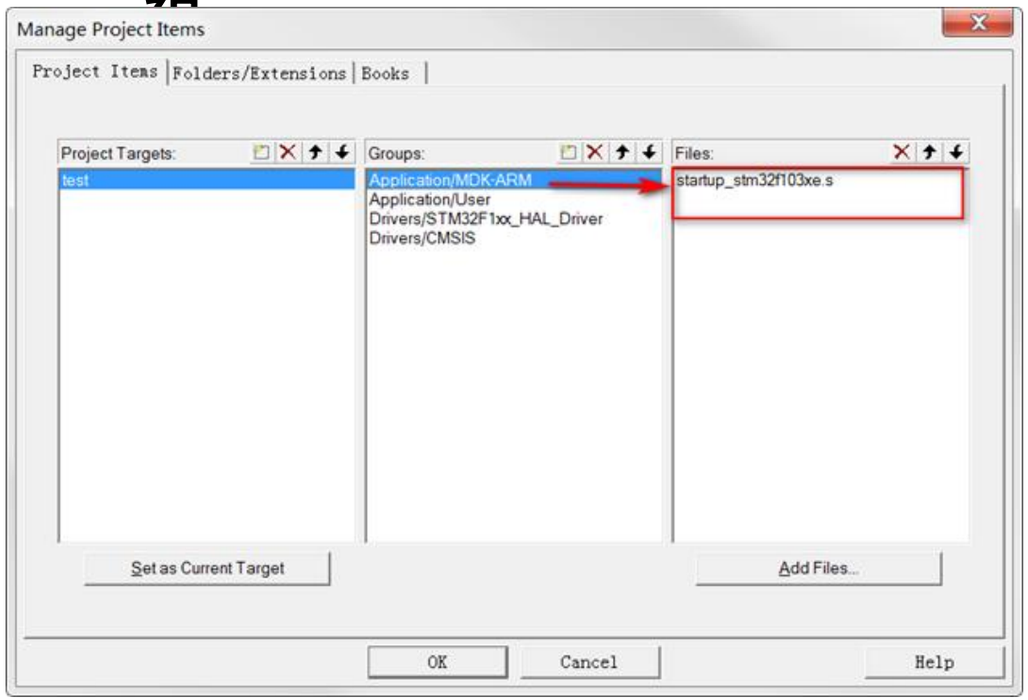
3.5.4 STM32CubeMX生成的工程架构解析

MDK-ARM是编译器ARM-Keil v5所产生的文件



test文件夹下是keil v5在编译过程中产生的中间文件，如后缀为.o和.d的文件

MDK-ARM文件夹对应的ARM-Keil v5中的Application/MDK-ARM工程组



本章小结

3.1

STM32介绍

3.2

STM32微控制器开发模式

- 寄存器开发模式
- 标准外设库开发模式
- HAL库开发模式

3.3

开发平台的搭建

- 嵌入式开发环境
- 集成开发环境Keil MDK
- 下载和安装packs包

3.4

基于标准外设库开发平台搭建

- STM32标准外设库文件结构
- 基于标准外设库新建工程模板

3.5

基于HAL库开发平台的搭建

- JAVA运行环境JRE安装
- 安装STM32CubeMX软件
- STM32CubeF1库文件结构
- STM32CubeMX生成的工程架构解析



The End!

